

Model order reduction for a transient problem using the Laplace transform

Keyvan Attarian^{1,3} and Amine Ammar^{2,3}

¹Ecole polytechnique, Institut Polytechnique de Paris

²LAMPA Laboratory & ESI Group Chair, Arts et Metiers Institute of Technology

³DesCartes Program, WP8, CNRS@CREATE

Printed July 23, 2026

Abstract

Deep-learning reduced-order models (DL-ROMs) keep one latent code per time step, so their output size and cost scale with the number of time steps N_t . We compress the time axis itself with the Laplace transform, predicting $K \ll N_t$ complex spectral coefficients in a single shot and recovering the trajectory by a differentiable regularized inversion, applied either in the physical domain (SLAE) or in the latent space (LLAE). A bias–amplification bound reduces the inversion to two scalars depending on the triplet (s_k, α_t, λ) alone, making the contour selectable offline without training any network. On a parametric transient combustion problem ($N_t = 150$), LLAE reaches 3.84% median relative L^2 error and SLAE 5.04% with an optimised contour, against 8.92% for a linear POD surrogate: LLAE matches a Laplace-free DL-ROM ablation to within 0.04 pp, so the temporal compression costs nothing in accuracy. What it buys is cost and flexibility, unevenly: LLAE shrinks the surrogate alone, still decoding N_t frames, whereas SLAE, whose decoder sees only the K frequency frames, runs $2.0\times$ faster on $3.9\times$ less memory at identical 150-step output. Both serve any resolution above a sampling floor set by K from one checkpoint, where the DL-ROM needs one training per grid. Optimising the inversion gains up to 1.85 pp on the deployed error at no inference cost while **degrading** the reconstruction floor by up to 7.12 pp: an inversion must be scored on the error that ships, not on the floor it permits.

Contents

1	Introduction	2
2	Methodology	4
3	Results	16
4	Comparison and Discussion	37
5	Conclusion	47
	Appendices	49

1 Introduction

High-fidelity simulations of parametric transient partial differential equations (PDEs), such as those governing combustion, reactive transport, and atmospheric dispersion, rely on computationally expensive numerical solvers. In many-query contexts like optimization, uncertainty quantification, real-time control, and digital twins, the complete spatiotemporal field $U(x, t; \mu)$ must be evaluated for thousands of parameter values μ [7, 22]. The prohibitive cost of these repeated evaluations motivates the development of surrogate models capable of reproducing the full dynamics in milliseconds. Applications such as air quality and pollution forecasting concretely illustrate this need: global physical models are accurate but unusable in real-time without approximation [11, 4].

Classical model order reduction (MOR) methods [13], such as Reduced Basis (RB) and Proper Orthogonal Decomposition (POD) combined with interpolation, construct a low-dimensional linear subspace by projecting high-fidelity solutions onto spatial modes. These approaches are theoretically well-founded and highly effective for smooth dynamics. However, they suffer from a **reconstruction floor** linked to the linearity of the basis: for problems with sharp transients, localized structures, or strongly non-linear dynamics (such as convection-diffusion-reaction equations), the number of required modes explodes. The transition toward **non-linear manifolds** becomes necessary as linear techniques struggle [5].

Beyond the spatial compression bottleneck, classical MOR methods also struggle with the temporal dimension. The treatment of time is either sequential (time-stepping on the projected ROM) or space-time (variational), both of which impose a computational cost that grows with the temporal horizon [8, 17, 26]. This dual bottleneck, spatial and temporal, motivates the approach developed in this paper. The **parameter-to-field** setting, predicting a full spatiotemporal field from a low-dimensional parameter vector $\mu \in \mathbb{R}^p$, is addressed by **DL-ROMs** (Deep Learning Reduced-Order Models) [18, 5, 6]: a convolutional autoencoder compresses the spatial field into a compact latent code, a surrogate network learns $\mu \mapsto z$, and the decoder reconstructs the full field at inference, yielding significant gains over linear ROMs. This differs from **field-to-field** operator learning (FNO [19], DeepONet [20, 21]), which takes an input function rather than a scalar parameter vector; the two settings are not directly comparable. Parameter-conditioned U-Nets [15] are a lighter DL-ROM variant for 2D problems, operating directly in the time domain. All existing DL-ROM approaches, however, share a temporal bottleneck: they retain the full temporal resolution of the trajectory, one latent vector per time step. Whether the surrogate takes time as an input coordinate and is queried once per step, $g(\mu, t) \mapsto z(t)$ [5], or propagates the latent sequentially, often autoregressively [25] at the cost of error accumulation, the temporal work, that is the number of network evaluations or the output size, scales with N_t . For long transients this makes the learning problem harder and inference heavier, however compact the spatial latent.

A natural path to address this bottleneck is to work in the **frequency domain**: compressing the time axis into $K \ll N_t$ spectral coefficients reduces the surrogate output dimension dramatically. The spectral intuition is well-established in the neural operator literature, FNO [19] parameterizing its kernel in Fourier space, but the Fourier transform is poorly suited for transient signals: it assumes periodicity and exhibits spectral leakage for non-periodic dynamics. The **Laplace transform** $s = \gamma + i\omega$ generalizes Fourier by adding a real decay axis γ that naturally encodes transients, exponential stability, and non-periodic regimes. Each Laplace frequency field $\hat{U}(s_k)$ is a static spatial field,

structurally simpler to compress than a full temporal sequence. The Laplace Neural Operator (LNO) [1] confirms this advantage in the field-to-field setting. On the linear reduction side, Reyes [28] and Goeckner et al. [9] show that Laplace-domain POD bases are exponentially accurate for parametric evolution problems, yet these approaches remain fundamentally linear. The remaining gap is to combine the Laplace spectral compression with the non-linear expressivity of autoencoders within the DL-ROM framework, enabling **end-to-end** differentiable training thanks to the exact invertibility of the transform [10].

This paper fills that gap. We situate our work in the DL-ROM family and introduce the Laplace transform as a **temporal** compression stage on top of the standard AE-surrogate pipeline: instead of predicting N_t latent frames, the surrogate predicts $K \ll N_t$ spectral coefficients, from which the full trajectory is recovered by inversion. Here $U(t) \in \mathbb{R}^{N_x}$ is the discretized spatial field at time step t , N_x the number of spatial nodes, N_t the number of time steps, and K the number of Laplace frequencies. Two instantiations are studied, differing in where the transform is applied: the physical domain (SLAE) or the latent space (LLAE). Both are compared against two baselines that each remove one ingredient, a linear POD baseline in the Laplace domain and a Laplace-free DL-ROM keeping the full temporal grid. Our contributions are as follows:

- **Temporal compression via the Laplace transform (main contribution):** we compress the temporal dimension of the surrogate target from N_t frames to $K \ll N_t$ spectral coefficients, making the surrogate’s size and output dimension independent of the temporal horizon. Whether the **complete** inference cost inherits that independence depends on where the transform is applied, and we quantify this in Section 4.2: it does for SLAE, whose decoder sees only the K frequency frames, and does not for LLAE, whose decoder still emits N_t physical frames. Independently of that, since the trajectory is materialised by a regularized linear solve assembled on whatever time nodes are requested, the predicted coefficients refer to no time grid: a single trained model answers at **any** output resolution above a sampling floor set by K (Section 4.3), whereas a model predicting one latent per time step is tied to its training grid, or at best pays one network evaluation per requested step. We state the trade once, here: temporal compression is an argument about cost and about what a trained model can afterwards be asked for, not about accuracy.
- **SLAE vs. LLAE:** we systematically compare the two instantiations and characterize the accuracy/cost trade-off they offer.
- **Designing the regularized inversion:** we treat the inversion as an object to be optimised rather than fixed by convention, and by that we mean the triplet (s_k, α_t, λ) , the poles together with the two regularisation weights, not the contour alone. For LLAE the triplet is learned end-to-end; for SLAE we derive a bias–amplification decomposition of the reconstruction error and optimise it offline, before any training, by minimizing a proxy loss that trades regularization bias against the amplification of the surrogate approximation error.

The remainder of this paper is organized as follows. Section 2 describes the methodology, detailing the Laplace transform, the two autoencoder families, the surrogate architectures and the two baselines. Section 3 presents the numerical results. Section 4 compares the pipelines on accuracy against cost, discusses the limitations of the study and outlines perspectives, and Section 5 concludes. The appendices detail the governing equations, the architectural hyperparameters and the derivations behind the contour optimisation.

2 Methodology

2.1 Laplace Transform and Inversion

Throughout this section, $u(t) \in \mathbb{R}$ denotes the field value at a single fixed spatial location (lowercase = one pixel); for the full spatial field $U(t) \in \mathbb{R}^{N_x}$ (uppercase = full grid of N_x nodes), all operations below are applied independently at each grid point. The forward Laplace transform is evaluated at discrete frequencies $s = \gamma + i\omega$ using a temporal quadrature rule (e.g., trapezoidal rule with weights w_n):

$$(1) \quad \hat{u}(s) = \int_0^T u(t) e^{-st} dt \approx \Delta t \sum_{n=0}^{N_t-1} w_n u(t_n) e^{-st_n}$$

Since $u(t)$ is strictly real-valued, its Laplace transform satisfies the conjugate symmetry property $\hat{u}(\bar{s}) = \overline{\hat{u}(s)}$. Consequently, we restrict our numerical evaluation to frequencies with non-negative imaginary parts $\omega_k \geq 0$, which halves the forward evaluation cost while providing the full spectral information required for inversion.

Regularized Inversion (Tikhonov). The inversion is posed not on the K stored frequencies but on their **conjugate-symmetric completion**

$$(2) \quad \mathcal{S} = \{s_k\}_{k=1}^K \cup \{\bar{s}_k : \omega_k > 0\}, \quad K_{\text{full}} = |\mathcal{S}| = 2K - \#\{k : \omega_k = 0\},$$

whose additional members follow from the stored half-spectrum through $\hat{u}(\bar{s}_k) = \overline{\hat{u}(s_k)}$. This doubles the sampling points for free: the conjugate coefficients require no additional forward evaluation, surrogate output or storage, yet enter the least-squares problem below as genuine extra constraints, so that the inversion is posed on $K_{\text{full}} \simeq 2K$ points while the network only ever predicts K . Let $\mathbf{F} \in \mathbb{C}^{K_{\text{full}} \times N_t}$ be the forward evaluation matrix assembled on $\mathcal{S}$, with $\mathbf{F}_{k,n} = \Delta t w_n e^{-s_k t_n}$, and let $\hat{\mathbf{u}} \in \mathbb{C}^{K_{\text{full}}}$, $\mathbf{u} \in \mathbb{R}^{N_t}$ be the corresponding vectors of Laplace coefficients and temporal values. Throughout this paper $\mathbf{F}$ and $\hat{\mathbf{u}}$ always denote these conjugate-extended objects. Taking advantage of the real-valued constraint on $\mathbf{u}$, the reconstruction is formulated as a regularized complex least-squares problem:

$$(3) \quad \mathbf{u}^* = \arg \min_{\mathbf{u} \in \mathbb{R}^{N_t}} \|\mathbf{F}\mathbf{u} - \hat{\mathbf{u}}\|_{\mathbb{C}^{K_{\text{full}}}}^2 + \alpha_t \|\mathbf{D}_t \mathbf{u}\|_2^2 + \lambda \|\mathbf{u}\|_2^2$$

where $\mathbf{D}_t \in \mathbb{R}^{(N_t-1) \times N_t}$ is the first-difference operator penalizing temporal roughness, and $\lambda > 0$ is a Tikhonov ridge regularization parameter. The corresponding optimality condition (normal equations) yields a real symmetric linear system:

$$(4) \quad \left(\Re(\mathbf{F}^H \mathbf{F}) + \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I} \right) \mathbf{u}^* = \Re(\mathbf{F}^H \hat{\mathbf{u}})$$

Two properties of $\mathcal{S}$ are worth making explicit, since the half-spectrum $\{s_k\}_{k=1}^K$ alone would give a different problem. First, a real $\mathbf{u}$ produces conjugate residuals at s_k and $\bar{s}_k$, so restoring the conjugates weights every frequency with $\omega_k > 0$ twice in (3) relative to a purely real pole, the discrete counterpart of integrating along the whole Bromwich contour rather than over $\omega \geq 0$ only. Second, on $\mathcal{S}$ both $\mathbf{F}^H \mathbf{F}$ and $\mathbf{F}^H \hat{\mathbf{u}}$ are real, their imaginary parts cancelling pairwise between conjugate members, so the operator $\Re(\cdot)$ in (4) records the real-valuedness of $\mathbf{u}$ without discarding information. The $N_t \times N_t$ system (4) is assembled and factorised in double precision (at $K = 16$ the columns of $\mathbf{F}$ are

nearly collinear and $\kappa(A)$ exceeds the reach of single precision) by Cholesky when A is reused across a batch of nodes and by a dense LU solve otherwise, at negligible cost. It is fully differentiable in (s_k, α_t, λ) , a property Section 2.4 exploits, and holds for any set of frequencies (s_k) , with no constraint on their distribution in the complex plane.

Choice of contour: truncated Bromwich. A key design choice is the set of evaluation frequencies $(s_1, \dots, s_K)$. We adopt throughout the **truncated Bromwich contour**: a vertical line in the complex plane at $\text{Re}(s) = \gamma$, with imaginary parts equal to the first K discrete Fourier frequencies,

$$(5) \quad s_k = \gamma + i \frac{2\pi k}{N_t \Delta t}, \quad k = 0, \dots, K-1,$$

where $\gamma \geq 0$ is the **damping parameter** and $K \leq \lfloor N_t/2 \rfloor + 1$ controls the spectral truncation. We allow $\gamma = 0$, a purely imaginary contour, i.e. a truncated DFT, since it is one of the settings we report, and the optimisation of Section 2.4 may even return slightly negative abscissae; over a finite horizon the quadrature (1) is a finite sum and remains well defined for any $s \in \mathbb{C}$. With this choice, the forward sum (1) coincides with a truncated DFT of the exponentially damped sequence:

$$(6) \quad \hat{u}(s_k) = \text{DFT}\{u(t_n) \Delta t w_n e^{-\gamma t_n}\}_k, \quad k = 0, \dots, K-1,$$

and could therefore be evaluated by an FFT; with $K \leq 32$ and $N_t = 150$, our implementation simply forms the dense $K \times N_t$ matrix product, whose $O(KN_t)$ cost per node is negligible against the convolutional stages. Since $K < \lfloor N_t/2 \rfloor + 1$, the spectrum is incomplete and we rely on the Tikhonov inversion (3)–(4) in all cases. The Bromwich structure, however, forces all poles onto a common real part γ and a regular frequency grid, a strong constraint that may be suboptimal. Since the inversion accepts any set of frequencies, the poles can in principle be placed freely in the complex plane; this is the motivation for Section 2.4.

2.2 Autoencoders in the Laplace Domain

We investigate two autoencoder architectures that differ solely in **where** the Laplace transform is applied, as illustrated in Figure 1. In the **Spatial Laplace AE (SLAE)**, the transform is applied first in the physical domain, and a frequency-conditioned autoencoder then compresses each of the K resulting complex frequency frames independently. In the **Latent Laplace AE (LLAE)**, the order is reversed: a time-conditioned autoencoder first encodes the full temporal sequence into a compact latent trajectory, and the Laplace transform is subsequently applied within that low-dimensional space.

In both cases, an L_2 ridge penalty on the latent representations enforces a sufficiently regular structure, which makes the latent space easier for the surrogate to navigate (Section 2.3).

Both architectures build on the same convolutional encoder–decoder backbone, applied directly to the spatial field with no POD pre-reduction; this is the standard deep convolutional-autoencoder compression of the DL-ROM lineage [18, 5], as opposed to the POD-then-autoencoder pre-reduction of POD-DL-ROM [6]. On top of this shared backbone, both architectures use the same conditioning mechanism: a sinusoidal coordinate encoding combined with Feature-wise Linear Modulation (FiLM) [27]. The coordinate $\tau \in [0, 1]$ represents either the frequency ratio $f_k = k/(K-1)$ for SLAE, or the normalised time step t/T for LLAE.

Sinusoidal Coordinate Encoding. The coordinate $\tau \in [0, 1]$ is embedded via sinusoidal basis functions inspired by NeRF positional encoding [23]:

$$(7) \quad \mathbf{e}(\tau) = \text{MLP}\left(\left[\sin(2^i \pi \tau), \cos(2^i \pi \tau)\right]_{i=0}^{L-1}\right), \quad L = 8.$$

A plain scalar τ would conflate nearby coordinates and fail to capture sharp transitions in either the frequency or time domains. The multi-scale sinusoidal basis explicitly represents τ at L different resolutions, providing the network with a rich, non-saturating signal that allows fine discrimination across the entire range.

FiLM Conditioning. To share encoder/decoder weights across all coordinates (frequencies or time steps) while adapting to each specific configuration, the FiLM mechanism is employed. At each convolutional layer ℓ , the coordinate embedding $\mathbf{e}(\tau)$ is projected to a channel-wise gain $\alpha_\ell \in \mathbb{R}^{C_\ell}$ and bias $\delta_\ell \in \mathbb{R}^{C_\ell}$, modulating the activations element-wise:

$$(8) \quad \mathbf{x}_\ell \leftarrow (\mathbf{x}_\ell \odot (1 + \alpha_\ell(\mathbf{e}(\tau)))) + \delta_\ell(\mathbf{e}(\tau)).$$

The residual formulation $(1 + \alpha_\ell)$ initializes the modulation close to the identity, which stabilizes early training. All coordinates share the same convolutional weights, and the FiLM parameters act as a lightweight per-coordinate adapter at each layer, enabling a single autoencoder to process the full spectrum or temporal sequence without training separate model copies.

2.2.1 Spatial Laplace AE (SLAE)

SLAE first applies the Laplace transform at the pixel level, converting the N_t -frame temporal sequence $U(t)$ into K complex-valued spatial fields $\hat{U}(s_k)$. Each frequency frame, represented as a two-channel (real/imaginary) image, is independently compressed by a FiLM-conditioned convolutional encoder into a real latent vector $z(s_k) \in \mathbb{R}^{d_z}$:

$$(9) \quad U(t) \xrightarrow{\mathcal{L}_{\text{pixel-wise}}} \hat{U}(s_k) \xrightarrow{\text{Encoder}(f_k)} z(s_k) \xrightarrow{\text{Decoder}(f_k)} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

The decoder reconstructs $\hat{U}(s_k)$ from $z(s_k)$ via separate output heads for the real and imaginary components. Training minimizes the Laplace-domain reconstruction error together with an L_2 regularization term:

$$(10) \quad \mathcal{L}_{\text{SLAE}} = \|\tilde{U}(s_k) - \hat{U}(s_k)\|_{\mathbb{C}^{N_x}}^2 + \beta \|z(s_k)\|_2^2$$

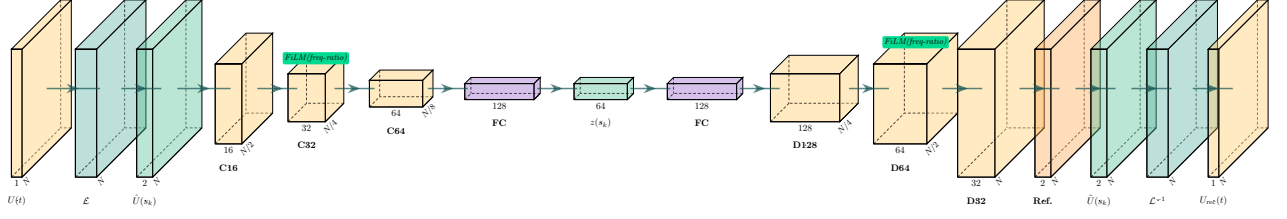
where $\|\cdot\|_{\mathbb{C}^{N_x}}^2$ is the squared L_2 norm over the N_x spatial nodes (real and imaginary parts combined), and β is a regularization hyperparameter.

2.2.2 Latent Laplace AE (LLAE)

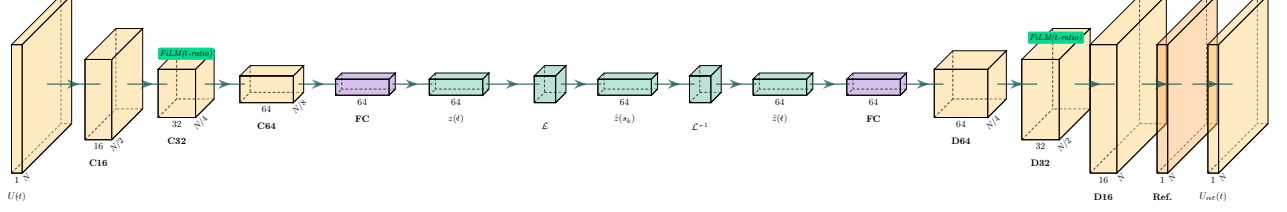
LLAE encodes the physical frames $U(t)$ into a latent trajectory $z(t) \in \mathbb{R}^{d_z}$ using a time-conditioned convolutional encoder (conditioning on the normalised ratio t/T), then applies the Laplace transform directly within this compact latent space:

$$(11) \quad U(t) \xrightarrow{\text{Encoder}(t)} z(t) \xrightarrow{\mathcal{L}} \hat{z}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}(t) \xrightarrow{\text{Decoder}(t)} U_{\text{rec}}(t)$$

Compressing in the latent space rather than the physical domain offers two advantages: the transform operates on d_z -dimensional vectors instead of N_x -dimensional fields, and



(a) SLAE: pixel-wise Laplace transform, then per-frequency convolutional AE ($d_z = 64$, $K = 16$).



(b) LLAE: convolutional AE first, then Laplace transform in latent space ($d_z = 64$, $K = 16$).

Figure 1: SLAE (a) and LLAE (b) autoencoder pipelines.

the encoder produces a smoother temporal representation than the raw complex frequency frames, easing the spatial compression task. The main counterpart is computational: at each training step and at inference, the encoder and decoder must process all N_t physical frames, whereas SLAE processes only the K frequency frames. Since $N_t \gg K$, LLAE is significantly heavier, which limits the network capacity that can be deployed under a fixed compute budget. The loss combines three terms:

$$\begin{aligned}
 \mathcal{L}_{\text{LLAE}} = & \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|U_{\text{rec}}(t_n) - U(t_n)\|_2^2 \\
 & + \beta_{\text{latent}} \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|\tilde{z}(t_n) - z(t_n)\|_2^2 \\
 & + \beta \frac{1}{K} \sum_{k=0}^{K-1} \|\hat{z}(s_k)\|_{\mathbb{C}^{d_z}}^2
 \end{aligned}
 \tag{12}$$

which combines the physical-time reconstruction error, a latent consistency term penalizing the round-trip error $\tilde{z}(t_n) - z(t_n)$, and an L_2 regularization penalty on the Laplace-domain latent codes $\hat{z}(s_k)$. The latent consistency term is unique to LLAE: it explicitly enforces the strict invertibility of the continuous Laplace transform within the learned discrete latent space, a structural constraint not required by SLAE since it encodes each frequency frame independently.

2.3 Surrogate Training

Following the DL-ROM paradigm of [5, 6], the surrogate MLP g_θ (shared trunk + per-frequency heads; architecture detailed in Figure 2 and Appendix G) replaces the encoding steps by learning the direct mapping $\mu \mapsto w_k$, where w_k denotes the Laplace-domain latent at frequency s_k that feeds the decoder: $w_k = z(s_k) \in \mathbb{R}^{d_z}$ for SLAE, and $w_k = \hat{z}(s_k) \in \mathbb{C}^{d_z}$ for LLAE. Two strategies for constructing the surrogate target w_k are described below.

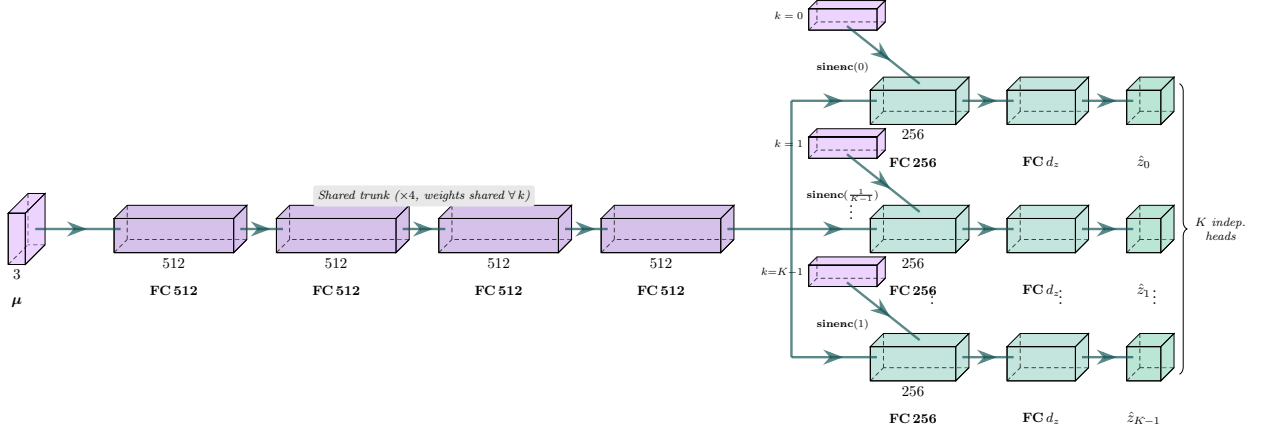


Figure 2: **Surrogate MLP architecture.** A shared trunk maps physical parameters μ to a latent representation, which is broadcast to K independent heads. Each head is conditioned on a frequency index via sinusoidal encoding and predicts the target latent w_k for that frequency.

2.3.1 Direct Embedding Prediction

In the direct approach, the surrogate predicts the full latent vector at each index without any preliminary compression. For SLAE, the encoder produces frequency-domain latents $z(s_k) \in \mathbb{R}^{d_z}$ directly, and the pipeline is:

$$(13) \quad \mu \xrightarrow{g_\theta} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

For LLAE, the time-domain latent sequence $z(t) \in \mathbb{R}^{d_z}$ is first transformed to the Laplace domain, and the surrogate predicts those Laplace-domain codes directly:

$$(14) \quad \mu \xrightarrow{g_\theta} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t)$$

2.3.2 Latent Sequence SVD Compression

To reduce the effective output dimension of the surrogate and simplify its learning task, one can apply a preliminary offline SVD compression to the Laplace-domain latents before training the surrogate, an approach inspired by [24]. For both SLAE and LLAE, the SVD is applied in the frequency domain: given the K Laplace-domain latent codes $\{\hat{z}(s_k)\}_{k=1}^K$ collected over the training set, a truncated SVD yields a projection matrix $V \in \mathbb{R}^{d_z \times k_{\text{svd}}}$ (with $k_{\text{svd}} \ll d_z$) such that $\hat{z}(s_k) \approx (\hat{z}(s_k)V)V^\top$. The reduced codes $\hat{G}(s_k) = \hat{z}(s_k)V \in \mathbb{R}^{k_{\text{svd}}}$ become the surrogate targets, reducing the prediction problem from dimension d_z to k_{svd} .

In the **SLAE-SVD** variant (latents indexed by frequency k): the projection yields $G(s_k) = z(s_k)V$, and the surrogate predicts in the compressed frequency-domain space:

$$(15) \quad \mu \xrightarrow{g_\theta} G_{\text{pred}}(s_k) \xrightarrow{V^\top} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

In the **LLAE-SVD** variant, the SVD is likewise applied in the Laplace domain: the projection yields $\hat{G}(s_k) = \hat{z}(s_k)V$, and the surrogate predicts the compressed frequency-domain latents before back-projection and inversion:

$$(16) \quad \mu \xrightarrow{g_\theta} \hat{G}_{\text{pred}}(s_k) \xrightarrow{V^\top} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t)$$

Because V is real and acts on the latent channel index while $\mathcal{L}^{-1}$ acts on the frequency index, the two commute: back-projecting before or after the inversion is equivalent, and the implementation applies whichever order is cheaper.

In both strategies, the entire pipeline is trained end-to-end to minimise the total physical-time loss:

$$(17) \quad \mathcal{L}_{\text{total}} = \alpha_{\text{lat}} \|w_k^{\text{pred}} - w_k^{\text{true}}\|^2 + \alpha_{\text{spat}} \|U_{\text{rec}} - U\|^2$$

where w_k^{pred} and w_k^{true} denote the predicted and true Laplace-domain latents (either $z(s_k)$ or $\hat{G}(s_k)$ depending on the chosen strategy). End-to-end backpropagation allows gradients from the spatiotemporal MSE to flow through the decoder and the inverse transform, jointly updating the surrogate weights, the AE decoder, and, in the SVD variant, the projection basis V .

2.3.3 Tucker Decomposition in Latent Space

Just like the SVD approach described in the previous section, the goal here is to further compress the latent representations of both LLAE and SLAE before training the surrogate MLP, which leads to two distinct variants: **LLAE-Tucker** and **SLAE-Tucker**. While the SVD approach reduces the latent sequence along a single mode at a time (for LLAE-SVD it collapses the latent dimension d_z ; for SLAE-SVD it collapses d_z independently at each frequency, ignoring cross-frequency correlations), Tucker decomposition [16] generalises this by simultaneously factorising two modes of the latent tensor.

Crucially, for both variants, the Tucker decomposition is applied in the latent space **after** the Laplace transform. We assemble the frequency-domain latents of the training set into a three-way tensor $\hat{\mathcal{Z}} \in \mathbb{C}^{N_{\text{train}} \times K \times d_z}$ (real-valued for SLAE, complex-valued for LLAE). A Tucker- (r_μ, r_s, r_z) decomposition approximates each scalar element of the tensor as a triple sum:

$$(18) \quad \hat{\mathcal{Z}}_{i,j,m} \approx \sum_{p=1}^{r_\mu} \sum_{l=1}^{r_s} \sum_{q=1}^{r_z} \hat{C}_{p,l,q} U_{i,p}^{(\mu)} U_{j,l}^{(s)} U_{m,q}^{(z)},$$

where $\hat{C} \in \mathbb{C}^{r_\mu \times r_s \times r_z}$ is the global core tensor, and $U^{(\mu)}, U^{(s)}, U^{(z)}$ are the orthogonal factor matrices for the parameter (sample), frequency, and latent modes, respectively. To enable predictions on unseen test samples, we absorb the parameter-dependent terms into a simulation-specific core matrix $\hat{\mathcal{G}}_i \in \mathbb{C}^{r_s \times r_z}$, whose elements are $(\hat{\mathcal{G}}_i)_{l,q} = \sum_{p=1}^{r_\mu} \hat{C}_{p,l,q} U_{i,p}^{(\mu)}$. This allows us to rewrite the approximation for the full frequency-domain latent matrix $\hat{\mathcal{Z}}_i \in \mathbb{C}^{K \times d_z}$ of the i -th simulation in a compact matrix form:

$$(19) \quad \hat{\mathcal{Z}}_i \approx U^{(s)} \hat{\mathcal{G}}_i U^{(z)\top}, \quad U^{(s)} \in \mathbb{C}^{K \times r_s}, \quad U^{(z)} \in \mathbb{C}^{d_z \times r_z}.$$

The shared orthogonal factors $U^{(s)}$ and $U^{(z)}$ are computed offline by the Higher-Order Orthogonal Iteration (HOOI) algorithm [2] and kept frozen. The surrogate then learns the mapping $\mu \mapsto \hat{\mathcal{G}}(\mu) \in \mathbb{C}^{r_s \times r_z}$, whose dimensionality $r_s \cdot r_z$ is substantially smaller than $K \cdot d_z$ because it jointly exploits cross-frequency and latent correlations.

For LLAE-Tucker, the surrogate predicts the complex-valued core matrix, from which the full Laplace-domain latents are reconstructed and inverted back to the time domain before decoding:

$$(20) \quad \mu \xrightarrow{g_\theta} \hat{\mathcal{G}}_{\text{pred}} \xrightarrow{U^{(s)}, U^{(z)\top}} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t)$$

For SLAE-Tucker, the surrogate predicts the real-valued core matrix, which is expanded to the real-valued frequency latents, decoded to spatial Laplace fields, and finally inverted to the physical time domain:

$$(21) \quad \mu \xrightarrow{g_\theta} \mathcal{G}_{\text{pred}} \xrightarrow{U^{(s)}, U^{(z)\top}} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

Unlike the real basis V of the SVD variant, the frequency factor $U^{(s)}$ mixes the Laplace frequencies and $U^{(z)}$ is complex-valued; neither commutes with $\mathcal{L}^{-1}$, so the back-projection necessarily precedes the inversion. Both variants share the same conceptual advantage: instead of predicting K independent vectors of size d_z , the surrogate predicts a highly compact core matrix that captures the essential spatiotemporal dynamics.

2.4 Designing the Regularized Inversion

The object to be designed is not the contour alone but the **triplet** (s_k, α_t, λ) : the K poles together with the two regularization weights of (3). The reference against which we compare fixes all three by convention, poles on the truncated Bromwich contour and (α_t, λ) set a priori and never tuned. Whether the triplet can be designed instead depends on where the transform is applied.

In LLAE it is applied in the low-dimensional latent space, where the triplet is directly differentiable: the forward quadrature (1) is a finite sum of complex exponentials $e^{-s_k t_n}$ smooth in s_k , and the Tikhonov inversion (4) a linear solve whose matrix $\mathbf{A}$ depends smoothly on s_k , α_t and λ . The triplet can therefore be carried as parameters of the model and optimized end-to-end, which is what our LLAE implementation does: $\sigma_k = \text{Re}(s_k)$ and $\omega_k = \text{Im}(s_k)$ are learned independently for each pole, together with α_t and λ in log-parameterisation, from an initialisation on the truncated Bromwich contour (ω_k at the K first FFT frequencies, $\sigma_k = \gamma_{\text{init}}$ uniform). This case requires no further development, and we report it as the latent counterpart of the procedure below: it yields the most accurate autoencoder of the study without improving the deployed error (Appendix E).

In SLAE the poles enter the transform of the physical field, and one gradient step on them would require recomputing all N_t spatial forward passes, negating the computational advantage of the family. The triplet must therefore be selected before training, hence scored without a trained network to score it against.

We present a theoretical framework for this purpose. The goal is to decompose the total reconstruction error $\mathbf{u}(\mu) - \mathbf{u}_{\text{rec}}(\mu)$ into a **bias** term (introduced by regularization and the finite contour) and a **variance** term (the surrogate error propagated through the inversion), and to jointly optimize the contour and regularization parameters by minimising an upper bound on the expected reconstruction error.

2.4.1 Bounding the Reconstruction Error

Formal setup. Using the notation of Section 2.1, we fix a spatial location and work with scalar (per-pixel) quantities. All Laplace-domain objects are those of (2): coefficient vectors carry the K_{full} entries of the conjugate-symmetric set $\mathcal{S}$, the K predicted ones being extended by conjugation exactly as the exact ones are, and $\mathbf{F} \in \mathbb{C}^{K_{\text{full}} \times N_t}$ is assembled on $\mathcal{S}$. Let $\mathbf{u}(\mu) \in \mathbb{R}^{N_t}$ be the true temporal signal $(u(t_n))_n$ at that location, $\hat{\mathbf{u}}(\mu) \in \mathbb{C}^{K_{\text{full}}}$ its exact Laplace coefficients $(\hat{u}(s_k))_k$, and $\tilde{\mathbf{u}}(\mu) \in \mathbb{C}^{K_{\text{full}}}$ the surrogate-predicted coefficients (the per-pixel counterpart of the full-field $\tilde{U}(s_k)$ of Section 2.2.1). The regularised

reconstructions from the true and predicted coefficients are, respectively,

$$(22) \quad \mathbf{u}^*(\boldsymbol{\mu}) = \mathbf{A}^{-1} \mathfrak{K} \left(\mathbf{F}^H \hat{\mathbf{u}}(\boldsymbol{\mu}) \right), \quad \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) = \mathbf{A}^{-1} \mathfrak{K} \left(\mathbf{F}^H \tilde{\mathbf{u}}(\boldsymbol{\mu}) \right),$$

where $\mathbf{A} = \mathfrak{K}(\mathbf{F}^H \mathbf{F}) + \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I}$ is the normal-equation matrix of (4) and $\mathbf{L}_{\text{reg}} = \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I}$. The unique existence of these solutions is guaranteed since $\lambda > 0$ and $\mathbf{D}_t^T \mathbf{D}_t$ is positive semi-definite, making $\mathbf{L}_{\text{reg}} \succeq \lambda \mathbf{I}$ and hence $\mathbf{A}$ symmetric positive definite (see Appendix H).

Bias–amplification decomposition. We decompose the total reconstruction error by introducing $\mathbf{u}^*(\boldsymbol{\mu})$ and adding and subtracting it:

$$(23) \quad \begin{aligned} \mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) &= (\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu})) + (\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})) \\ &= (\mathbf{u}(\boldsymbol{\mu}) - \mathbf{A}^{-1} \mathfrak{K}(\mathbf{F}^H \mathbf{F}) \mathbf{u}(\boldsymbol{\mu})) + \mathbf{A}^{-1} \mathfrak{K} \left(\mathbf{F}^H (\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu})) \right). \end{aligned}$$

Using $\mathbf{A} = \mathfrak{K}(\mathbf{F}^H \mathbf{F}) + \mathbf{L}_{\text{reg}}$, the bias term simplifies to:

$$(24) \quad \mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu}) = \mathbf{A}^{-1} \left(\mathbf{A} - \mathfrak{K}(\mathbf{F}^H \mathbf{F}) \right) \mathbf{u}(\boldsymbol{\mu}) = \mathbf{A}^{-1} \mathbf{L}_{\text{reg}} \mathbf{u}(\boldsymbol{\mu}).$$

Taking norms in (23), applying the triangle inequality, and using $\|\mathfrak{K}(\mathbf{F}^H \cdot)\| \leq \|\mathbf{F}^H\| \|\cdot\|$ yields:

$$(25) \quad \|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\| \leq \underbrace{\|\mathbf{A}^{-1} \mathbf{L}_{\text{reg}} \mathbf{u}(\boldsymbol{\mu})\|}_{\text{regularization bias}} + \underbrace{\|\mathbf{A}^{-1} \mathbf{F}^H\|}_{\text{amplification factor}} \cdot \underbrace{\|\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu})\|_{\mathbb{C}^{K_{\text{full}}}}}_{\text{surrogate error}},$$

valid for any $\boldsymbol{\mu}$; the derivation, an elementary triangle inequality followed by sub-multiplicativity, is given in Appendix H. We state it for the structure it exposes: the two coefficients multiplying 1 and the coefficient error depend on (s_k, α_t, λ) and on nothing else. This is what makes a triplet scorable without training a network, and it is the whole basis of the offline spatial procedure below.

The regularization bias depends on the true field $\mathbf{u}(\boldsymbol{\mu})$ and on $\mathbf{L}_{\text{reg}}$, and is mitigated by the well-posedness of $\mathbf{A}^{-1}$; any surrogate error $\hat{\mathbf{u}} - \tilde{\mathbf{u}}$ is magnified during reconstruction by the amplification factor $\|\mathbf{A}^{-1} \mathbf{F}^H\|$, a condition number for the inversion. In (25) and everything derived from it, $\|\cdot\|$ applied to a matrix is the operator 2-norm; the implementation evaluates instead the Frobenius norm $\|\mathbf{A}^{-1} \mathbf{F}^H\|_F \geq \|\mathbf{A}^{-1} \mathbf{F}^H\|_2$, which is cheaper to differentiate and preserves the inequality.

2.4.2 The Offline Selection Criterion

Optimisation problem. The matrices $\mathbf{F}$ and $\mathbf{L}_{\text{reg}}$, and consequently $\mathbf{A}$, depend explicitly on the contour $(s_1, \dots, s_K)$, on λ , and on α_t . We aim to jointly optimize these parameters to minimise the expected mean reconstruction error $\mathbb{E}_{\boldsymbol{\mu}} [\|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\|]$. Since evaluating this true error directly is challenging without ground truth for every candidate contour, we treat the hyperparameters as learnable parameters and optimize a proxy loss $\mathcal{L}^{\text{proxy}}$ derived directly from the bound (25):

$$(26) \quad \mathcal{L}^{\text{proxy}} = \mathcal{L}_{\text{bias}} + \mathcal{L}_{\text{var}}, \quad \mathcal{L}_{\text{bias}} = \mathbb{E}_{\boldsymbol{\mu}} [\|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu})\|], \quad \mathcal{L}_{\text{var}} = \|\mathbf{A}^{-1} \mathbf{F}^H\|_F \cdot \epsilon_{\text{surr}},$$

where $\epsilon_{\text{surr}} = \mathbb{E}_{\mu}[\|\hat{\mathbf{u}}(\mu) - \tilde{\mathbf{u}}(\mu)\|_{\mathbb{C}^{K_{\text{full}}}}]$ is the expected surrogate error. By Jensen's inequality ($\mathbb{E}[\sqrt{X}] \leq \sqrt{\mathbb{E}[X]}$), this expected norm is bounded by:

$$(27) \quad \begin{aligned} \epsilon_{\text{surr}} &\leq \sqrt{\mathbb{E}_{\mu}[\|\hat{\mathbf{u}}(\mu) - \tilde{\mathbf{u}}(\mu)\|_{\mathbb{C}^{K_{\text{full}}}}^2]} \\ &= \sqrt{\sum_{s_k \in \mathcal{S}} \mathbb{E}_{\mu}[|\hat{u}_k(\mu) - \tilde{u}_k(\mu)|^2]}, \end{aligned}$$

Surrogate error via an SVD residual. Training a full neural surrogate for every proposed contour is computationally intractable. We therefore replace the expected surrogate error by the residual of the best **linear** reduction of the same rank, which is available in closed form from a singular value decomposition. Here $\hat{U}(s_k; \mu) \in \mathbb{C}^{N_x}$ denotes the **full spatial** Laplace field at frequency s_k (the vector of per-pixel values $\hat{u}_k(\mu)$ across all N_x locations), as opposed to the scalar $\hat{u}_k(\mu)$ at a single location used above. Each instance is represented by the real vector obtained by stacking its K spatial Laplace fields and splitting them into real and imaginary parts,

$$(28) \quad \mathbf{a}(\mu) = [\Re \hat{U}(s_1; \mu); \Im \hat{U}(s_1; \mu); \dots; \Re \hat{U}(s_K; \mu); \Im \hat{U}(s_K; \mu)] \in \mathbb{R}^{2N_x K},$$

and the design is summarised by the matrix $\mathbf{U} = [\mathbf{a}(\mu_1), \dots, \mathbf{a}(\mu_{N_{\text{opt}}})] \in \mathbb{R}^{2N_x K \times N_{\text{opt}}}$ assembled over a sample of N_{opt} training instances. The expected squared error of the optimal linear rank- d_z reduction is the mean squared column-wise error, whose minimum over all rank- d_z approximations is attained by the truncated SVD and equals the sum of the discarded squared singular values:

$$(29) \quad \mathcal{E}_{\text{SVD}} = \sqrt{\min_{\text{rank } \hat{U} \leq d_z} \frac{1}{N_{\text{opt}}} \|\mathbf{U} - \hat{U}\|_F^2} = \sqrt{\frac{1}{N_{\text{opt}}} \sum_{j > d_z} \sigma_j^2},$$

where σ_j are the singular values of $\mathbf{U}$. Two features of (29) are deliberate. First, the basis is **single and shared across the K frequencies**: the residual scores a rank- d_z reduction of the entire space–frequency field of a simulation, not K independent ones. SLAE shares its weights across frequencies as well, the FiLM modulation of Section 2.2.1 being its only per-frequency component, but it emits d_z coefficients **per** frequency, Kd_z in total. The joint residual is thus the degenerate case of the same structure, weight sharing stripped of its conditioning and a budget of d_z spread over the whole spectrum instead of d_z at each frequency, hence more constrained; we retain it because it depends on the contour through a single spectrum and is what our implementation computes. Second, $\mathbf{U}$ is never formed: the $N_{\text{opt}} \times N_{\text{opt}}$ Gram matrix $\mathbf{U}^\top \mathbf{U}$ is accumulated over spatial chunks and (29) read off its eigenvalues, at a memory cost independent of N_x ; the truncation rank is therefore capped at $\min(d_z, N_{\text{opt}})$, and we use $N_{\text{opt}} = 100$ instances against $d_z = 64$.

What this residual says about the surrogate is fixed by one observation, proved in Appendix H: over the optimisation sample, $\mathcal{E}_{\text{SVD}}$ is exactly the smallest coefficient error attainable by **any** map $\mu \mapsto \mathbf{D} g(\mu)$ with a rank- d_z linear decoder $\mathbf{D}$, the regressor g being arbitrary — unconstrained in regularity, and free to interpolate the sample exactly. It is therefore a floor on the parameter-to-coefficient problem itself, imposed by the bottleneck and the contour alone and owing nothing to how a regressor is trained. This is what we ask of it: a contour whose Laplace fields nearly span a d_z -dimensional subspace across the parameter range poses an easier prediction problem than one whose fields do not, and $\mathcal{E}_{\text{SVD}}$ scores that difficulty.

Two gaps separate this floor from the error a deployed surrogate actually makes, and we state them rather than model them. The floor holds for a **linear** decoder reading a single unconditioned basis and a budget of d_z for the whole spectrum, whereas SLAE decodes non-linearly, modulates that decoder per frequency, and disposes of d_z coefficients at each frequency: the two share their weights across frequencies alike and differ by the conditioning and the budget, on which counts SLAE is strictly more expressive, so $\mathcal{E}_{\text{SVD}}$ does not bind it — and conversely a trained network is guaranteed to attain no floor at all, SLAE exceeding even the linear reconstruction floor of Section 3.2.1 in four of its seven configurations. And $\mathcal{E}_{\text{SVD}}$ is a truncation residual, silent on the regression error committed on the d_z retained coordinates. That the ordering of contours under $\mathcal{E}_{\text{SVD}}$ tracks their ordering under the true surrogate error is accordingly a working hypothesis and not a consequence of the proposition: the linear rank of the image and the regularity of the map $\mu \mapsto \mathbf{a}(\mu)$ are independent properties in general. We retain $\mathcal{E}_{\text{SVD}}$ because it is deterministic, differentiable in the triplet and evaluable without training anything, and we carry one scalar $\lambda_{\text{AE}} \geq 1$ to set the magnitude at which the variance term enters: ϵ_{surr} is replaced in (31) by $\mathcal{E}_{\text{SVD}}/\lambda_{\text{AE}}$, a nominal error level fixed a priori rather than calibrated against a measured surrogate. Dropping the guarantee costs nothing here: the smoothness heuristic below removes the upper-bound status of the objective in any case, and a selection criterion is judged on the triplets it selects.

Smoothness heuristic on the bias term. We add one purely **heuristic** ingredient, which does not follow from the bound (25). At equal bias norm, a smooth offset is far less damaging than Gibbs-like oscillations, the characteristic artefact of spectral truncation, yet the L^2 norm does not distinguish them. We therefore also penalise the finite differences of the bias error. Writing $\mathbf{e}(\mu) = \mathbf{U}(\mu) - \mathbf{U}^*(\mu) \in \mathbb{R}^{N_x \times N_t}$ and $\mathbf{D}_t, \mathbf{D}_x, \mathbf{D}_y$ for the first-order difference operators along the temporal axis and along the two axes of the spatial grid,

$$(30) \quad \mathcal{L}_{\text{bias}} = \frac{1}{1 + \lambda_{\Delta t} + \lambda_{\Delta x}} \mathbb{E}_{\mu} \left[\|\mathbf{e}(\mu)\|_F + \lambda_{\Delta t} \|\mathbf{D}_t \mathbf{e}(\mu)\|_F + \lambda_{\Delta x} (\|\mathbf{D}_x \mathbf{e}(\mu)\|_F + \|\mathbf{D}_y \mathbf{e}(\mu)\|_F) \right],$$

the normalisation keeping $\mathcal{L}_{\text{bias}}$ on the scale of the variance term irrespective of the weights. Setting $\lambda_{\Delta t} = \lambda_{\Delta x} = 0$ recovers the plain bias of (25); with positive weights the objective is no longer an upper bound on the reconstruction error, the guarantee being traded for a preference over the **shape** of the residual. The final proxy objective is therefore:

$$(31) \quad \begin{aligned} \mathcal{L}^{\text{proxy}} = & \frac{1}{N_{\text{opt}} (1 + \lambda_{\Delta t} + \lambda_{\Delta x})} \sum_{i=1}^{N_{\text{opt}}} \left(\|\mathbf{e}(\mu_i)\|_F + \lambda_{\Delta t} \|\mathbf{D}_t \mathbf{e}(\mu_i)\|_F \right. \\ & \left. + \lambda_{\Delta x} (\|\mathbf{D}_x \mathbf{e}(\mu_i)\|_F + \|\mathbf{D}_y \mathbf{e}(\mu_i)\|_F) \right) \\ & + \frac{\|\mathbf{A}^{-1} \mathbf{F}^H\|_F}{\lambda_{\text{AE}}} \cdot \sqrt{\frac{1}{N_{\text{opt}}} \sum_{j>d_z} \sigma_j^2}. \end{aligned}$$

Minimizing (31) by gradient descent jointly optimizes the contour frequencies and the regularization parameters, trading bias against variance in a principled and fully differentiable manner. Both terms are evaluated on the same sample of N_{opt} training instances, drawn at a single rotation angle so that the sample spans the (k, C) plane rather than 36 views of a few simulations; the whole procedure runs in double precision and touches no

network. The remaining settings (optimiser, step count, the values of $\lambda_{\Delta t}$, $\lambda_{\Delta x}$, λ_{AE} , and the box constraints $\sigma_k \geq -0.05$, $\omega_k \geq 0$, $\alpha_t, \lambda \in [10^{-6}, 1]$ under which (31) is minimised) are reported in Appendix G; the optimised contours are used in Section 3.5.

2.5 Baselines

The approach described so far combines two ingredients: a non-linear autoencoder for spatial compression, and a Laplace transform for temporal compression. We therefore compare against two references, each removing exactly one of them. The first is **linear** in the spatial compression step but keeps the Laplace domain: a per-frequency POD basis with a non-intrusive surrogate on the projection coefficients (Section 2.5.1). The second is **non-linear** but Laplace-free, and keeps the full temporal grid: a standard DL-ROM (Section 2.5.2). Read together, they attribute the observed performance to one ingredient or the other rather than to their combination. Both reuse the surrogate of Section 2.3 and the two-phase training protocol unchanged.

2.5.1 Linear Baseline: POD in the Laplace Domain

As a linear reference, we construct a Proper Orthogonal Decomposition (POD) basis for each Laplace frequency independently. For each s_k , the training-set spatial fields are stacked into a data matrix $\mathbf{U}^{(k)} \in \mathbb{R}^{N_x \times N_{\text{train}}}$ (one per physical component), and a truncated basis of rank k_{pod} is extracted. Because $N_x = N^2 = 16,384$ and $N_{\text{train}} = 5,184$, a direct SVD of $\mathbf{U}^{(k)}$ would be memory-intensive; we instead use a two-pass randomized SVD [12] that reduces the cost to two passes over the data and a small k_{pod} -dimensional eigenvalue problem. The real and imaginary parts of $\hat{U}(s_k)$ are treated as separate real-valued fields, each with its own basis $\mathbf{V}^{(k,r)}, \mathbf{V}^{(k,i)} \in \mathbb{R}^{N_x \times k_{\text{pod}}}$ fitted on the training set:

$$(32) \quad \Re(\hat{U}(s_k)) \approx \mathbf{V}^{(k,r)} \mathbf{c}^{(k,r)}, \quad \Im(\hat{U}(s_k)) \approx \mathbf{V}^{(k,i)} \mathbf{c}^{(k,i)},$$

where $\mathbf{c}^{(k,r)}, \mathbf{c}^{(k,i)} \in \mathbb{R}^{k_{\text{pod}}}$ are the projection coefficients. This is the classical Galerkin-POD representation applied frequency-by-frequency in the Laplace domain [13].

Surrogate for the POD coefficients. Given the compact coefficient vectors, the surrogate task reduces to predicting $2K k_{\text{pod}}$ scalar targets from $\boldsymbol{\mu}$. This is the classical non-intrusive POD-NN strategy [14], here applied frequency-by-frequency in the Laplace domain. The surrogate MLP g_θ of Section 2.3 predicts the normalized coefficients for all frequencies jointly:

$$(33) \quad \mathbf{c}^{(k,r)}, \mathbf{c}^{(k,i)} = g_\theta(\boldsymbol{\mu}, k), \quad k = 0, \dots, K-1.$$

Training minimizes a combined latent-coefficient MSE and physical reconstruction loss (after Tikhonov inversion):

$$(34) \quad \mathcal{L}_{\text{POD-surr}} = \alpha_{\text{lat}} \sum_{k=0}^{K-1} \left(\|g_\theta^{(k,r)}(\boldsymbol{\mu}) - \bar{\mathbf{c}}^{(k,r)}\|_2^2 + \|g_\theta^{(k,i)}(\boldsymbol{\mu}) - \bar{\mathbf{c}}^{(k,i)}\|_2^2 \right) + \alpha_{\text{spat}} \|U_{\text{rec}} - U\|_2^2,$$

where $\bar{\mathbf{c}}^{(k,\cdot)}$ denotes the normalized coefficients and U_{rec} is the field reconstructed by back-projecting the predicted coefficients through the fixed bases and applying the inverse Laplace transform. The spatial bases $\mathbf{V}^{(k,\cdot)}$ remain **fixed** throughout surrogate training.

The time series $U(t)$ is reconstructed by back-projecting the predicted coefficients, assembling the complex frequency field, and applying the Tikhonov inversion (3). This pipeline is fully linear in the spatial compression step, making it a clean reference that quantifies the gain of non-linear autoencoders. The irreducible truncation error of the linear basis, controlled by the discarded singular values of $U^{(k)}$, sets a hard floor that non-linear AEs are designed to overcome.

2.5.2 Nonlinear Baseline: DL-ROM without Laplace

The POD baseline removes the non-linearity; our second reference removes the Laplace transform instead. It follows the DL-ROM paradigm of [5] — a convolutional autoencoder for spatial compression, a network mapping μ to the latent trajectory, and no spectral compression anywhere in the pipeline:

$$(35) \quad U(t) \xrightarrow{\text{Encoder}(t)} z(t) \xrightarrow{\text{Decoder}(t)} U_{\text{rec}}(t)$$

This is exactly the LLAE pipeline of Section 2.2.2 with the latent round trip $z(t) \rightarrow \hat{z}(s_k) \rightarrow \tilde{z}(t)$ removed: the same convolutional backbone, the same FiLM conditioning on the normalised time t/T , the same L_2 ridge on the latent codes. The loss retains the first and third terms of (12),

$$(36) \quad \mathcal{L}_{\text{DL-ROM}} = \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|U_{\text{rec}}(t_n) - U(t_n)\|_2^2 + \beta \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|z(t_n)\|_2^2,$$

the ridge now applying to $z(t)$ directly rather than to $\hat{z}(s_k)$. The latent consistency term has no counterpart here: with no round trip, there is no invertibility to enforce. The comparison is therefore a controlled ablation of the Laplace block rather than a comparison between two independently tuned models. We take [5] as the inspiration for that ablation rather than reproducing it: its surrogate is a single time-conditioned network queried once per step, ours carries one head per time step, so that the predictor stays architecturally fixed across all four pipelines and only the Laplace block varies. “DL-ROM” accordingly designates our Laplace-free ablation throughout, in the sense of the family; Section 4.5 states which cost comparisons depend on that choice. Within the family we follow [5] rather than the POD-DL-ROM variant [6], whose POD pre-reduction is an efficiency device that leaves the learned representation unchanged; Section 4.4 returns to this point.

Surrogate cost. The surrogate of Section 2.3 carries over unchanged except in **what** it must predict. Where the Laplace pipelines regress the K Laplace-domain latents w_k , the DL-ROM surrogate must regress the latent trajectory at every time step, $\mu \mapsto z(t_n)$ for $n = 0, \dots, N_t - 1$: the per-frequency heads become per-time-step heads, conditioned on $t_n/(N_t - 1)$ in place of f_k , and their number grows from K to N_t . With a shared trunk of N_{trunk} parameters and heads of N_{head} parameters each (N_{head} depending only on d_z and the head width, not on their number),

$$(37) \quad N_{\text{surr}} = N_{\text{trunk}} + K N_{\text{head}}, \quad K = N_t \text{ (DL-ROM)}, \quad K \ll N_t \text{ (SLAE, LLAE)},$$

so the DL-ROM surrogate grows linearly with the temporal resolution, while the Laplace surrogates are independent of it. With the architecture of Appendix G and $d_z = 64$, this amounts to 23.5M parameters at $N_t = 150$ against 3.5M for LLAE at $K = 16$, a factor of 6.8, the two becoming equal in size at $N_t \approx 18$.

Temporal-resolution sweep. Since N_t is the sole driver of this cost, we sweep it by uniform subsampling of the time grid with a stride κ , so that $N_t \mapsto \lceil N_t/\kappa \rceil$ and $\Delta t \mapsto \kappa \Delta t$ while the horizon T is held fixed. This traces the DL-ROM cost/accuracy curve while the Laplace pipelines remain at $K = 16$, and it supplies a parameter-matched operating point: at $\kappa = 10$ the DL-ROM surrogate has essentially the size of LLAE at $K = 16$, both compressing time by roughly a factor of ten, uniformly on one side and spectrally on the other. The sweep concerns the DL-ROM alone; the Laplace pipelines are always trained on the full $N_t = 150$ grid, since their temporal compression is already carried by K .

2.6 Architecture Summary

Table 1 provides a synthetic overview of all eleven pipelines studied. The three top rows describe the **autoencoder reconstruction** (Phase 1, input $U(t)$); the eight remaining rows compare the **surrogate inference** pipelines (Phase 2, input μ), the two baselines first, then grouped by the domain in which the Laplace transform is applied.

3 Results

3.1 Problem Setting

We consider the emulation of the methane mass fraction Y in a two-dimensional parametric transient reacting flow. The transport equation is formulated as a convection-diffusion-reaction problem over a spatial domain $\Omega \subset \mathbb{R}^2$ and a time interval $t \in [0, T]$:

$$(38) \quad \frac{\partial(\rho Y)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y) - \nabla \cdot (\mu_{\text{eff}} \nabla Y) = \dot{\omega}_{\text{eff}}(Y; \mu)$$

where ρ is the fluid density, $\mathbf{U}$ is the velocity field, μ_{eff} is the effective diffusivity, and $\dot{\omega}_{\text{eff}}$ is the effective reaction rate. A detailed formulation of the complete system of governing equations is provided in Appendix F. The system is governed by a vector of three physical parameters $\mu = [k, \phi, C]^\top \in \mathbb{R}^3$, specifically representing:

- k : Diffusion parameter,
- ϕ : Injection angle,
- C : Injection rate (kg/m³/s).

The temporal domain is discretized into $N_t = 150$ time steps with uniform step size $\Delta t = 1$ s (total duration $T = 150$ s). The quantity of interest in the methodology notation is $U(t) \in \mathbb{R}^{N_x}$, the discretized spatial field of Y at time step t ; the full space–time field is a matrix in $\mathbb{R}^{N_x \times N_t}$. The number of Laplace frequency samples K is a model hyperparameter ($K = 16$ for both SLAE and LLAE; see Table 14).

The transient rises, peaks near $t \approx 70$ s and decays back to zero inside the window (Figure 3).

Dataset construction. The parameter sweep is carried out over (k, C) only: 225 high-fidelity simulations on the native 200×200 solver grid. The third parameter, the injection angle ϕ , is realised geometrically. The governing operator being isotropic and the injection centred, a rotation by ϕ about the domain centre maps a solution onto the solution of the problem with the injection rotated by ϕ , and each simulation is replicated at 36 uniformly spaced angles $\phi \in \{0^\circ, 10^\circ, \dots, 350^\circ\}$. The rotation is applied at native

Model	Pipeline	$\mathcal{L}$ applied in
<i>Autoencoder reconstruction</i> (Phase 1, input: $U(t)$)		
SLAE		phys. domain
LLAE		latent space
DL-ROM		none
<i>Surrogate inference</i> (Phase 2, input: μ)		
POD		phys. domain
DL-ROM		none
SLAE-Direct		phys. domain
SLAE-SVD		phys. domain
SLAE-Tucker		phys. domain
LLAE-Direct		latent space
LLAE-SVD		latent space
LLAE-Tucker		latent space

Table 1: Architecture summary of the eleven pipelines studied. **Top block** (Phase 1): autoencoder reconstruction, input $U(t)$. **Bottom block** (Phase 2): surrogate inference pipelines mapping $\mu \rightarrow U(t)$, the two baselines first, then grouped by the domain in which $\mathcal{L}$ is applied, physical domain (SLAE family) or latent space (LLAE family). The DL-ROM applies no Laplace transform at either phase. Colour code: ■ MLP surrogate; ■ Laplace $\mathcal{L}$ / inverse $\mathcal{L}^{-1}$; ■ SVD back-projection; ■ Tucker back-projection; ■ convolutional encoder E (wide→narrow) / decoder D (narrow→wide); ● latent-space representation on the arrow.

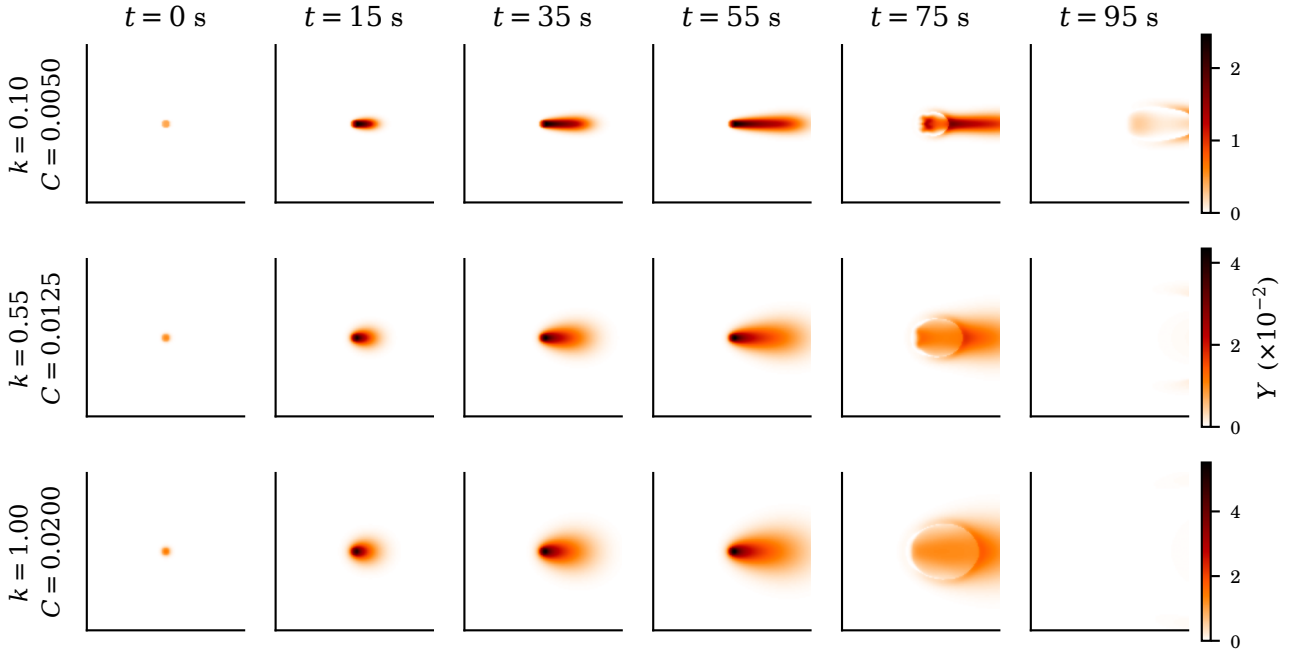


Figure 3: Methane mass fraction Y at six instants, for three instances spanning the parameter box all at $\phi = 0^\circ$. Colour scale shared along a row.

resolution and centre-cropped to 128×128 ; the corners of the retained window lying at a radius of 90.5 cells, inside the disc of radius 100 inscribed in the solver domain, the crop retains only the region untouched by the rotation fill, with no interpolation involved. This yields $N_{\text{sim}} = 225 \times 36 = 8,100$ parameter instances on a uniform grid of $N_x = 128 \times 128 = 16,384$ nodes.

Each parameter acts differently on the field (Figure 4).

Partition. A fixed random permutation reserves 20% of the instances, 1,620, as a held-out test set; the remaining 6,480 are split 80/20 into 5,184 training and 1,296 validation instances. Every quantity reported in this paper is measured on the 1,620 test instances. The validation set alone drives early stopping and checkpoint selection (Appendix G); the test set is never used for any training or selection decision, and the POD bases of Section 2.5.1 are fitted on the training partition only.

This replication is a solver shortcut rather than data augmentation: each of the 8,100 instances is an exact solution of the governing system at its own point (k, ϕ, C) , and ϕ enters the design of experiments on the same footing as k and C .

The performance of the models is evaluated quantitatively using the relative L^2 error over the held-out test set, defined as:

$$(39) \quad \epsilon_{\text{rel}} = \frac{\|Y_{\text{true}} - Y_{\text{pred}}\|_2}{\|Y_{\text{true}}\|_2}$$

We first characterize the autoencoder architectures in isolation, then evaluate the end-to-end surrogate models.

3.2 Baseline Results

The two references of Section 2.5 remove one ingredient each (POD in the Laplace domain drops the non-linearity, the DL-ROM drops the Laplace transform) and both

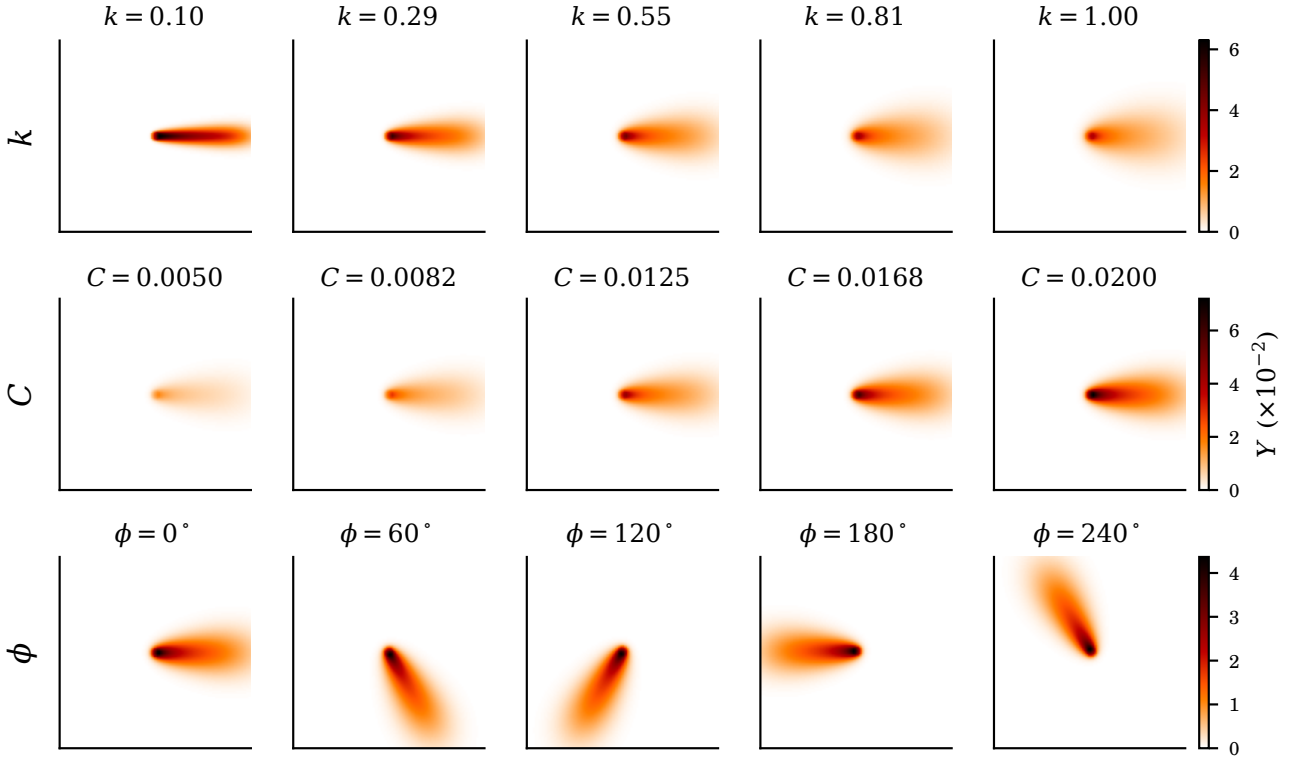


Figure 4: Effect of each parameter at $t = 60$ s, the other two held at the centre of the grid. Colour scale shared along a row.

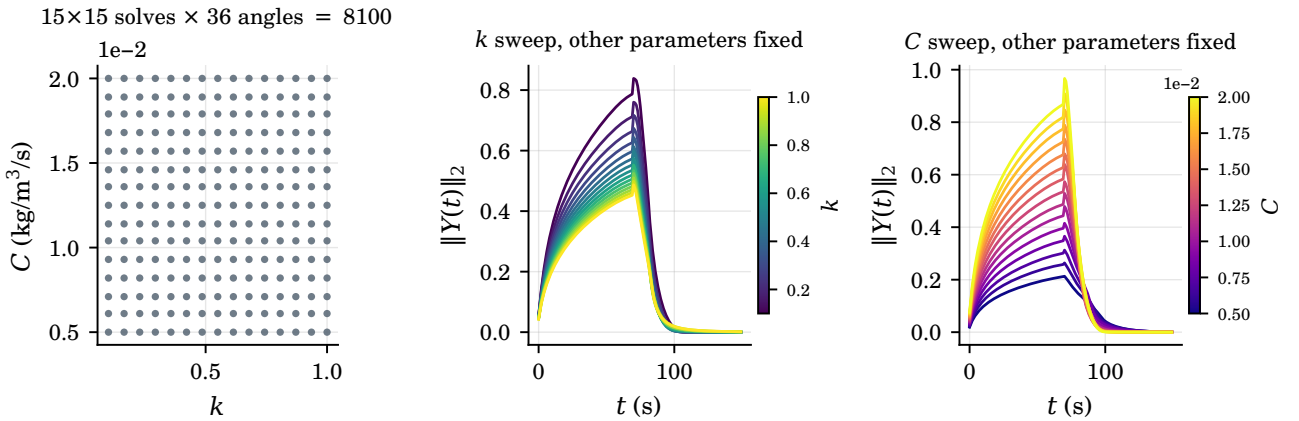


Figure 5: **Left:** design of experiments, a 15×15 grid on (k, C) replicated at 36 angles. **Centre and right:** field norm $\|Y(t)\|_2$ along each sweep, coloured by the swept parameter. All transients decay to zero inside the $T = 150$ s horizon.

are characterised in the same two stages as the Laplace pipelines. The first isolates the **representation**, passing the true field through the compression and back with no surrogate involved, which sets the floor of the pipeline; the second is the deployed **surrogate**, predicting from μ alone. Reporting both separates what the representation cannot represent from what the regression fails to predict, and their difference, the Gap, decides whether a better floor is worth having.

3.2.1 Linear Baseline: POD in the Laplace Domain

The three hyperparameters of the linear baseline (Section 2.5.1) are swept independently around a base configuration ($K = 16$, $k_{\text{pod}} = 64$, $\gamma = 0.01$): $K \in \{8, 16, 32\}$, $k_{\text{pod}} \in \{32, 64, 128\}$ and $\gamma \in \{0, 0.001, 0.01\}$. Each configuration is evaluated at both stages on the 1620 test simulations: as a pure **reconstruction**, in which the true fields are projected onto the training bases with no surrogate involved, and **end-to-end**, in which a network predicts the coefficients from μ . Table 2 reports both.

k_{pod}	K	γ	Rec med	Rec p90	Surr med	Surr p90	Gap (pp)	Lat (ms)
64	32	0.01	7.57	15.08	8.75	17.45	+1.18	5.6 ± 0.0
64	16	0.01	8.09	14.67	8.92	16.35	+0.83	3.5 ± 0.2
64	16	0.001	8.38	14.29	8.95	15.37	+0.57	3.5 ± 0.1
64	16	0.0	8.42	14.22	8.98	15.56	+0.56	3.5 ± 0.1
128	16	0.01	7.04	9.79	11.30	21.76	+4.26	3.7 ± 0.1
32	16	0.01	11.42	22.99	11.72	23.47	+0.30	3.4 ± 0.2
64	8	0.01	12.01	17.39	12.45	18.42	+0.44	2.9 ± 0.2

Table 2: POD baseline on the test set (1620 simulations), sorted by surrogate median error. All errors are relative L^2 errors in %. **Rec**: pure POD reconstruction error (linear floor, no surrogate). **Surr**: end-to-end surrogate error. **Gap**: difference surrogate – reconstruction (positive = surrogate adds error on top of the linear floor). **Lat**: GPU inference latency at batch size 1 (50 timed runs).

Linear reconstruction floor. The floor is governed by the singular-value decay of the Laplace-domain field matrices $U^{(k)}$: the linear basis is accurate at low frequencies, where the spatial structure is smooth and low-rank, and degrades at higher ones, where the fields become localized. Increasing k_{pod} beyond 64 yields only a marginal improvement, indicating that the residual is an intrinsic limit of the linear subspace rather than a truncation artefact. The damping is nearly neutral on the physical field (8.42, 8.38 and 8.09% for $\gamma = 0, 0.001$ and 0.01) because two error sources cancel: a purely imaginary contour concentrates the energy and lowers the POD truncation error at every frequency, whereas a larger γ attenuates the inversion error that propagates those residuals back to the time domain. Appendix B reports the complete sweeps and the per-frequency breakdown that exposes this trade-off.

Surrogate overhead. The regression stage adds little above the floor, between +0.30 and +1.18 pp, with one exception: at $k_{\text{pod}} = 128$, $K = 16$, the best floor of the sweep (7.04%) yields its worst surrogate (11.30%, +4.26 pp). With $2Kk_{\text{pod}} = 4,096$ scalar targets, the higher-dimensional regression problem overwhelms the benefit of the larger basis, a

trade-off between representation quality and learnability that recurs at every stage of this study. The POD reference used throughout is therefore $k_{\text{pod}} = 64$, $K = 16$, $\gamma = 0.01$, the configuration matching the common operating point of the deep models ($d_z = k_{\text{pod}}$, same K , same γ) rather than the lowest surrogate error of the sweep, at 8.92% median error and 2.9–5.6 ms per sample at batch size 1, its cost dominated by the back-projection through $V^{(k)}$ and the Tikhonov solve rather than by the network forward pass. This is the floor that the non-linear autoencoders of Section 3.3 are designed to push below; per-sample distributions are provided in Appendix A.

3.2.2 Nonlinear Baseline: DL-ROM without Laplace

The second reference removes the Laplace transform instead of the non-linearity (Section 2.5.2). It is trained on the full temporal grid ($N_t = 150$, $\Delta t = 1$ s) with the same convolutional backbone, the same latent dimension $d_z = 64$, the same two-phase protocol and the same data partition as the Laplace pipelines, the only structural difference being that its surrogate carries one head per time step instead of one per Laplace frequency. Table 3 reports the two stages.

Stage	Median (%)	Mean (%)	p90 (%)	Gap (pp)
Autoencoder (frame-wise reconstruction)	2.49	2.71	3.87	—
End-to-end surrogate	3.80	4.15	6.12	+1.31

Table 3: DL-ROM baseline at full temporal resolution ($N_t = 150$, $d_z = 64$), relative L^2 error on the physical field over the test set ($n = 1620$). **Autoencoder**: each of the 150 frames encoded and decoded, no surrogate and no Laplace round-trip. **Surrogate**: prediction from μ alone through the 150 per-time-step heads. **Gap**: surrogate – autoencoder.

Autoencoder stage. The frame-wise autoencoder reconstructs the field to 2.49% median error (2.71% mean, 3.87% p90). This is the lowest reconstruction error of the entire study, and necessarily so: sharing the backbone, the latent dimension and the training protocol with the Laplace pipelines, it differs from them by the absence of the Laplace round-trip alone. Because nothing else differs, the distance between this floor and theirs prices that round-trip exactly; the comparison is made at the matched configuration in Section 3.3.

Surrogate stage. The end-to-end error distribution over the 1620 held-out simulations (Figure 23, Appendix A) is unimodal and tightly concentrated: median 3.80%, mean 4.15%, p90 6.12%, with a thin upper tail reaching 10%. The regression stage adds 1.31 pp to the reconstruction floor, more than the +0.83 pp of the POD reference, as expected of a target of 150 latent frames against $2 \times 16 \times 64$ POD coefficients. At full temporal resolution the DL-ROM is the most accurate emulator of this study, 57% below the POD reference (8.92%, Section 3.2.1). The direction is the expected one: predicting the latent trajectory at every one of the 150 time steps discards no spectral content, a strictly easier approximation problem than one constrained to $K = 16$ coefficients. The price is on the cost side, its surrogate carrying one head per time step rather than one per frequency (23.5M parameters, per equation (37)), and in the trajectory being delivered only on the training grid. Sections 4.1–4.3 weigh that accuracy against those costs alongside the Laplace surrogates.

3.3 Autoencoder Performance

Table 4 reports the relative L^2 reconstruction error on the held-out test set (1620 simulations) for all trained AE checkpoints, evaluated on the physical field $U(t)$.

Model	d_z	K	γ	Median (%)	Mean (%)	p90 (%)	vs. POD (pp)
LLAE	128	16	0.01	2.87	3.09	4.34	−4.17
	64	32	0.01	3.12	3.47	5.08	−4.45
	64	16	0.01	3.13	3.41	4.99	−4.96
	64	16	0.0	3.19	3.55	5.21	−5.23
	64	16	0.001	3.24	3.57	5.11	−5.14
	64	8	0.01	3.28	3.59	5.18	−8.73
	32	16	0.01	3.37	3.79	5.55	−8.05
SLAE	64	16	0.0	7.79	8.12	9.92	−0.63
	64	16	0.001	7.83	8.28	10.37	−0.55
	128	16	0.01	7.99	8.41	10.46	+0.95
	64	32	0.01	8.36	9.28	13.46	+0.79
	64	16	0.01	8.93	9.54	12.58	+0.84
	32	16	0.01	10.76	11.15	14.78	−0.66
	64	8	0.01	12.27	12.75	15.40	+0.26

Table 4: Relative L^2 reconstruction error on the test set (1620 simulations). The last column gives the difference in median error relative to the linear POD baseline at matching $(K, k_{\text{pod}} = d_z, \gamma)$: negative values indicate improvement over POD, positive values indicate that POD outperforms the AE. Best per family in bold.

Reconstruction floor. Across all 14 checkpoints the two families contrast starkly. LLAE reaches a best median error of 2.87% ($d_z = 128, K = 16$), a factor of 2.5 below the POD reconstruction at matching configuration (7.04% at $k_{\text{pod}} = 128, K = 16$) and of 2.5–3.5 against the other POD configurations. SLAE yields 7.79%–12.27%, broadly comparable to POD: the **vs. POD** column hovers around zero, positive as often as negative. As a pure compression stage SLAE therefore offers no meaningful gain over the linear reference, though the picture changes substantially at the surrogate stage (Section 3.4).

Price of the Laplace round-trip. The Laplace-free DL-ROM autoencoder of Section 3.2.2 supplies the second reference, and the only one that prices the temporal compression on its own. At the matched configuration ($d_z = 64, K = 16, \gamma = 0.01$) it reconstructs to 2.49% against 3.13% for LLAE, so the latent Laplace round-trip costs 0.64 pp; sharing the backbone, the latent dimension, the training protocol and the data partition, the two differ by that round-trip alone, which makes this the cleanest measurement of the truncation penalty in the study. It is a modest one, and the best LLAE closes it further, to 0.38 pp at $d_z = 128$ against a DL-ROM trained at half that latent width. The same comparison is severe for SLAE (8.93% against 2.49%): what costs 0.64 pp in latent space costs 6.44 pp in the physical domain, K complex frequency frames being a far more aggressive compression of the field itself than of its latent trajectory.

Why the two families diverge. Two compounding factors explain the gap (Sections 2.2.1–2.2.2). First, the input spaces differ: SLAE compresses high-dimensional complex frequency frames $\hat{U}(s_k)$, whereas LLAE encodes smoother physical frames, an intrinsically easier task. Second, the training objectives are asymmetric: LLAE’s loss (12) includes a direct physical-time reconstruction term that provides dense temporal supervision, while SLAE deliberately omits it during Phase 1 (Section 2.2.1) to preserve its computational advantage.

Hyperparameter sensitivity. The one-at-a-time ablations (Appendix C) separate the two families further. LLAE is insensitive to everything but its bottleneck: K moves the median by 0.16 pp over a fourfold range (3.28, 3.13 and 3.12% at $K = 8, 16$ and 32) and γ by less over $[0, 0.01]$, so 8 to 16 frequencies already carry the dominant transient dynamics, whereas d_z still pays at the top of the range ($3.37 \rightarrow 2.87\%$ from 32 to 128) and has therefore not saturated at $d_z = 64$. SLAE binds on both bottlenecks at once, degrading sharply at $K = 8$ (12.27%) and at $d_z = 32$ (10.76%); raising K to 32 does not help either (8.36% against 7.79% at $K = 16$), the higher frequencies carrying less spatial energy while the spatial bottleneck stays where it is. Its error distribution also carries the heavier right tail of the two.

Per-frequency breakdown. For SLAE, Figure 6 breaks the reconstruction error down per Laplace frequency k . The error is lowest at low frequencies ($k = 0$ –3, where most signal energy is concentrated) and rises monotonically toward higher frequencies, confirming that the spatial compression bottleneck is more severe for the fine-scale, high-frequency content. Against the POD per-frequency breakdown (Figure 28), SLAE achieves roughly half the per-frequency error of POD at each k , the convolutional encoder capturing non-linear structure the linear projection cannot. This advantage does not translate into a better overall spatial reconstruction (Table 4), because the inverse transform (3)–(4) damps all frequencies uniformly and partially cancels the benefit before the spatial field is recovered: the global spatial error is a **regularisation-weighted** aggregate of the per-frequency errors, not a direct reflection of per-frequency compression quality.

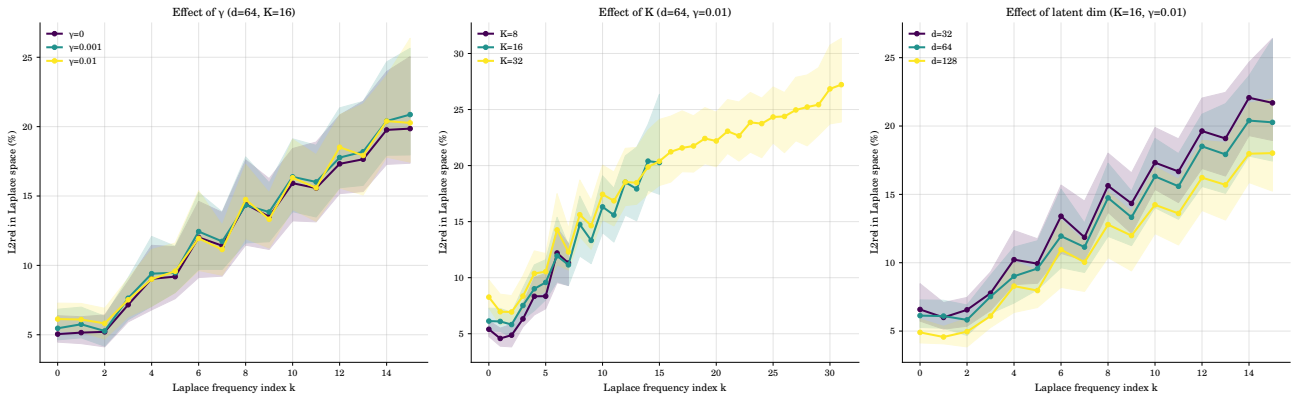


Figure 6: SLAE: per-frequency reconstruction error in the Laplace domain (median $\pm$ IQR, 200 test samples). Left: effect of γ ; centre: effect of K ; right: effect of d_z .

Inference cost. Table 5 reports the per-sample cost of a full reconstruction along three axes: arithmetic work (FLOPs, counted on the executed graph), wall-clock latency (batch size 1, 50 timed CUDA Event runs after 10 warm-up runs), and peak GPU

memory. FLOPs measure the cost of the **architecture**, independently of how well a given implementation exploits the hardware; latency is what a user experiences, and the two do not agree here. The arithmetic follows the architectural prediction of Sections 2.2.1–2.2.2 exactly: SLAE decodes K frequency frames, so it scales linearly with K (15.5, 31.1 and 62.2 GFLOPs at $K = 8, 16$ and 32), whereas LLAE decodes all $N_t = 150$ time frames regardless and stays flat at 54.2 GFLOPs across a fourfold variation of K . At $K = 16$ SLAE performs $1.74\times$ fewer operations, $3.5\times$ fewer at $K = 8$, the ordering reversing only above $K \approx 28$.

The wall-clock gain is smaller than the arithmetic one, $1.2\times$ at $K = 16$ (22.7 ± 1.1 against 27.0 ± 1.7 ms), because reconstruction from a physical field also pays the forward transform (1) and its regularised inverse, a fixed cost the FLOP ratio does not see and which the surrogate does not pay at all (Section 3.4.1). That same fixed cost governs the memory axis, the one on which the two families are comparable here (518 against 556 MB at $K = 16$): holding the entire physical field in memory to transform it is what brings SLAE up to LLAE’s level, and the surrogate, which never sees a physical time frame, recovers a $2.6\times$ advantage. The two regimes remain distinct underneath, SLAE’s allocation being set by K (404 to 794 MB from $K = 8$ to 32) and LLAE’s by N_t (within 3% of 556 MB throughout).

Model	K	d_z	GFLOPs	Latency (ms)	Peak GPU (MB)
SLAE	8	64	15.5	18.4 ± 1.3	404
SLAE	16	32	31.0	22.7 ± 1.1	510
SLAE	16	64	31.1	22.7 ± 1.1	518
SLAE	16	128	31.2	22.7 ± 1.0	534
SLAE	32	64	62.2	27.9 ± 1.4	794
LLAE	8	64	54.2	26.8 ± 2.1	556
LLAE	16	32	53.7	26.9 ± 1.8	550
LLAE	16	64	54.2	27.0 ± 1.7	556
LLAE	16	128	55.2	27.2 ± 2.0	568
LLAE	32	64	54.2	27.1 ± 1.9	556

Table 5: Reconstruction cost at $\gamma = 0.01$. GFLOPs are counted on the executed graph for one sample. Latency: batch size 1, 50 timed CUDA Event runs after 10 warm-up runs, median $\pm$ standard deviation. Peak GPU memory is the maximum allocated during one reconstruction. Best per column in bold.

3.4 Surrogate Prediction Performance

3.4.1 Direct surrogate performance

Table 6 reports the relative L^2 error on the held-out test set (1620 simulations) for all 14 end-to-end surrogate checkpoints, evaluated on the physical field $U(t)$. All three families (POD, SLAE, LLAE) use the same surrogate MLP architecture (Appendix G), so the comparison isolates the spatial compression strategy rather than the predictor capacity. Two columns aid interpretation: **Gap**, the difference between the surrogate median and the standalone AE median of Table 4 (positive for surrogate overhead, negative when

joint fine-tuning of the decoder **improves** upon the frozen-AE baseline), and **vs POD**, the difference to the matching POD surrogate of Table 2 at the same (K, d_z, γ) , negative throughout.

Model	d_z	K	γ	Median (%)	Mean (%)	p90 (%)	Gap (pp)	vs POD (pp)
LLAE	128	16	0.01	3.84	4.12	6.03	+0.97	−7.46
	64	32	0.01	4.33	4.63	6.79	+1.21	−4.42
	64	16	0.01	4.06	4.37	6.47	+0.93	−4.86
	64	16	0.0	4.34	4.68	6.93	+1.15	−4.64
	64	16	0.001	4.07	4.37	6.48	+0.83	−4.88
	64	8	0.01	4.01	4.34	6.42	+0.73	−8.44
	32	16	0.01	4.34	4.71	7.08	+0.97	−7.38
SLAE	64	16	0.0	7.39	7.53	8.81	−0.40	−1.59
	64	16	0.001	7.43	7.57	8.90	−0.40	−1.52
	128	16	0.01	7.50	7.67	9.18	−0.49	−3.80
	64	32	0.01	5.70	6.07	7.98	−2.66	−3.05
	64	16	0.01	7.59	7.87	9.50	−1.34	−1.33
	32	16	0.01	7.62	7.85	9.49	−3.14	−4.10
	64	8	0.01	11.62	11.89	13.84	−0.65	−0.83

Table 6: Direct surrogate results on the test set (1620 simulations), same row order as Table 4. **Gap**: surrogate median – AE reconstruction median; negative values indicate that end-to-end fine-tuning improves upon the standalone AE. **vs POD**: surrogate median – matching POD surrogate median (Table 2). Best per family in bold.

Overall accuracy. LLAE achieves 3.84–4.34% median L^2 error across all configurations, compared to 5.70–11.62% for SLAE. The best LLAE surrogate ($d_z = 128$, $K = 16$, $\gamma = 0.01$, 3.84%) represents a **57%** reduction in median error relative to the POD reference (8.92%), while the best SLAE surrogate ($d_z = 64$, $K = 32$, $\gamma = 0.01$, 5.70%) achieves a **36%** reduction.

End-to-end fine-tuning effect. The Gap column reveals a qualitative asymmetry between the two families. For LLAE, the surrogate step introduces a small positive overhead of +0.83 to +1.44 pp above the AE reconstruction floor: the surrogate approximation is not perfectly lossless, but the cost is modest given that the AE floor itself already saturates below 3.5%.

For SLAE, by contrast, the Gap is **negative** across all configurations, from −0.40 to −3.14 pp: the end-to-end pipeline **outperforms** the standalone SLAE autoencoder. The mechanism is that the Phase-1 loss $\mathcal{L}_{\text{SLAE}}$ (equation (10)) is defined purely in the Laplace domain, the decoder never being supervised on physical-time reconstruction, since such a loss would require N_t -frame decoder passes and negate SLAE’s speed advantage. Phase 2 introduces that term for the first time through $\mathcal{L}_{\text{total}}$ (Section 2.3) and fine-tunes the decoder through it, correcting residual errors the Laplace-domain loss could not resolve. The effect is largest where the AE floor is highest: at $d_z = 32$ (−3.14 pp) and $K = 32$ (−2.65 pp).

Hyperparameter sensitivity. LLAE surrogates remain highly robust: varying K from 8 to 32, d_z from 32 to 128, or γ from 0 to 0.01 all yield errors in a tight 3.84–4.34% band, consistent with the insensitivity already observed at the AE level (Section 3.3). SLAE is more sensitive: $K = 8$ collapses to 11.62% (the spectral truncation is too severe for the surrogate to compensate), while $K = 32$ is the best SLAE configuration precisely because the decoder, given access to 32 Laplace frequencies, can recover finer spatial structure during fine-tuning. Ablation violin plots are provided in Appendix D.

Where the error lives in the design. The parameter design is factorial, 15 values of k crossed with 15 of C and 36 injection angles ϕ , the angle being a numerical rotation of the same base solution (Section 3.1), so an aggregate median mixes three directions of unequal difficulty. Table 7 decomposes the variance of the per-instance error ε by the correlation ratio

$$(40) \quad \eta_X^2 = \frac{\text{Var}(\mathbb{E}[\varepsilon \mid X])}{\text{Var}(\varepsilon)}, \quad X \in \{\phi, k, C\}.$$

The angle explains almost nothing, $\eta_\phi^2 = 0.02$ – 0.03 for every checkpoint of both families, against 0.13–0.26 for k and 0.53–0.74 for C : the error is flat over the 36 rotations and is carried by the two physical parameters, C foremost. The maps of Figure 7 locate it, the upper decile being confined to the low- C , low- k corner where reaction and diffusion are slowest and the front takes longest to establish, an identifiable physical regime rather than dispersion. The flat dependence on ϕ does not, however, clear the aggregate of the optimism the rotation augmentation induces: the partition is random over the 8100 augmented instances, so η^2 measures variation **within** the design, not what a group-wise partition would give.

Checkpoint	η_ϕ^2	η_k^2	η_C^2
LLAE $d_z = 128, K = 16$	0.027	0.264	0.529
LLAE $d_z = 64, K = 16$	0.028	0.258	0.572
LLAE $d_z = 64, K = 8$	0.028	0.242	0.568
LLAE learnable, $K = 8$	0.025	0.185	0.690
LLAE learnable, $K = 16$	0.022	0.136	0.737
LLAE learnable, $K = 32$	0.021	0.127	0.755

Table 7: Correlation ratio η^2 of the per-instance relative L^2 error against each design factor, on the test set ($n = 1620$). k and C are binned into eight quantiles; ϕ is used at its 36 levels. Values sum to less than one because the factors do not exhaust the variance.

Inference cost. At the surrogate stage the arithmetic and wall-clock orderings coincide, the forward transform being absent: the network emits $\hat{U}(s_k)$ directly and only the inversion is evaluated, so the fixed overhead that attenuated the reconstruction ratios (Section 3.3) no longer applies. At batch size 1 and $K = 16$, SLAE requires 30.3 GFLOPs, 15.6 ± 0.3 ms and 235 MB against 48.2 GFLOPs, 22.0 ± 4.2 ms and 622 MB for LLAE, factors of 1.59, 1.41 and 2.6. The decoder, not the predictor, governs the cost: LLAE’s arithmetic is invariant at 48.2 GFLOPs for $K \in \{8, 16, 32\}$, whereas SLAE scales from 15.2 to 60.7 GFLOPs and crosses LLAE within the sweep; its latency dispersion is an

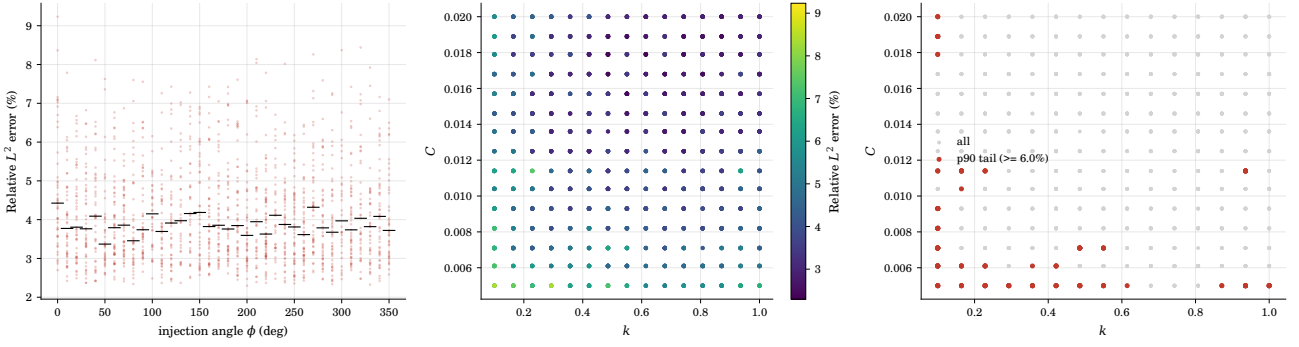


Figure 7: Error over the design of experiments for the best LLAE surrogate. Left: error against the injection angle ϕ , median per angle in black. Centre: error over the physical plane (k, C) , colour encoding the relative L^2 error. Right: location of the upper decile.

order of magnitude smaller (0.1–0.7 against 2.8–4.2 ms), the K frequency frames being decoded in a single batch. The POD surrogate bounds the attainable gain: 0.2 GFLOPs, some $150\times$ fewer than SLAE, yet 10.1 ms on 212 MB, within $1.5\times$ and $1.1\times$ of it, its cost being dominated by the Tikhonov solve and the back-projection through $V^{(k)}$. The regularised inversion is thus an incompressible component of every Laplace-domain pipeline at this problem size. Figure 8 details the 14 checkpoints.

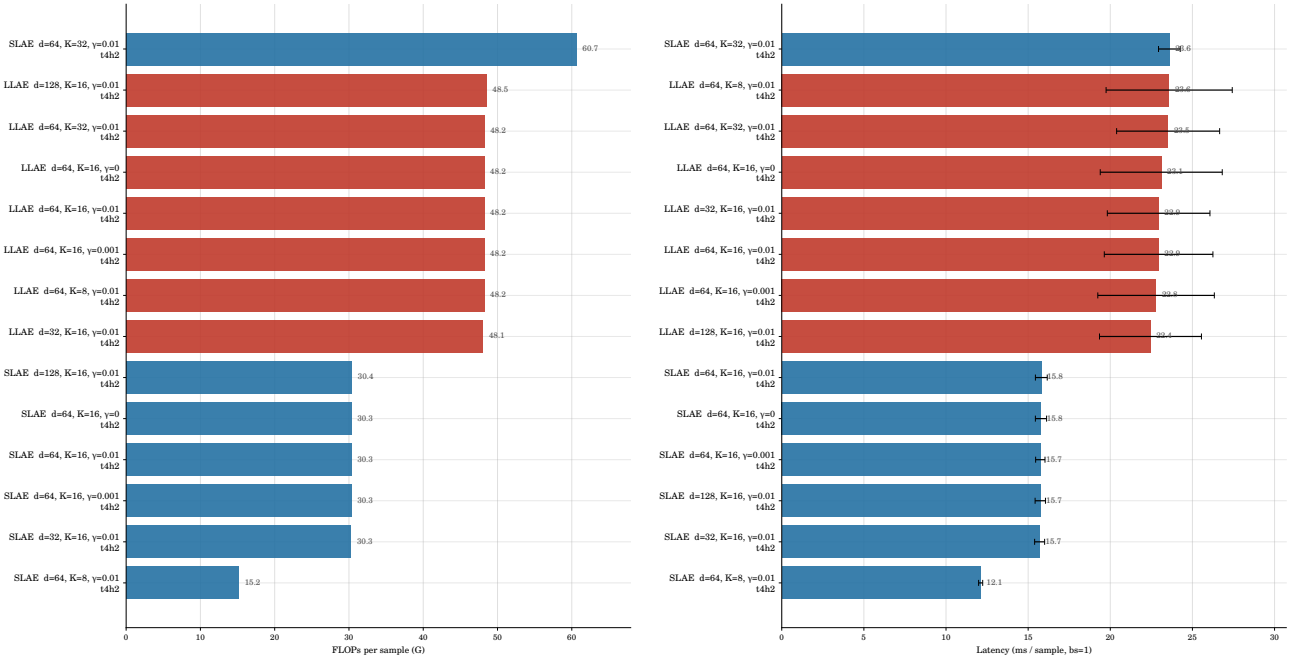


Figure 8: End-to-end surrogate cost for the 14 checkpoints (batch size 1, 50 timed CUDA Event runs). Left: arithmetic work per sample. Right: wall-clock latency.

Per-checkpoint error histograms for all 14 surrogate checkpoints are provided in Appendix A (Figure 22).

3.4.2 SVD surrogates: single-mode latent compression

The direct surrogate of Section 3.4.1 predicts the full d_z -dimensional latent at each frequency. To test whether compressing that representation first improves the predictor’s accuracy, we assess a hybrid approach inspired by Rank Reduction Autoencoders (RRAE) [24]: projecting the latent trajectories onto a truncated SVD basis before the

surrogate. We evaluate 26 AE+SVD checkpoints across the LLAE and SLAE families; the encoder is frozen while the decoder and the basis V are trained end-to-end alongside the surrogate, the latent dimension being reduced to $k_{\text{svd}} \in \{16, 32\}$ before an MLP predicts the reduced coefficients from μ .

Table 8 reports the relative L^2 error on the held-out test set evaluated on the physical field $U(t)$.

Model	d_z	K	γ	k_{svd}	Median (%)	Mean (%)	p90 (%)
LLAE	64	16	0.01	32	4.24	4.61	6.75
	64	32	0.01	32	4.29	4.69	7.02
	64	16	0.001	32	4.44	4.96	7.41
	64	16	0.0	32	4.50	4.97	7.29
	64	8	0.01	32	4.60	5.04	7.47
	128	16	0.01	32	4.77	5.30	7.82
	32	16	0.01	16	5.55	6.29	9.39
	64	32	0.01	16	5.58	6.22	9.35
	128	16	0.01	16	5.83	6.51	9.48
	64	16	0.01	16	5.84	6.58	9.79
	64	16	0.001	16	5.90	6.72	9.92
	64	16	0.0	16	5.94	6.76	10.18
	64	8	0.01	16	6.54	7.37	11.05
SLAE	64	32	0.01	32	6.03	6.58	9.02
	64	32	0.01	16	6.96	7.80	11.24
	64	16	0.0	32	7.50	7.73	9.26
	64	16	0.001	32	7.62	7.88	9.52
	64	16	0.01	32	7.90	8.32	10.54
	32	16	0.01	16	7.95	8.38	10.71
	64	16	0.001	16	8.01	8.43	10.80
	128	16	0.01	32	8.12	8.57	10.98
	64	16	0.0	16	8.13	8.61	11.06
	64	16	0.01	16	8.71	9.40	12.68
	128	16	0.01	16	8.89	9.71	13.43
	64	8	0.01	32	11.64	11.93	13.96
	64	8	0.01	16	11.97	12.38	14.88

Table 8: AE+SVD surrogate results on the test set ($n = 1620$), evaluating the physical field reconstruction error using LLAE and SLAE autoencoders coupled with an intermediate SVD compression ($k_{\text{svd}} \in \{16, 32\}$). Best median per family in bold.

Does compressing the latent space help? The central question is whether predicting on a smaller, orthogonalised latent space simplifies the MLP’s task enough to offset the information lost to SVD truncation. It does not. Measured **at matching autoencoder**

configuration, the SVD bottleneck adds a small penalty over the direct surrogate for both families: fixing $d_z = 64$, $\gamma = 0.01$, the best LLAE-SVD ($K = 16$, $k_{\text{svd}} = 32$, 4.24%) sits +0.18 pp above its direct counterpart (4.06%), and the best SLAE-SVD ($K = 32$, $k_{\text{svd}} = 32$, 6.03%) +0.33 pp above its own (5.70%). The MLP navigates the full d_z -dimensional latent space perfectly well, so the SVD projection merely discards spatial detail the direct surrogate had already learned to predict: compressing the latent before the surrogate degrades the reconstruction rather than helping it.

Impact of k_{svd} . Comparing pairs of configurations differing only in k_{svd} , Figure 9 shows that $k_{\text{svd}} = 32$ systematically improves accuracy (all points below the $y = x$ diagonal). The gain is markedly larger for LLAE (+1.06 to +1.94 pp reduction in median error) than for SLAE (+0.33 to +0.93 pp), indicating that LLAE latents carry more fine-grained information in the additional 16 modes, whereas SLAE discards spatial detail already at the AE stage.

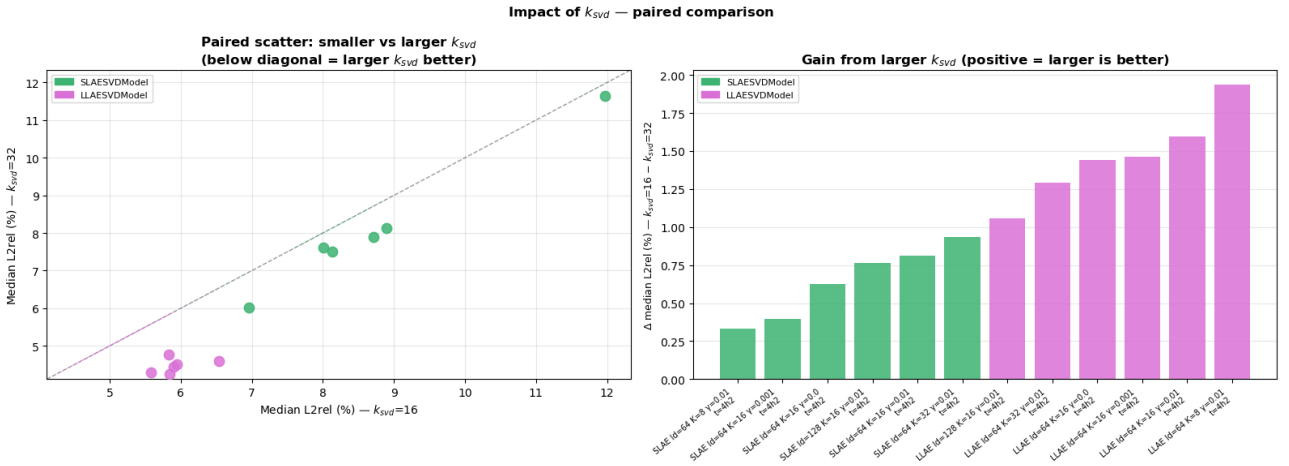


Figure 9: Impact of k_{svd} on SVD surrogate performance. Left: paired median errors, $k_{\text{svd}} = 16$ against $k_{\text{svd}} = 32$. Right: absolute error reduction from 16 to 32 modes, sorted by magnitude.

Hyperparameter sensitivity. Sensitivity to the AE hyperparameters K , d_z , and γ mirrors the direct case (Section 3.4.1): SLAE again degrades sharply at $K = 8$ (11.6–12.0%), while LLAE stays within a tight band. The lever specific to this pipeline is k_{svd} , whose effect is quantified above: retaining 32 rather than 16 modes is uniformly beneficial, and more so for LLAE than for SLAE.

Inference latency. The SVD back-projection inserted between the MLP and the spatial decoder adds a slight overhead. At batch size 1, LLAE-SVD requires approximately 22 ms per sample against about 9 ms for SLAE-SVD, which retains a $\approx 2.4\times$ speed advantage from its lighter decoder. Figure 10 presents the latency–accuracy trade-off.

3.4.3 Tucker surrogates: joint-mode latent compression

Section 3.4.2 compressed the latent representation along a single mode. We now evaluate the Tucker variant of Section 2.3, which factorises the frequency and latent modes **jointly**: the surrogate predicts a compact core matrix $\hat{\mathcal{G}} \in \mathbb{C}^{r_s \times r_z}$ (real-valued for SLAE) instead of K independent latents of dimension d_z , back-projecting through the frozen HOOI

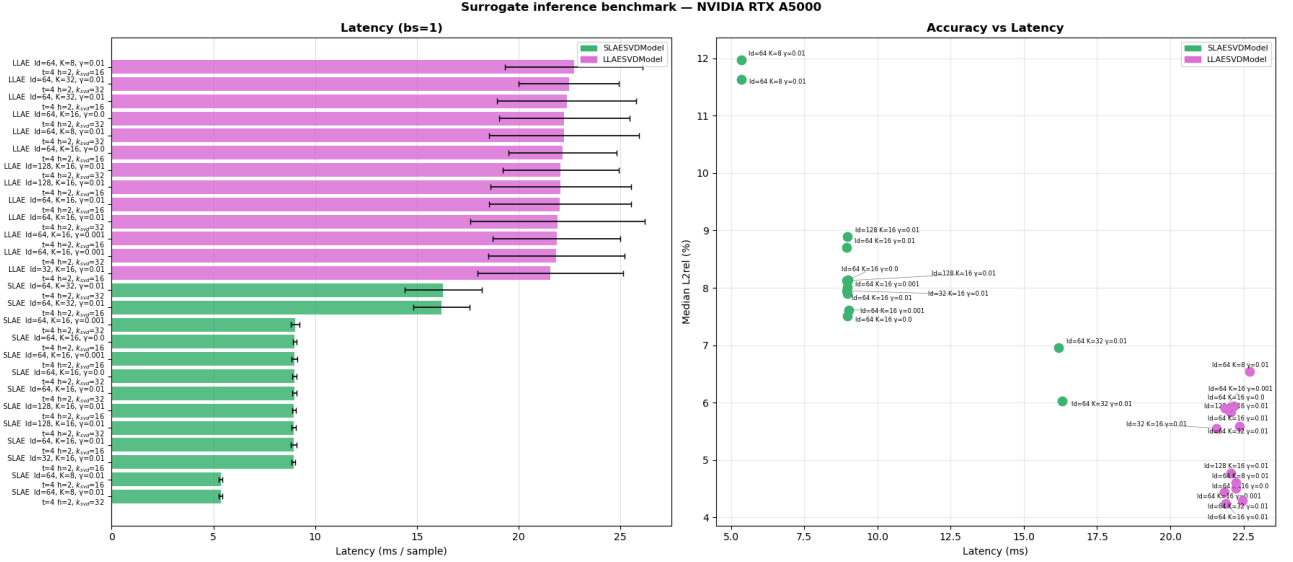


Figure 10: AE+SVD surrogate inference benchmark (batch size 1). Left: per-sample latency across checkpoints. Right: accuracy–latency scatter representing median L^2 error vs. latency.

factors $U^{(s)}, U^{(z)}$ before decoding. We sweep the Tucker ranks over a grid $r_s \in \{4, 8\}$ (frequency mode, $K = 16 \rightarrow r_s\}) \times r_z \in \{8, 16, 32\}$ (latent mode, $d_z = 64 \rightarrow r_z$), for both families, yielding 12 checkpoints. All share the same Phase 1 autoencoder ($d_z = 64$, $K = 16$, $\gamma = 0.01$); only the offline factors and the surrogate differ. The compression ratio $\rho = r_s r_z / (K d_z)$ measures the reduction of the surrogate target dimension relative to the direct pipeline (which predicts $K d_z = 1024$ scalars per sample).

Beyond the physical-field error $\varepsilon_{\text{field}}$ (relative L^2 on $U(t)$, full pipeline), we report two decoder-free diagnostics computed in the latent space, which split the difficulty between the offline compression and the surrogate MLP:

$$(41) \quad \varepsilon_{\text{trunc}} = \frac{\|\hat{Z} - \Pi \hat{Z}\|}{\|\hat{Z}\|}, \quad \varepsilon_{\text{reg}} = \frac{\|\hat{G}_{\text{pred}} - \hat{G}_{\text{true}}\|}{\|\hat{G}_{\text{true}}\|},$$

where $\Pi = U^{(s)} U^{(s)\top} (\cdot) U^{(z)} U^{(z)\top}$ is the Tucker projector. $\varepsilon_{\text{trunc}}$ measures the energy discarded by the rank truncation (factors only, independent of the surrogate), while ε_{reg} measures how well the MLP predicts the true core (surrogate only, independent of the truncation). Table 9 reports all three on the held-out test set.

Overall accuracy. As with SVD, the relevant question is what the extra compression costs relative to the direct and single-mode pipelines **at matching autoencoder configuration**. Fixing the shared AE ($d_z = 64$, $K = 16$, $\gamma = 0.01$), the median error grows only gently along the compression axis: 4.06% $\rightarrow$ 4.24% $\rightarrow$ 4.33% for LLAE (direct $\rightarrow$ SVD with $k_{\text{svd}} = 32 \rightarrow$ Tucker with $r_s = 8, r_z = 32$), and 7.59% $\rightarrow$ 7.90% $\rightarrow$ 7.92% for SLAE. Tucker thus adds +0.27 pp over the direct surrogate for LLAE and +0.33 pp for SLAE, and at most +0.09 pp over SVD, while shrinking the surrogate target to $r_s r_z = 256$ scalars, a 4 $\times$ reduction over the direct target ($K d_z = 1024$) and a 2 $\times$ reduction over the SVD target ($K k_{\text{svd}} = 512$). Factorising both modes at once is what buys this: exploiting cross-frequency correlations lets Tucker match the single-mode SVD accuracy at half the target dimension.

Model	r_s	r_z	ρ	Median (%)	Mean (%)	p90 (%)	$\varepsilon_{\text{trunc}}$ (%)	ε_{reg} (%)	Lat (ms)
LLAE	8	32	0.250	4.33	4.75	7.16	36.98	7.25	18.2
	4	32	0.125	4.57	5.04	7.61	40.34	5.67	18.0
	8	16	0.125	4.87	5.43	8.11	61.86	4.38	18.3
	4	16	0.062	5.16	5.77	8.70	61.59	3.83	18.0
	8	8	0.062	6.85	7.78	11.75	81.21	4.99	18.3
	4	8	0.031	6.87	8.02	12.73	81.03	3.80	18.0
SLAE	8	32	0.250	7.92	8.35	10.55	42.40	6.76	7.4
	4	32	0.125	8.15	8.68	11.14	50.82	6.58	7.1
	4	16	0.062	8.68	9.36	12.79	69.59	5.18	7.1
	8	16	0.125	8.85	9.51	12.89	67.59	5.83	7.5
	4	8	0.031	9.93	10.92	15.61	80.63	5.70	7.1
	8	8	0.062	9.96	11.05	16.00	80.05	7.16	7.5

Table 9: Tucker surrogate results on the test set ($n = 1620$), sorted by median error within each family. $\rho = r_s r_z / (K d_z)$ is the compression ratio of the surrogate target relative to the direct pipeline. $\varepsilon_{\text{trunc}}$: median latent-energy discarded by the rank truncation (factors only). ε_{reg} : median relative error of the predicted core (surrogate only). **Lat**: GPU inference latency at batch size 1. Best median per family in bold.

Rank sensitivity: r_z dominates r_s . The rank grid of Table 9 is strongly anisotropic, and this is the most informative outcome of the sweep. Increasing the latent-mode rank r_z from 8 to 32 nearly halves the LLAE error ($6.85\% \rightarrow 4.33\%$ at $r_s = 8$), and the SLAE error falls in step ($9.96\% \rightarrow 7.92\%$). Doubling the frequency-mode rank r_s from 4 to 8, by contrast, barely moves either ($4.57\% \rightarrow 4.33\%$ for LLAE, $8.15\% \rightarrow 7.92\%$ for SLAE at $r_z = 32$), an effect an order of magnitude smaller. The rank sweeps of Figure 11 confirm the pattern distributionally: the r_z violins separate cleanly while the r_s violins overlap almost completely.

The interpretation reaches beyond this ablation. A rank-4 basis over the frequency mode already captures almost all of the joint variation of the $K = 16$ Laplace coefficients across the training set: the spectral representation is redundant enough that discarding three quarters of its dimensions costs nothing measurable. The $d_z = 64$ latent channels behave in the opposite way, carrying richer, less redundant information that a rank-8 basis truncates too aggressively. A tensor factorisation, fitted blind to the physics on training statistics alone, thus rediscovers the premise of this work from a direction independent of the argument that motivated the transform: the Laplace representation is intrinsically compact along the frequency axis, and $K = 16$ already sits close to its useful floor.

In practice $r_s = 4$ is the parameter-efficient choice, halving the surrogate size (1.35 M vs. 1.9 M parameters for $r_s = 8$) at negligible accuracy cost. One caveat bounds the claim: the frequency axis is sampled at two points only ($r_s \in \{4, 8\}$), so the sweep establishes insensitivity to r_s over that range, not saturation. The uncompressed limit $r_s = K$ is structurally the single-mode compression of Section 3.4.2, the two differing only in the nature of the basis, real and trained there, complex and frozen here.

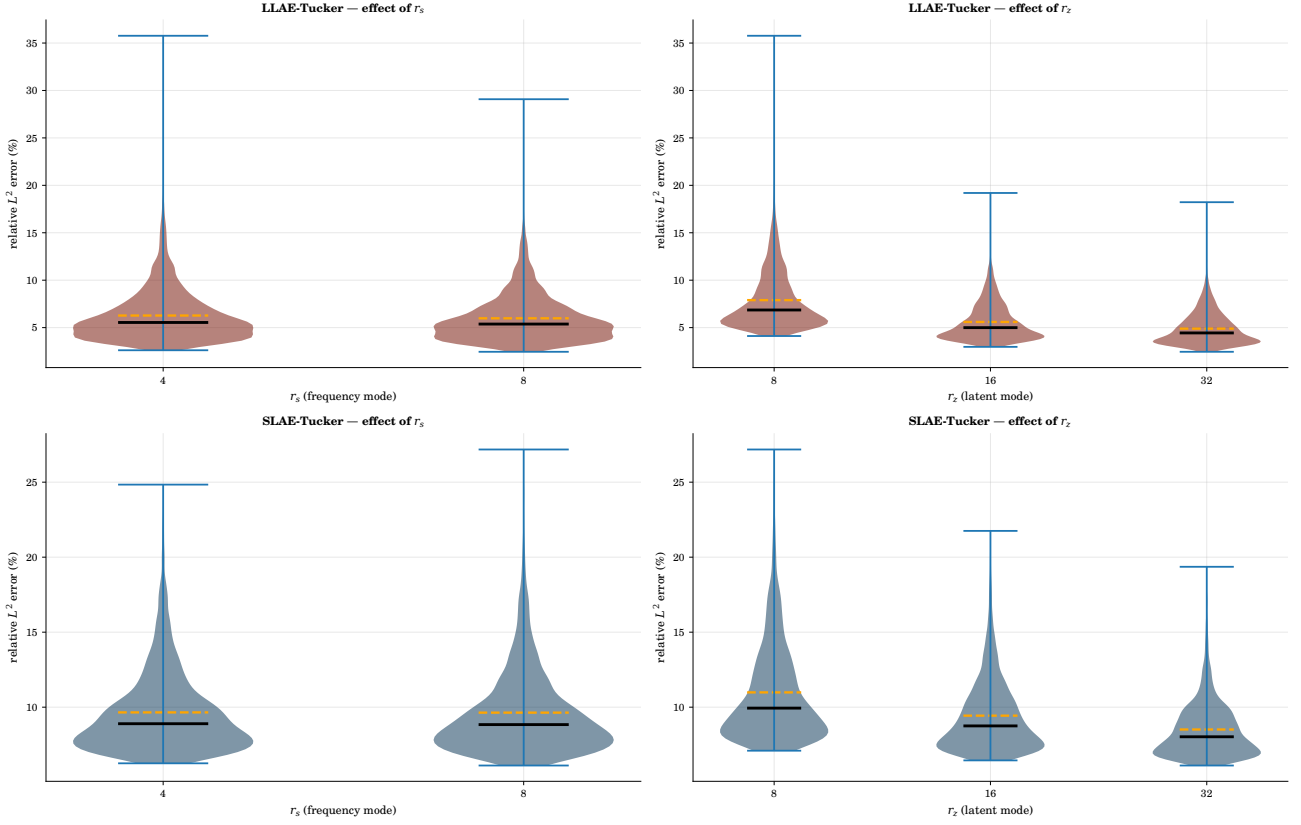


Figure 11: Tucker rank sweeps (test set, violin plots pooled over the other rank; black line = median, dashed orange = mean). Top row LLAE-Tucker, bottom row SLAE-Tucker; left column effect of r_s , right column effect of r_z .

Truncation vs regression. The two decoder-free diagnostics of Table 9, shown in Figure 12, locate the accuracy bottleneck. The regression error ε_{reg} stays small throughout (3.8–7.3%) and grows only mildly as the core enlarges ($r_z = 32$ being harder to fit than $r_z = 8$): the MLP predicts the compact core reliably. The truncation error $\varepsilon_{\text{trunc}}$ is large and is what the field error tracks, falling from $\sim 81\%$ at $r_z = 8$ to $\sim 37\%$ at $r_z = 32$ (LLAE) in step with the field error. It is a **latent-space energy**, not a field error: discarding 80% of the latent norm costs only $\sim 7\%$ on $U(t)$, the removed Tucker directions carrying little decodable information, so its **variation** with the ranks rather than its absolute value governs accuracy. The Tucker compression, not the surrogate, is therefore the binding constraint, and any accuracy gain must come from a richer latent basis (larger r_z) rather than a larger predictor.

Compression trade-off. Figure 13 plots the field error against the compression ratio ρ and against the surrogate size. Along the Pareto front, LLAE-Tucker degrades gracefully as the target is compressed: $r_s = 4$, $r_z = 32$ retains 4.57% median error at $\rho = 0.125$ (an $8\times$ smaller target than the direct surrogate), and $r_s = 4$, $r_z = 16$ still reaches 5.16% at $\rho = 0.062$ ($16\times$). Tucker therefore offers a tunable accuracy/compactness dial that the direct and SVD surrogates lack: when the surrogate output must be kept small, whether for storage, for a lightweight predictor or for downstream many-query use, Tucker sustains near-direct accuracy at a fraction of the target dimension. It does not, however, **beat** the direct surrogate on raw accuracy; as with SVD (Section 3.4.2), reducing the latent space before the surrogate discards information the direct predictor is capable of learning.

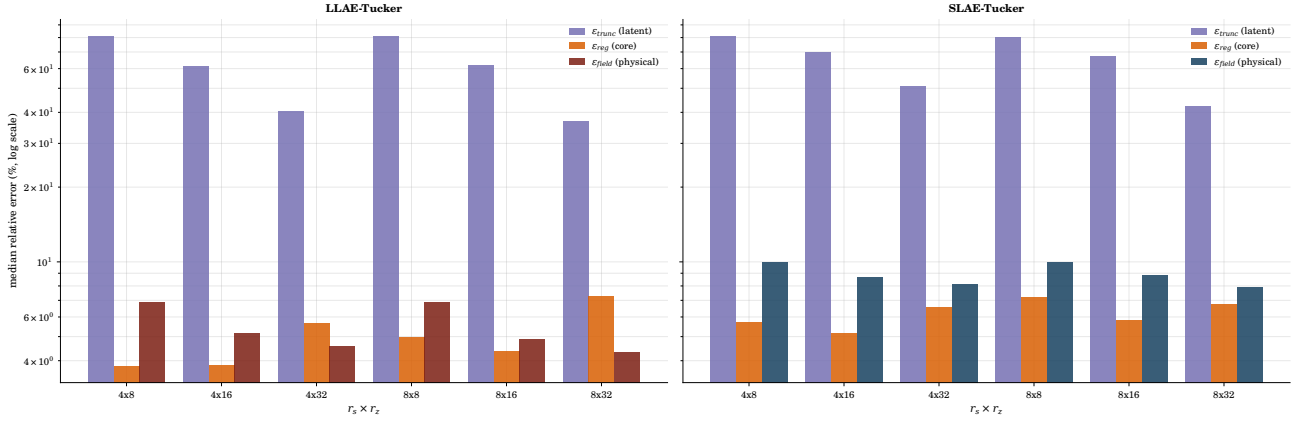


Figure 12: Error decomposition for LLAE-Tucker (left) and SLAE-Tucker (right): median latent truncation error $\varepsilon_{\text{trunc}}$, core regression error ε_{reg} , and physical-field error $\varepsilon_{\text{field}}$ for each (r_s, r_z) pair (log scale).

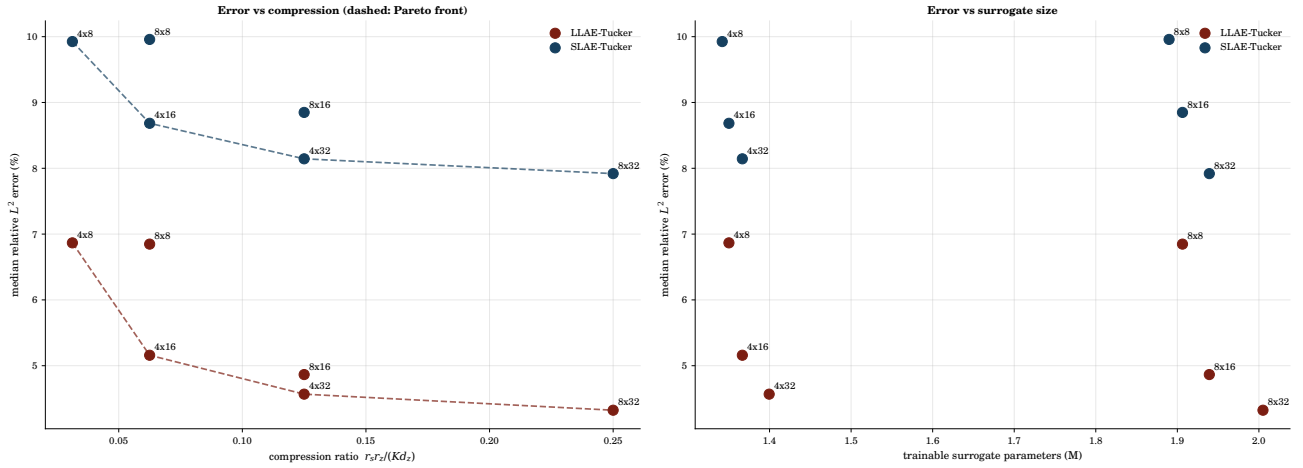


Figure 13: Left: median field error vs. compression ratio $\rho = r_s r_z / (K d_z)$ (dashed line = Pareto front per family; points sharing a ratio differ in (r_s, r_z)). Right: median field error vs. trainable surrogate parameters.

Decoder co-adaptation. Because the decoder is jointly updated during surrogate training ($\text{lr_decoder}=5 \times 10^{-5}$), it can co-adapt to the distribution of **predicted** cores rather than the true ones. We probe this by decoding the exact projected core in place of the prediction: had the decoder not co-adapted, the exact core would yield a lower error and serve as a valid oracle bound. For LLAE-Tucker the substitution is neutral (+0.07 pp) and the bound holds; for SLAE-Tucker it **raises** the error sharply (+11.1 pp on the probe subsample at $r_s = 8$, $r_z = 16$), the decoder having specialised to the predicted-core distribution, exactly the mechanism behind SLAE’s negative surrogate Gap in Section 3.4.1. A physical-space oracle is therefore not a valid lower bound for SLAE-Tucker, which is why the diagnostics above are computed in the latent space, where they remain decoder-independent.

Inference cost. The Tucker back-projection $(U^{(s)} \hat{\mathcal{G}} U^{(z)})^\top$, two small matrix products) is negligible in the inference budget, and the measurements make this unusually clear: across the twelve checkpoints, the FLOP count is **constant** within each family (30.3 GFLOPs for every SLAE-Tucker and 48.2 for every LLAE-Tucker), although the ranks vary by a factor of eight in r_z and two in r_s . Compressing the surrogate target changes what the predictor must learn, not what the decoder must compute. Latency follows suit (Figure 14): ~ 15 ms for SLAE-Tucker and ~ 24 ms for LLAE-Tucker at batch size 1, essentially independent of the ranks and within 3% of the direct pipelines of Section 3.4.1. SLAE-Tucker thus keeps the $1.6\times$ arithmetic and $1.6\times$ wall-clock advantage of its lighter decoder, at the cost of the higher error floor discussed above, and the rank reduction, whose entire purpose was to lower cost, buys none.

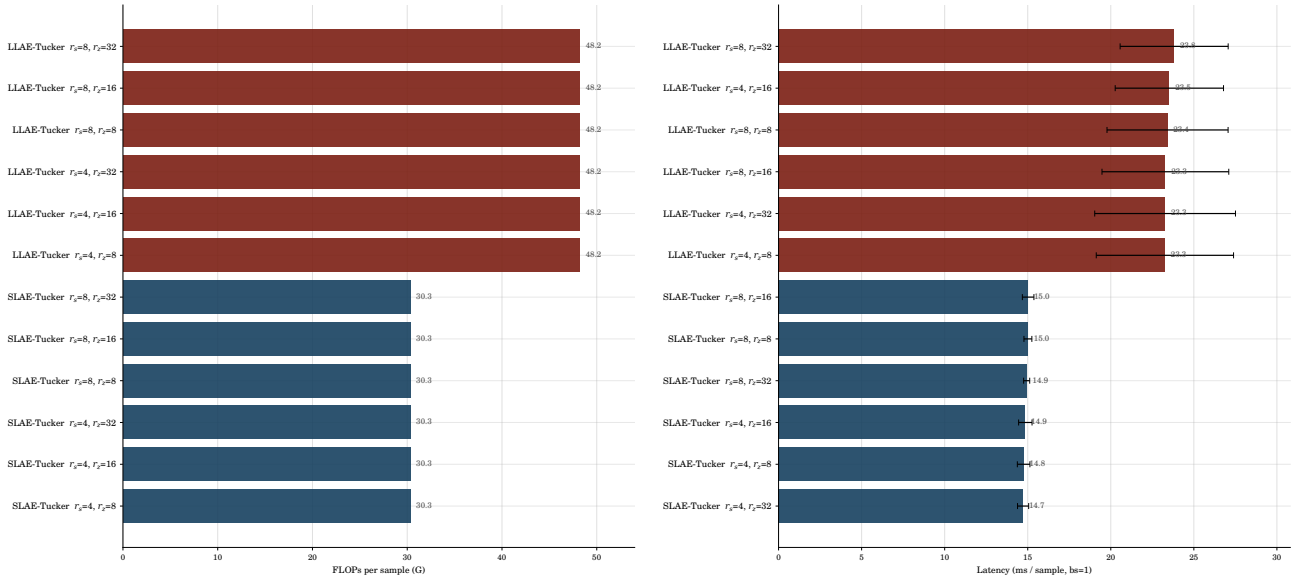


Figure 14: Tucker surrogate inference benchmark (50 CUDA Event timed runs, batch size 1). Left: per-checkpoint latency. Right: accuracy–latency scatter.

Per-checkpoint error histograms for all 12 Tucker surrogates are provided in Appendix A (Figure 25).

3.5 Choosing the Laplace contour

Every result reported so far rests on the prescribed triplet (s_k, α_t, λ) of Section 2.4. Six configurations are trained against it, one per (family, K) pair with $K \in \{8, 16, 32\}$ and $d_z = 64$: LLAE with the latent learnable contour, SLAE with the offline spatial one.

Architecture, optimiser, schedule and data partition are held at the values of Sections 3.3–3.4.1, so each configuration differs from its prescribed counterpart by the triplet alone. The offline triplet is selected before training and then held fixed throughout both phases, defining the Laplace-domain dataset, the inversion inside the surrogate and the inversion at inference alike; its minimisation settings are given in Appendix G. The latent variant is treated in Appendix E and the discussion below concerns SLAE.

Geometry of the optimised contour. Figure 15 shows the optimised sampling points against their prescribed counterparts, the bottom row being the offline contour. Its abscissae leave the imaginary axis for a positive real part ($\sigma_k \approx 0.035$), yielding a genuine Bromwich contour rather than a purely oscillatory one, while the imaginary parts dilate and widen the sampled spectral window. It also retains a heavy ridge, $\lambda \approx 4.3 \times 10^{-3}$ at $K = 16$ rising to 8.9×10^{-2} at $K = 32$, which is what the criterion is built to do: (31) charges the amplification factor $\|A^{-1}F^H\|$ against a nominal predicted-coefficient error, so conditioning is worth buying with bias at that magnitude.

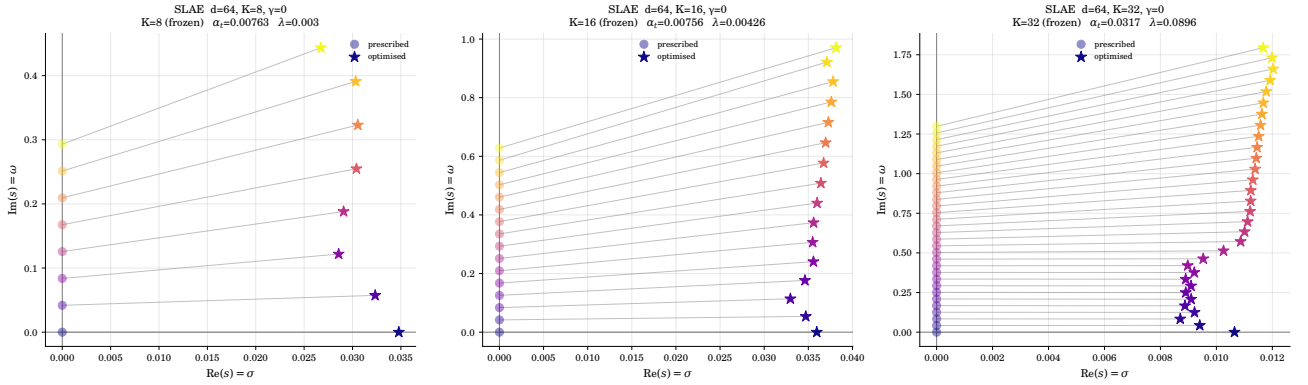


Figure 15: SLAE offline spatial contours in the complex plane ($d_z = 64$, $K \in \{8, 16, 32\}$). Circles: prescribed sampling points on the truncated Bromwich contour; stars: optimised points, joined to their prescribed position by a grey segment; colour encodes the frequency index k . The associated (α_t, λ) are reported in each panel title.

Autoencoder stage. Table 10 reports both stages for the six configurations, with the variation induced relative to the corresponding prescribed contour at $\gamma = 0.01$. At the reconstruction stage the offline contour **degrades** the error floor, moderately at $K = 8$ and $K = 32$ (+0.65 and +0.17 pp) and pronouncedly at $K = 16$ ($8.93 \rightarrow 16.05\%$). The prescribed damping was itself swept at that K , and the reference matters: the best prescribed floor is not the one tabulated but 7.79% at $\gamma = 0$ (against 7.83 and 8.93% at $\gamma = 0.001$ and 0.01), so the penalty is +7.12 pp against the conventional damping and +8.26 pp against the best prescribed configuration. The reconstruction stage is not the deliverable, however, and it is not what the criterion scores: (31) weighs the amplification factor against a nominal coefficient-error magnitude, and a triplet selected at any such magnitude has no reason to be the one minimising the reconstruction floor. The surrogate columns accordingly reverse this ordering; the full distributions are given in Appendix E.

Surrogate stage. The surrogate columns of Table 10 invert the ordering, and Figure 24 shows the corresponding error distributions. For SLAE, the offline spatial contour reduces the end-to-end error at $K = 8$ ($11.62 \rightarrow 9.76\%$, -1.85 pp) and at $K = 32$ ($5.70 \rightarrow 5.04\%$, -0.66 pp), notwithstanding the reconstruction penalty incurred at each. At $K = 16$, where the prescribed damping was swept, the gain does not survive the comparison:

Family	Contour	K	AE med. (%)	Δ AE (pp)	Surr. med. (%)	Δ Surr. (pp)	Gap (pp)
LLAE	learnable	8	2.89	−0.39	4.54	+0.53	+1.65
	learnable	16	2.68	−0.45	4.63	+0.57	+1.95
	learnable	32	2.60	−0.52	4.58	+0.25	+1.97
SLAE	offline	8	12.92	+0.65	9.76	−1.85	−3.16
	offline	16	16.05	+7.12	7.46	−0.13	−8.59
	offline	32	8.53	+0.17	5.04	−0.65	−3.49

Table 10: Optimised-contour configurations ($d_z = 64$), test set ($n = 1620$). **AE med.:** standalone reconstruction error; **Surr. med.:** end-to-end surrogate error. The Δ columns give the variation with respect to the corresponding prescribed-contour configuration of Tables 4–6 at identical (d_z, K) and $\gamma = 0.01$; negative values indicate an improvement brought by the optimised contour. **Gap:** surrogate median – AE median. Best per family in bold.

7.46% against 7.59% at $\gamma = 0.01$, but 7.39 and 7.43% at $\gamma = 0$ and 0.001, so the optimised contour improves on the conventional damping and matches, without beating, the best prescribed configuration. The Gap column is negative at every SLAE configuration and reaches −8.59 pp at $K = 16$: the entire reconstruction-stage penalty is recovered once the surrogate is trained. At $K = 32$, SLAE attains 5.04%, the most accurate SLAE result of this study and a 43% reduction relative to the POD reference (8.92%), while retaining the reduced decoding cost characteristic of the family.

Why the two stages disagree. The sign flip is not a trade the pipeline makes, and reading it as one would mistake the meter for the quantity. The two stages are the same bound (25) evaluated at two levels of coefficient error, of which only the second is ever deployed. That bound is affine in the coefficient error ϵ , with both of its coefficients depending on the triplet and on nothing else (25), so two triplets cross at a single ϵ^* : below it the extra bias of one dominates, above it the lower amplification does. SLAE reconstructs at ϵ_{AE} and deploys at $\epsilon_{\text{surr}} \gg \epsilon_{\text{AE}}$, and the proxy (31) is minimised at a nominal magnitude fixed a priori (Section 2.4), at neither stage in particular: nothing in the criterion asks the optimised triplet to be better at ϵ_{AE} , and nothing forbids it either. What the measurement adds is that here it is not, ϵ^* falling between the two stages. The reconstruction penalty is therefore a meter the criterion never optimised rather than a price paid for the surrogate gain, and $K = 16$ shows how loosely the two levels constrain each other, +7.12 pp of floor against −0.13 pp of deployed error. Nor is the exchange the ridge term’s doing, $K = 32$ carrying the heaviest regularisation ($\lambda \approx 8.9 \times 10^{-2}$) for the smallest penalty (+0.17 pp) while $K = 16$ incurs +7.12 pp at $\lambda \approx 4.3 \times 10^{-3}$: bias and amplification alike are set by the joint geometry of (s_k, α_t, λ) . That the penalty is not merely amortised but recovered in full is a separate property, and it belongs to the bias term: $A^{-1}L_{\text{reg}}$ being linear and known, the decoder pre-compensates it as soon as Phase 2 introduces the physical-time term that SLAE’s Phase-1 objective lacks (Section 2.2.1) and LLAE’s (12) already contains.

What the offline criterion buys. The gain is real but uneven, and the uneven part is the informative one: −1.85 pp at $K = 8$, −0.66 at $K = 32$, and nothing beyond the best prescribed abscissa at $K = 16$, where the three prescribed values already span 0.20 pp

end-to-end (7.39 to 7.59%) and the optimised contour lands at 7.46%. The criterion helps where the prescribed contour is least adequate, that is where the spectral truncation is most severe, and is neutral where a damping sweep already suffices; on this problem it is not a general-purpose accuracy lever. That $K = 16$ comparison also bounds what the abscissa alone is worth, 0.20 pp against up to 1.85 for the full triplet, the residual being carried by the imaginary parts and the regularisation weights jointly (Section 2.4). Two properties are not in doubt: the selection requires no network training (500 gradient steps on a purely linear-algebraic functional over 100 instances, against a complete two-phase training per candidate had the selection been empirical), and it alters no tensor dimension, the optimised surrogates consuming exactly the arithmetic of their prescribed counterparts. The effect on the accuracy–cost trade-off is accordingly modest but free: SLAE at $K = 32$ attains 5.04%, within 1.2 pp of the most accurate configuration of the study where a prescribed contour left 1.9 pp.

4 Comparison and Discussion

4.1 Global synthesis

The three references of this study isolate one ingredient each. POD in the Laplace domain (Section 3.2.1) keeps the spectral compression and removes the non-linearity; the DL-ROM (Section 3.2.2) keeps the non-linearity and removes the spectral compression; SLAE and LLAE keep both. Table 11 places them side by side at a common operating point ($d_z = 64$, $K = 16$ where applicable, one full 150-step trajectory per query), with accuracy and the four cost axes measured on the same hardware in a single benchmarking run.

Model	Contour	Targets	N_{surr}	Lat. (ms)	FLOPs (G)	Mem. (MB)	Median (%)
POD ($k_{\text{pod}} = 64$)	prescribed	16	—	10.0	0.2	212	8.92
DL-ROM	—	150	23.50 M	31.4	48.3	927	3.80
LLAE	prescribed	16	3.48 M	22.7	48.2	622	4.06
	learnable	16	3.48 M	23.8	48.2	621	4.63
SLAE	prescribed	16	3.22 M	15.7	30.3	235	7.59
	offline	16	3.22 M	15.6	30.3	235	7.46

Table 11: Synthesis at the common operating point ($d_z = 64$, $K = 16$, $\gamma = 0.01$ for the prescribed contours), test set ($n = 1620$), one 150-step trajectory per query. **Targets**: number of coefficient blocks the surrogate must predict (K Laplace frequencies, or N_t time steps for the DL-ROM). N_{surr} : surrogate parameters, equation (37). Latency, FLOPs and peak memory are measured at batch size 1 on an idle GPU. The four deep rows come from one benchmarking run and are therefore mutually comparable; the POD row is measured separately by the same procedure. FLOPs, memory and parameter counts are deterministic across runs; latency varies by two to three percent. Best per column in bold, excluding accuracy. Median errors are those of Tables 2, 6 and 10.

Two orderings emerge, and they are not the same. On accuracy, the DL-ROM leads (3.80%), LLAE follows within half a point (4.06%), SLAE trails by a further three points (7.46–7.59%), and POD is last (8.92%). The non-linear autoencoder is what buys the bulk of the accuracy: every deep model improves on POD, by 15 to 57% at this operating

point and by up to 43% for SLAE once its contour is optimised at $K = 32$ (Section 3.5). Temporal compression, by contrast, buys nothing in accuracy: it costs a little, 0.26 pp for LLAE and around 3.7 pp for SLAE, the price of representing a 150-step trajectory by 16 complex coefficients and inverting a truncated spectrum.

On cost the ordering reverses, unevenly across axes. Among the deep models SLAE is the cheapest on all four (30.3 GFLOPs, 15.7 ms, 235 MB, 3.22 M parameters) and the DL-ROM the most expensive on all four. LLAE is intermediate, and the two axes describe it differently: it matches the DL-ROM’s arithmetic to 0.2% (48.2 against 48.3 GFLOPs, the shared decoder being essentially the whole computation) yet runs $1.38\times$ faster (22.7 against 31.4 ms) on $1.5\times$ less memory, its surrogate carrying 16 frequency heads instead of 150 temporal ones. The linear POD is cheaper than everything by a wide margin on arithmetic (0.2 GFLOPs, some $150\times$ below SLAE) but only $1.6\times$ faster in wall-clock time and no lighter in memory, the Tikhonov inversion being a floor no reduction in network cost can go below. Figure 16 gives the corresponding Pareto view. No model dominates: the frontier at 150-step output is occupied by the DL-ROM (most accurate, most expensive), LLAE (near-equal accuracy at a seventh of the parameters and $1.38\times$ faster) and SLAE (twice as fast for three points of error), POD being dominated on accuracy by all three. Section 4.2 explains why the latency axis behaves differently from the parameter axis, and Section 4.3 why this ranking holds only at a fixed output resolution.

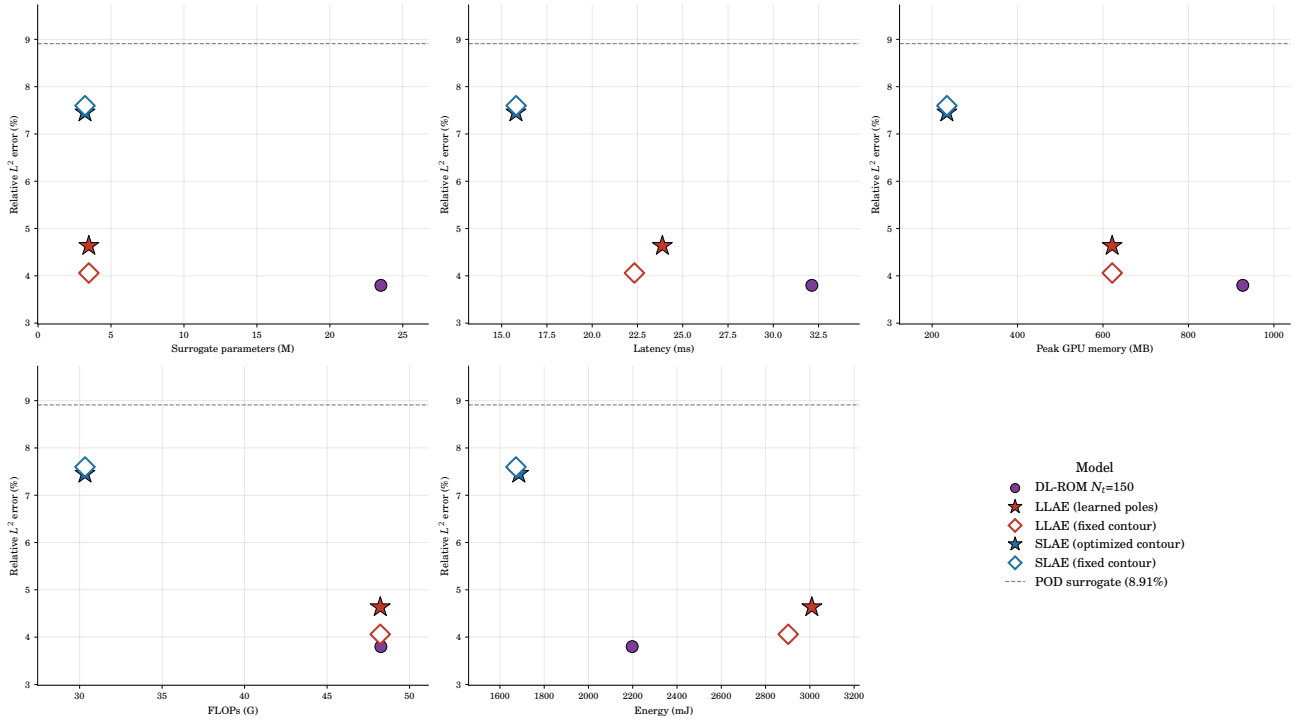


Figure 16: Accuracy against five inference-cost axes at fixed 150-step output (batch size 1): surrogate parameters, latency, peak GPU memory, FLOPs and energy. Bottom-left is best. Stars: optimised contours (LLAE latent learnable, SLAE offline spatial); open diamonds: prescribed contours; green disc: DL-ROM at $N_t = 150$. The dashed line marks the POD reference (8.92%).

4.2 Where temporal compression reaches

Equation (37) predicts that the Laplace surrogates are $6.8\times$ lighter than the DL-ROM, and Table 11 confirms it exactly. It does **not** predict a $6.8\times$ speed-up, and none is observed: LLAE runs $1.38\times$ faster than the DL-ROM it replaces while performing exactly as much arithmetic, its parameters falling to 0.15 of the DL-ROM’s while its FLOPs stay at 1.00. The pipeline spends its time in the decoder, not the surrogate, as already observed across the direct, SVD and Tucker variants (Sections 3.4.1–3.4.3): both models invoke the same convolutional decoder once per output time step, 150 passes each. Compressing the time axis **in latent space** shrinks what the MLP must regress, from 150 latent frames to 16 complex ones, but the inverse transform expands it back to 150 frames before decoding, leaving the dominant term of the forward pass untouched — hence 48.2 against 48.3 GFLOPs, an agreement to 0.2% that identifies the shared decoder as essentially the entire computation. The residual differences follow from the surrogate and favour LLAE: $1.5\times$ less peak memory (622 against 927 MB, the 20M parameters it does not allocate) and $1.38\times$ less wall-clock time (22.7 against 31.4 ms), the DL-ROM evaluating 150 per-time-step heads where LLAE evaluates 16 per-frequency ones. That the two differ in latency while agreeing in arithmetic is why we report both: at equal FLOPs, the model whose predictor is split into ten times fewer kernels is the faster one, and no operation count can express that. Energy tracks latency closely (2.88 against 2.89 J), the board drawing near-constant power over these millisecond kernels.

SLAE is the configuration in which temporal compression does reach the decoder. Applying the transform in the physical domain leaves $K = 16$ complex frequency frames to decode instead of 150 temporal ones, so the whole forward pass, and not merely the surrogate, is $O(K)$: 30.3 GFLOPs against 48.3, a $1.6\times$ reduction in arithmetic that this time **does** convert into wall-clock time, with $2.0\times$ lower latency (15.7 against 31.4 ms), $3.9\times$ lower peak memory (235 against 927 MB) and $1.7\times$ lower energy at identical 150-step output. The three points of accuracy this costs are the counterpart, the decoder never seeing a physical time frame and the trajectory being reconstructed from a 16-point spectrum; the offline spatial contour of Section 3.5 recovers part of it, down to 5.04% at $K = 32$ while retaining a $1.6\times$ speed advantage. The principle is that a Laplace-domain pipeline is cheap on the axes its compressed representation actually reaches: latent-space compression reaches the surrogate alone, delivering the parameter and memory savings and leaving the arithmetic unchanged, whereas physical-domain compression reaches the decoder as well and converts into latency, FLOPs and energy. Reporting parameter counts alone, the natural reading of equation (37), would have credited LLAE with a $6.8\times$ gain that no user would ever measure, an instance of the efficiency-misnomer effect documented by Dehghani et al. [3]; hence the several axes reported jointly in Table 11 and Figure 16.

The cost of a fixed output grid. All the comparisons above are stated at a fixed output resolution of 150 steps, and that qualification is not cosmetic. The DL-ROM has exactly one lever for reducing its temporal cost, producing fewer time steps, and sweeping the stride κ over the training grid (Section 2.5.2) traces that lever, one trained model per point (Table 12). The sweep is informative in two ways. First, the DL-ROM’s per-step accuracy is flat: 3.73 to 3.97% across a tenfold reduction of N_t , with no monotone trend. Subsampling the time grid does not make the individual frames harder to predict, the shorter latent trajectory being if anything slightly easier, so the $9\times$ cost reduction from $\kappa = 1$ to $\kappa = 10$ is bought purely by delivering fewer steps. Second, the parameter-matched point falls at $\kappa = 10$: at 3.06M parameters the DL-ROM surrogate is the size of

LLAE at $K = 16$ (3.48M), both compressing the time axis roughly tenfold, uniformly on one side and spectrally on the other. The difference is what comes out: 15 time steps for one, the complete 150-step trajectory for the other, at 4.06% median error. Recovering the missing 135 steps from the DL-ROM requires interpolating between samples 10 time units apart, or retraining at a finer stride.

Model	κ	Output steps	N_{surr}	Lat. (ms)	FLOPs (G)	Median (%)
DL-ROM	1	150	23.50 M	31.4	48.3	3.80
DL-ROM	2	75	12.15 M	15.8	24.1	3.97
DL-ROM	3	50	8.36 M	10.6	16.1	3.84
DL-ROM	5	30	5.33 M	6.7	9.7	3.86
DL-ROM	10	15	3.06 M	3.8	4.8	3.73
LLAE ($K = 16$)	—	any	3.48 M	Fig. 17	4.8–48.2	4.06
SLAE ($K = 16$)	—	any	3.22 M	Fig. 17	30.2–30.3	7.59

Table 12: Temporal-resolution sweep of the DL-ROM against the two Laplace pipelines. Latencies come from the same benchmarking run as Table 11. Each DL-ROM row is a separately trained model; the two Laplace rows are a **single** checkpoint each, queried at output resolutions $N_t \in \{15, 30, 50, 75, 150\}$, the FLOPs range spanning that interval and the corresponding latency profile being given in Figure 17. Errors are measured on each model’s own output grid (the DL-ROM at stride κ against the ground truth subsampled at the same stride), with no interpolation. The two Laplace medians are those measured at the 150-step output; accuracy at coarser grids is reported in Table 13.

One model, any Δt . That asymmetry is structural rather than incidental. The Laplace coefficients $\hat{z}(s_k)$ predicted by the surrogate carry no reference to any time grid; the trajectory is materialised by the Tikhonov solve (3)–(4), whose matrix $\mathbf{F}_{k,n} = \Delta t w_n e^{-s_k t_n}$ is assembled from whatever nodes t_n are requested. A single trained model therefore answers at any output resolution, any Δt at fixed horizon T , for the cost of reassembling a small $K \times N_t$ matrix, whereas the per-time-step heads of the DL-ROM surrogate (equation (37)) are tied to their training grid.¹ The resulting cost profiles (Figure 17) again separate the two families: LLAE keeps a constant surrogate (3.48M parameters at every N_t) but decodes N_t frames, so its arithmetic follows the DL-ROM curve exactly, 4.8 to 48.2 GFLOPs, while SLAE decodes $K = 16$ frames whatever is asked and is flat on every axis, 30.2–30.3 GFLOPs from $N_t = 15$ to 150. Below $N_t \approx 100$ the DL-ROM and LLAE are cheaper in FLOPs than SLAE, above it SLAE wins with a margin growing without bound: refining the output grid is free for SLAE, and the crossover is a property of the decoder, not of the transform.

This is where temporal Laplace compression is decisive rather than merely economical. The five DL-ROM rows of Table 12 are five trainings and five fixed answers to a question often settled only after deployment, and differently for each downstream use: at what temporal resolution is the trajectory needed? The two Laplace rows are one training each, answering the entire range at inference time. For a many-query setting in which a coarse

¹A DL-ROM whose surrogate takes t as an input coordinate rather than through indexed heads, as in [5], is queryable off-grid; its **cost** nonetheless remains proportional to the number of requested steps, one network evaluation per step, which is the property at issue here.

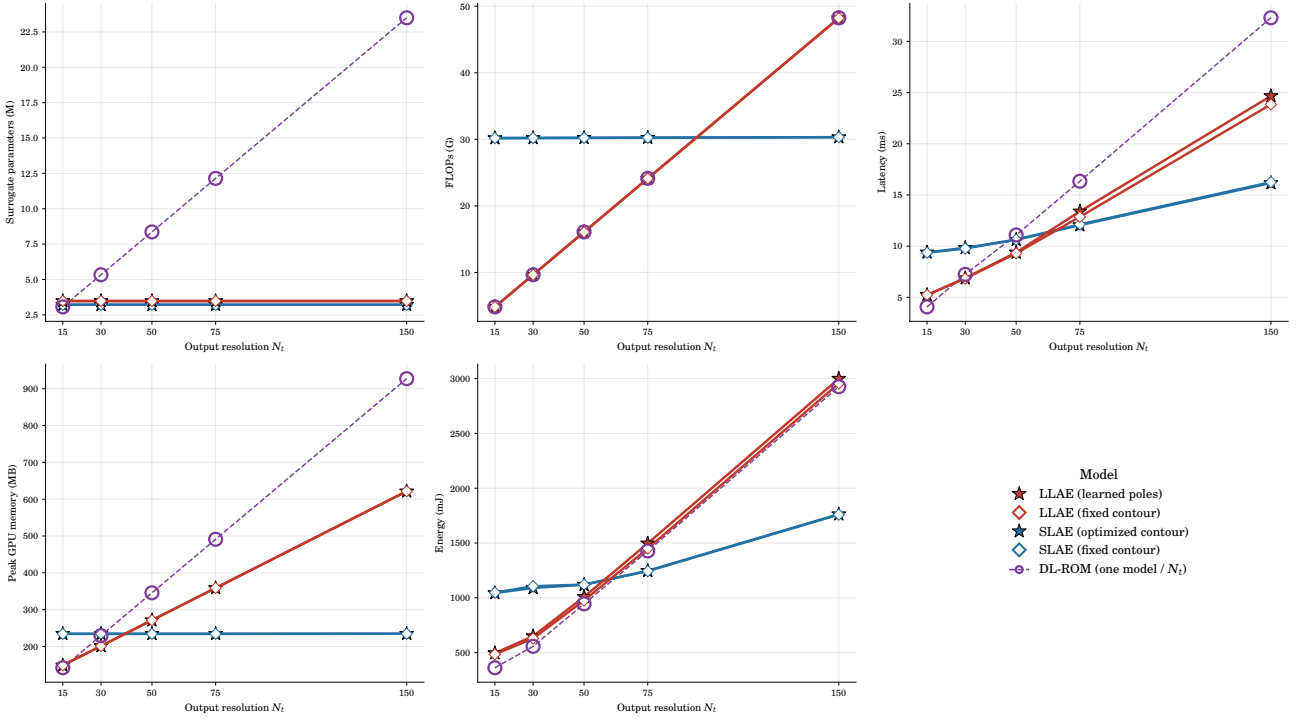


Figure 17: Inference cost against requested output resolution N_t at fixed horizon T (batch size 1). Solid lines: a **single** Laplace checkpoint per curve, queried at each N_t . Dashed line with open circles: the DL-ROM, one separately trained model per point, each evaluable only at its own N_t .

scan precedes a refined analysis of a few selected parameter values, that flexibility, and not the parameter count, is the operative advantage.

4.3 Accuracy across output resolutions

Cost was reported at a fixed 150-step output; we now sweep the requested resolution $N_t \in \{15, 30, 50, 75, 150\}$ at fixed horizon T , querying the same $K = 16$ checkpoints and comparing against the DL-ROM, which requires one trained model per resolution. The comparison sets a specialised model against one evaluated away from its training grid, which is the property at issue rather than a confound. A sampling floor bounds the exercise from below, and it is known before any measurement. The contour carries frequencies up to $\omega_{\max} = 2\pi(K - 1)/(N_t^{\text{train}}\Delta t)$, here 0.628 rad per step, a period of ten steps. Representing that component on the output grid requires at least two samples per period, hence $\Delta t \leq \pi/\omega_{\max}$ and

$$(42) \quad N_t \geq \frac{N_t^{\text{train}} \omega_{\max}}{\pi} = 2(K - 1),$$

that is $N_t \geq 30$ for a prescribed contour at $K = 16$; an optimised contour, which displaces the sampling points outward, raises its own floor accordingly. Below it the fastest encoded component is aliased and no inversion can recover it, whatever the model. This is a property of the representation rather than a limitation of the pipeline, and it is the structural price of encoding the transient spectrally: the resolution floor is set by the number of Laplace coefficients, so a model trained at $K = 8$ would serve down to $N_t = 14$. The measurements follow (42) closely, every configuration collapsing at $N_t = 15$ by one to two orders of magnitude, and we report that regime only to locate the floor.

Clear of its own floor, every configuration is essentially resolution-independent, and this holds for both families (Table 13). At any resolution at least $1.25\times$ its floor, no configuration departs from its 150-step error by more than 0.16 pp; LLAE under a prescribed contour is even marginally more accurate at $N_t = 50$ (4.03%) than at full resolution (4.06%). Within that margin a single checkpoint serves the whole range at no cost our protocol resolves, which for the prescribed contours means a threefold reduction of the delivered grid. Degradation appears only on approaching the floor, and stays moderate there: +1.98 pp for the offline spatial contour at $N_t = 50$, a mere 8% above its floor, and +1.24 and +0.30 pp for the two prescribed contours sitting exactly at theirs at $N_t = 30$. The ordering between families is unaffected throughout, and widens slightly as the grid coarsens, LLAE leading by 3.53 pp at $N_t = 150$ and by 4.47 pp at $N_t = 30$. What separates the configurations is therefore not a rate at which accuracy decays with resolution — there is none — but where each one’s floor sits.

The optimised contours raise their own floor, and this is what sets the usable range apart across the rows of Table 13. Both procedures dilate the imaginary parts of the sampling points (Section 3.5), so they encode faster components and, by (42), require a finer output grid: $\omega_{\max}$ rises from 0.628 under the prescribed contour to 0.838 for the LLAE learnable contour and 0.971 for the SLAE offline one, moving the floor from $N_t = 30$ to 40 and 46 respectively, and the ordering of the floors predicts the ordering of the failures exactly. Already at $N_t = 50$ the offline spatial contour is the only configuration within reach of its floor, and the only one to degrade (9.44%, against 7.75 to 4.03% for the other three). At $N_t = 30$ the two prescribed contours sit at their floor and hold (4.36 and 8.83%), the latent learnable contour is below its own and degrades (5.93%), and the offline spatial contour, furthest below, collapses (93.2%). What Section 3.5 presents as accuracy bought at no inference cost is therefore bought at a cost in resolution range: the wider spectral window that lowers the error at $N_t = 150$ is exactly what makes the representation unserviceable on a coarse grid.

Rescaling the Tikhonov terms to the new grid ($\alpha_t \sim 1/\Delta t$, $\lambda \sim \Delta t$, which preserves the underlying continuous functional) recovers part of the sub-floor gap (631% against 1034% at $N_t = 15$ for the offline spatial contour, 93.2 against 105.7 at $N_t = 30$) and no more; for every configuration above its floor it is worth at most 0.66 pp. This is the expected signature of aliasing rather than of misweighted penalties: the rescaling restores the functional being minimised, not the information the output grid can no longer carry.

Model	Contour	$N_t = 150$	75	50	30	15
SLAE	prescribed	7.59	7.66	7.75	8.83	61.4
	offline	7.46	7.52	9.44	93.2	632
LLAE	prescribed	4.06	4.12	4.03	4.36	49.0
	learnable	4.63	4.74	4.79	5.93	84.5

Table 13: Median relative L^2 error (%) against requested output resolution, at fixed horizon T , test set ($n = 1620$). A single $K = 16$ checkpoint per row is queried at every resolution, with the Tikhonov terms rescaled to the output grid. Ground truth is the dataset subsampled on the same grid; no interpolation is involved. $N_t = 15$ lies below the sampling floor (42) and is reported to locate it. Best per column within each family in bold.

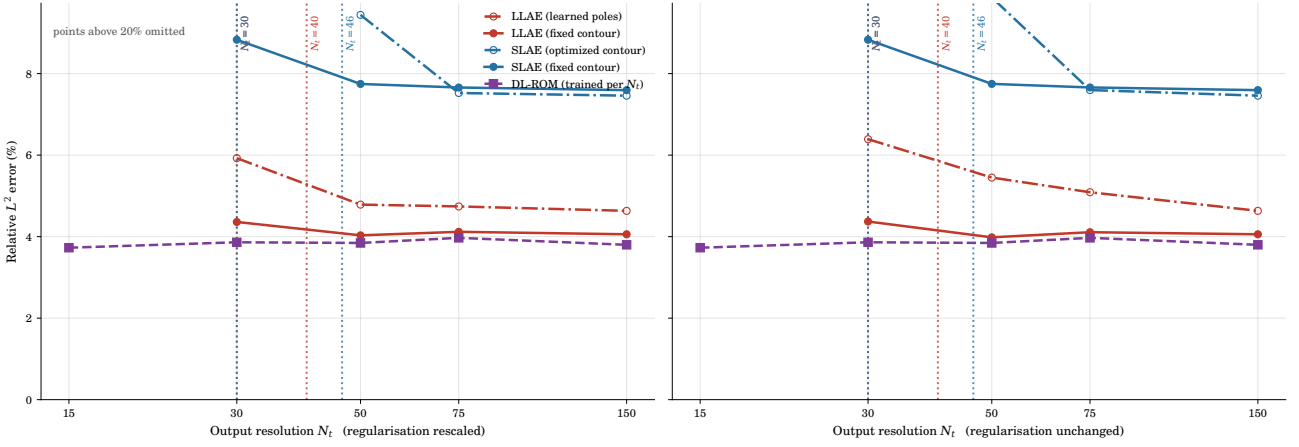


Figure 18: Relative L^2 error against requested output resolution at fixed horizon T (test set, $n = 1620$). Solid lines: a single Laplace checkpoint queried at each resolution. Dashed: the DL-ROM, one trained checkpoint per resolution. The dotted vertical lines mark the sampling floor (42) of each contour, computed from its own stored poles: $N_t = 30$ for the prescribed contours, 40 for the LLAE learnable one and 46 for the SLAE offline one. Left: Tikhonov terms rescaled to the output grid; right: left unchanged.

4.4 Positioning with respect to related work

Our pipeline reuses well-established components and combines them along a new axis. The **spatial** compressor is a convolutional encoder–decoder applied directly to the field, i.e. the deep convolutional-autoencoder ROM of Lee and Carlberg [18] and of the original DL-ROM [5]; we deliberately omit the preliminary POD reduction of POD-DL-ROM [6], which is an efficiency device (it lowers the autoencoder’s training cost) rather than an accuracy device, and is unnecessary for a convolutional network operating on a structured grid. The **linear** baseline against which we measure the nonlinear gain is a POD basis followed by a parameter-to-coefficient network, that is, the non-intrusive POD-NN of Hesthaven and Ubbiali [14], here applied frequency-by-frequency in the Laplace domain. Our optional SVD and Tucker stages (Sections 2.3.2–2.3.3) also use linear/multilinear projections, but in a role opposite to POD-DL-ROM: they act **after** the nonlinear autoencoder, in the latent space, to shrink the surrogate target, a post-compression rather than a spatial pre-reduction.

What distinguishes our approach is not the spatial compressor but the treatment of **time**. All the DL-ROMs above keep the full temporal resolution of the trajectory: the reduced dynamics are queried once per time step through a (μ, t) -conditioned network [5, 6] or propagated sequentially in time [25], so the temporal work scales with N_t . We instead compress the time axis itself with the Laplace transform, predicting $K \ll N_t$ spectral coefficients in a single shot and recovering the full trajectory by a cheap linear inversion. This exploits the same spectral intuition as Laplace-domain linear ROMs [28, 9] and the Laplace Neural Operator [1], but transposes it into the **nonlinear, parameter-to-field** DL-ROM setting, where, to our knowledge, temporal Laplace compression has not previously been used. The Laplace stage is therefore orthogonal to the spatial-compression choices above and could in principle be layered on top of any of them, including POD-DL-ROM.

Relation to neural operators. The learnable-contour variant of LLAE (Section 2.4) resembles operator-learning methods along two axes. First, it applies its spectral trans-

form **inside a learned low-dimensional latent space** rather than on the physical field, exactly the design principle of the Latent Neural Operator [29]: encode to a compact latent, act there, decode back. Second, learning the complex poles $s_k = \sigma_k + i\omega_k$ end-to-end is a latent-space counterpart of the learnable pole–residue representation at the heart of the Laplace Neural Operator [1], the dynamics being captured in both cases by a small set of trainable poles rather than by a fixed spectral grid or explicit time-stepping. The surrogate reinforces the analogy: its shared trunk conditioned on a sinusoidal encoding of s_k (Figure 2) maps $(\mu, s_k) \mapsto \hat{z}(s_k)$, so the Laplace frequency plays the role of the output coordinate much as the trunk coordinate does in a DeepONet [20]. The difference is one of **setting**, not of mechanism: genuine neural operators are field-to-field maps between function spaces, discretization-invariant in their input, whereas our input is a finite parameter vector μ and our decoder a fixed-resolution convolutional network. LLAE with a learnable contour is thus best understood as a **parameter-conditioned, latent Laplace operator**, importing the learnable-pole and latent-operator ideas of [1, 29] into the DL-ROM regime, rather than as a neural operator in the strict sense.

4.5 Discussion and perspectives

What the ablations attribute to what. The two references were chosen to remove one ingredient each, and read together they place the credit unambiguously. Removing the non-linearity costs a great deal: POD in the Laplace domain sits at 8.92% against 3.80–4.63% for every deep model, and no amount of additional POD modes closes the gap, $k_{\text{pod}} = 128$ improving the reconstruction floor to 7.04% while **degrading** the deployed error to 11.30%. Removing the Laplace transform costs nothing our measurements resolve: 0.04 pp separate the DL-ROM from LLAE, below the threshold this study treats as significant. SLAE is the one family where the deficit is real, and the confound examined below attributes a substantial share of it to a Phase-1 objective mismatch rather than to the transform.

The substantive findings lie on the cost and flexibility side. The first is that **where** the transform is applied decides which cost axes benefit, and that the parameter count, the quantity equation (37) predicts, is the axis least representative of what a user measures: LLAE is 6.8× lighter than the DL-ROM in surrogate parameters yet performs exactly as much arithmetic, both decoding N_t frames through the same backbone, whereas SLAE, whose decoder sees only the K frequency frames, converts the compression into 2.0× latency and 3.9× memory at identical output. A Laplace pipeline is cheap on exactly the axes its compressed representation reaches, and no further.

The second is that the regularized inversion, the poles **and** the weights (α_t, λ) that go with them, fixed by convention in every Laplace-domain reduced-order model we are aware of, is a design object of first order, optimisable for free, and one that must be scored on the deployed error rather than on the reconstruction floor. What makes it optimisable is that the bound (25) draws no distinction between placing a sampling point and weighting a penalty: both scalars depend on the triplet alone, so a candidate is scorable without training anything. The two strategies displace the sampling points comparably but are optimised against different objectives, one against the autoencoder’s own training loss and the other against a proxy scored at a nominal coefficient-error magnitude, and only the second improves the emulator: the offline spatial contour gains up to 1.85 pp end-to-end **while degrading** reconstruction by up to 7.12 pp, whereas the learnable route, the one LLAE’s latent contour affords cheaply, produces the best Laplace autoencoder of the study (2.60%) without our being able to turn it into a better surrogate

(Appendix E). The offline gain is uneven and modest, 1.85 pp where the truncation is severest and nothing at $K = 16$ once the prescribed abscissa is itself swept, but it is the only lever in this study that costs nothing, in training or at inference.

The third is a negative result: compressing the latent target before the surrogate, by SVD along one mode or by Tucker along two, buys neither accuracy nor latency. It shrinks the prediction target by up to $4\times$ for a fraction of a point, useful only when the surrogate rather than the decoder is the binding constraint. Its value here is diagnostic: that a rank-4 basis over the frequency mode is as good as a rank-8 one says the $K = 16$ Laplace coefficients are already close to their useful floor, evidence from a tensor factorisation fitted blind to the physics, independent of the argument that motivated the transform.

Limitations. Several qualifications bound the above, and we state them in decreasing order of consequence.

Cost measurements. Every cost figure in this paper was collected on the same idle GPU (Appendix G) with a single instrumentation path (CUDA Events for latency, a counter over the executed graph for FLOPs, and the allocator’s high-water mark for memory), so the numbers are mutually comparable across sections. Two caveats nonetheless bound what should be read into them. First, wall-clock latency measures an implementation as much as an architecture: at the reconstruction stage the latency ratios are compressed relative to the arithmetic ones (Table 5), the fixed cost of the forward transform and its inverse being invisible to a FLOP count. The surrogate stage, which applies no forward transform, is free of that asymmetry, and it is the stage the conclusions rest on. Second, energy is reported for completeness but carries little independent information at this scale: the board draws near-constant power over kernels of a few milliseconds, so the energy column is proportional to latency to within a few percent and should not be read as a separate axis.

Structure of the design. As set out in Section 3.1, the parameter design is full factorial, 225 points in (k, C) crossed with 36 injection angles, and partitioned uniformly over the product grid. This is a statement about the shape of the design, not about the validity of the instances, each of which is an exact solution at its own parameter value. Its consequence is that the test error measures interpolation at held-out points of a dense design rather than extrapolation to a (k, C) pair outside it, and that the aggregate is weighted towards the ϕ direction, which the factorial resolves 36 ways against 225 for the (k, C) plane. A per-direction error breakdown, or a second partition holding out whole (k, C) pairs, would separate the two; both are cheap and neither was done here.

Attribution within the triplet. The offline criterion moves the poles and the two regularization weights at once, so the gains of Section 3.5 are attributable to the triplet as a whole. The damping sweep bounds the abscissa’s share at 0.20 pp of the 1.85, but separating the imaginary parts from (α_t, λ) would require an ablation freezing the weights while optimising the poles alone, cheap since no network is involved and not done here.

Statistical resolution. Every number comes from a single training run per configuration: medians over 1,620 test instances resolve the error well with respect to test sampling, but we have no estimate of run-to-run variability under reseeding. We therefore apply one standard uniformly, that differences below roughly one point are unresolved by our evidence rather than measured effects. This covers the SVD and Tucker variants (+0.18 and +0.27 pp for LLAE) and the 0.04 pp separating LLAE from the DL-ROM,

which is accordingly not a penalty of the temporal compression, nor a claim of equivalence, which repeated runs alone could support. Differences above a point, the POD gap, the SLAE deficit, the contour gains and every cost ratio, are unaffected.

Confounds in the SLAE/LLAE comparison. The two families do not share a Phase-1 objective: LLAE is supervised on the physical-time reconstruction, SLAE only in the Laplace domain, by design, since a physical-time loss would require N_t decoder passes and negate SLAE’s advantage. Their batch sizes differ for the same reason. The comparison is therefore between two complete design points, not between two placements of the transform holding everything else fixed, and the gap we measure conflates the two. That the SLAE Gap is negative in every configuration, the surrogate phase which finally introduces a physical-time term systematically improving on the standalone autoencoder by up to 8.59 pp under an optimised contour, indicates that a substantial share of SLAE’s deficit is objective mismatch rather than architectural limitation. The control we did not run is the informative one: an SLAE trained in Phase 1 with a physical-time term on a subsampled set of frames, which would recover most of the supervision at a fraction of the cost.

Choice of DL-ROM formulation. Our baseline gives the DL-ROM surrogate one head per time step, which is what makes it 23.5M parameters against 3.48M for LLAE. A DL-ROM whose surrogate takes t as an input coordinate, as in [5], would be far smaller in parameters while paying one network evaluation per requested step. The $6.8\times$ parameter advantage is thus specific to the head-indexed formulation and should not be read as a general property of temporal compression; the FLOPs, latency and memory comparisons, which are decoder-dominated, are insensitive to this choice and are the ones to rely on.

Range of the resolution claim. A single checkpoint serves any output grid only above the sampling floor (42) set by its own poles, and the optimised contours raise that floor (Section 4.3). The flexibility is therefore real but bounded, and the bound is the price of the accuracy the optimised inversion buys.

Scope. All results concern one two-dimensional convection–diffusion–reaction problem with $p = 3$ parameters and a single output field, on a structured grid, over a fixed horizon. The proxy (31) is moreover calibrated at an assumed magnitude of surrogate error through λ_{AE} , held at one value throughout, so neither the sensitivity of the selected triplet to that setting nor the share of the gain attributable to the poles rather than to (α_t, λ) is characterised.

Perspectives. The most direct continuations follow from the limitations. Re-measuring all timings uniformly and repartitioning at the level of the underlying simulations are prerequisites to any quantitative claim beyond the orderings established here. Beyond that, three directions seem to us the most promising.

The first is to transport the offline criterion to the family that currently lacks it. LLAE has learnable poles, driven by reconstruction in Phase 1 and only weakly by the deployed objective thereafter; the proxy (31) scores a triplet at the surrogate’s error magnitude before any training, but is currently applied only to SLAE. Running the learnable arm with the Phase-2 pole learning rate set to zero, and again at the surrogate’s own rate, would separate the criterion from the schedule, which our runs do not. A learnable contour trained **through** a surrogate-error term, or simply initialised at the offline spatial optimum, would test whether the contour’s benefit is intrinsic to the criterion, as our reading holds, or specific to the SLAE pipeline. Relatedly, the interpretation

we offer for the latent learnable contour’s failure to transfer, namely that a widened spectral window makes $\mu \mapsto \hat{z}(s_k)$ less regular, is stated as a hypothesis; measuring that regularity independently of the surrogate would settle it.

The second is to exploit the resolution-freedom the representation already possesses. Because the trajectory is materialised by a linear solve on requested nodes, non-uniform time grids, dense during the transient and sparse in the tail, come at no additional cost and are exactly what a many-query workflow wants. Whether the same coefficients support extrapolation beyond the training horizon T , where the decay axis γ of the Laplace transform has a natural interpretation that Fourier lacks, is the more ambitious version of the same question and the one that would most distinguish the transform from a truncated DFT.

The third is compositional. The Laplace stage acts on time and is orthogonal to every spatial-compression choice in the DL-ROM literature, so it could be layered onto POD-DL-ROM [6] or onto a parameter-conditioned U-Net [15] without modification, and the SLAE variant, the one whose compression reaches the decoder, is where such a combination would pay most. Scaling to three-dimensional fields, to several coupled species, and to higher-dimensional parameter spaces is the necessary test of whether the accuracy cost of spectral truncation, 0.4 points here for LLAE, remains as modest when the transient structure is richer.

5 Conclusion

We have introduced the Laplace transform as a temporal compression stage within the DL-ROM framework, predicting $K \ll N_t$ complex spectral coefficients from a parameter vector and recovering the full trajectory through a differentiable Tikhonov inversion, and we have studied two instantiations differing only in where the transform is applied: in the physical domain (SLAE) or in the learned latent space (LLAE). Against a linear POD surrogate in the same Laplace domain and a Laplace-free DL-ROM ablation, on a parametric transient combustion problem with $N_t = 150$, LLAE reaches 3.84% median relative L^2 error and SLAE 5.04% with an optimised contour, against 8.92% for POD and 3.80% for the DL-ROM.

What temporal compression changes is what a fixed accuracy costs, and the benefit is confined to the axes the compressed representation actually reaches: applied in latent space it shrinks the surrogate alone, leaving the arithmetic of the forward pass untouched; applied in the physical domain it reaches the decoder and converts into $2.0\times$ latency and $3.9\times$ memory at identical output. It also changes what a trained model can be asked for: because the predicted coefficients refer to no time grid, one checkpoint answers at any output resolution above the sampling floor set by K , where the DL-ROM required one training per grid across our sweep.

The second contribution is to treat the regularized inversion, the poles and the regularisation weights jointly rather than the contour alone, as an object to be designed rather than fixed by convention. The bias–amplification bound (25) shows that everything it does to the reconstructed trajectory passes through two scalars that depend on the triplet (s_k, α_t, λ) and on nothing else, which makes a candidate triplet scorable without training any network. Acting on it improves the deployed error at every K , by up to 1.85 pp, for a few hundred gradient steps on a purely linear-algebraic functional and at no inference cost, while, instructively, degrading the reconstruction floor by as much as 7.12 pp. That

sign reversal is the methodological point we would most like to carry beyond this paper: an inversion, and more generally a representation, must be judged at the magnitude of error the deployed model actually incurs, not at the floor it permits. The same criterion applied to reconstruction alone selects the best autoencoder of our study and improves nothing that is delivered.

Appendices

A Per-Checkpoint Error Distributions

POD reconstruction. Figure 19 shows the distribution of relative L^2 errors for each of the 7 POD reconstruction configurations evaluated in Section 3.2.1.

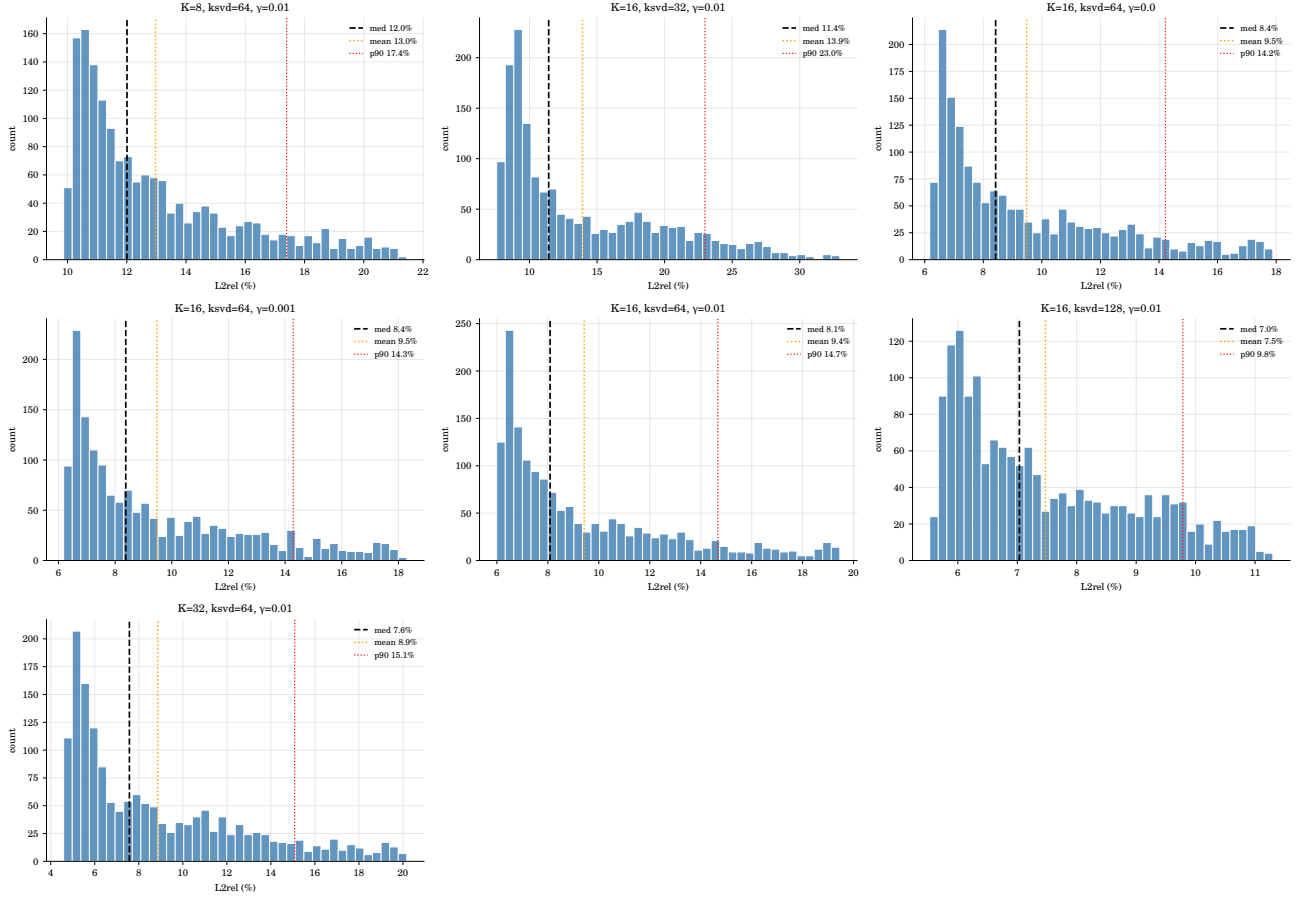


Figure 19: Per-configuration error histograms for the POD reconstruction (test set, $n = 1620$). The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

POD surrogate. Figure 20 shows the corresponding distributions for the end-to-end POD surrogate (Section 3.2.1).

AE checkpoints. Figure 21 shows the distribution of relative L^2 errors across the 1620 test simulations for each of the 14 AE checkpoints evaluated in Section 3.3. Dashed lines indicate the median (black) and mean (orange); the dotted red line marks the 90th percentile.

Surrogate checkpoints. Figure 22 shows the per-sample error distribution for each of the 14 direct surrogate checkpoints evaluated in Section 3.4.1.

DL-ROM baseline. Figure 23 shows the corresponding distribution for the Laplace-free DL-ROM baseline of Section 3.2.2.

Optimised contours. Figure 24 shows the end-to-end error distributions of the six optimised-contour configurations of Section 3.5, as a function of K .

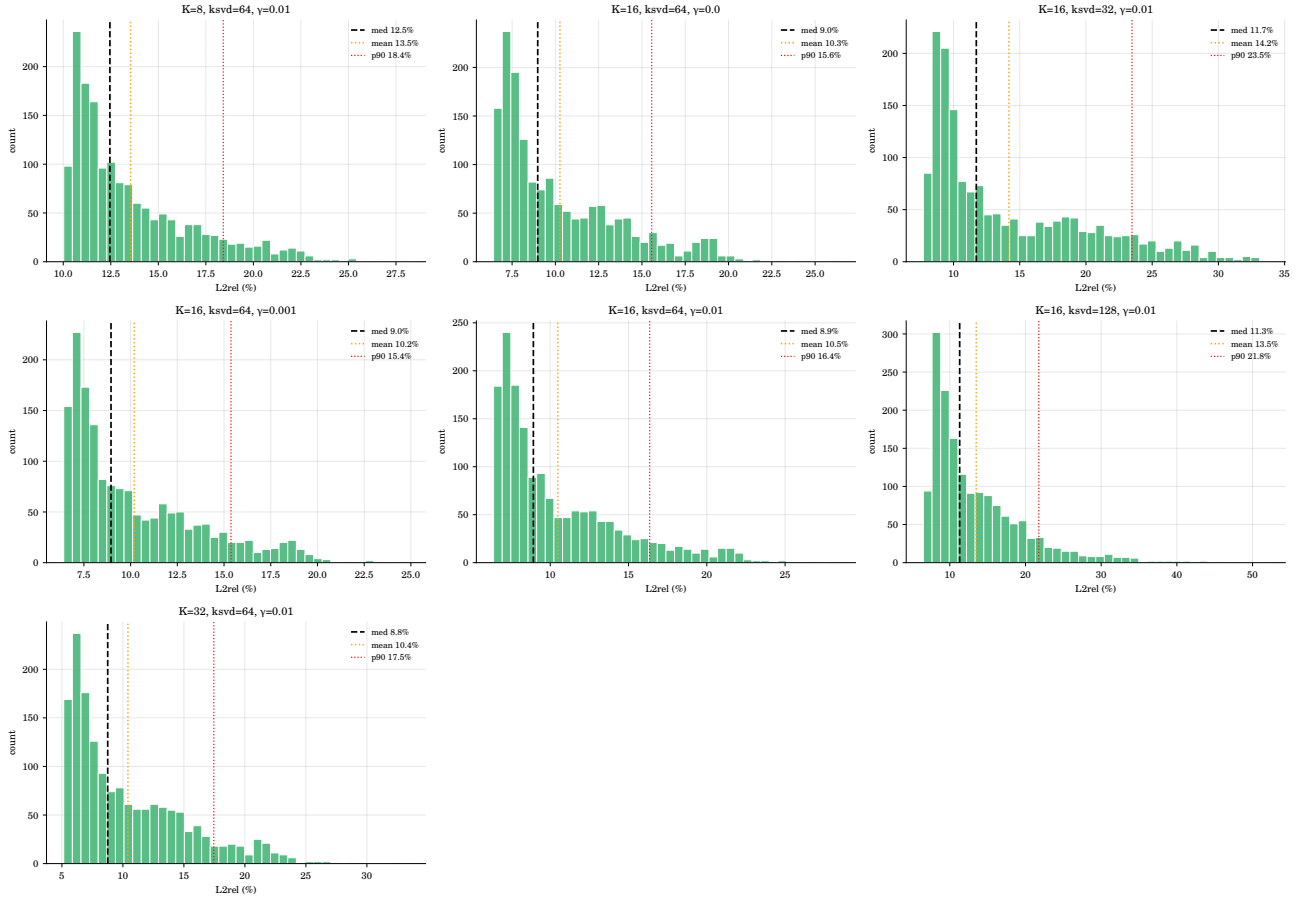


Figure 20: Per-configuration error histograms for the POD surrogate (test set, $n = 1620$). Same layout as Figure 19; green bars indicate surrogate predictions.

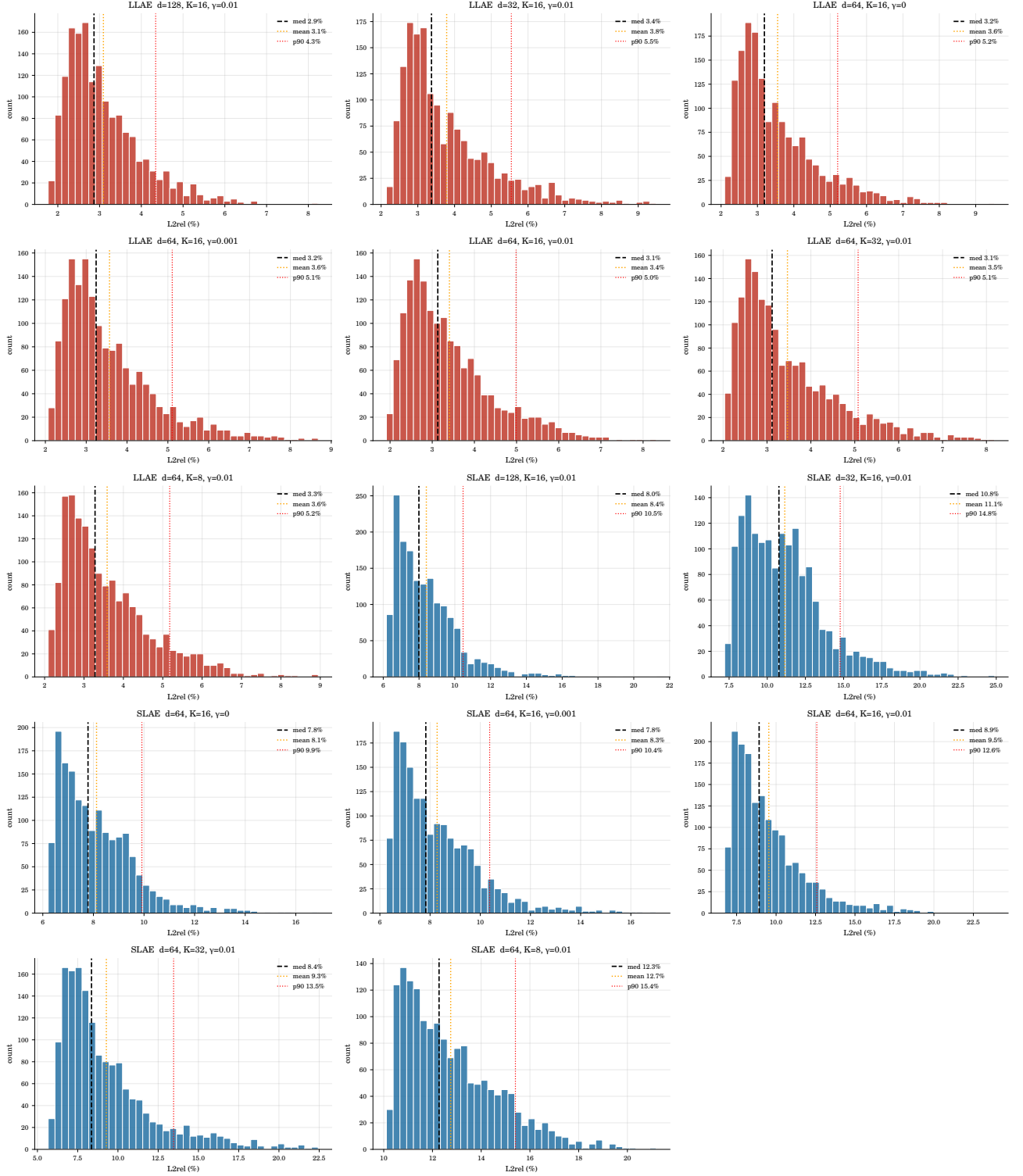


Figure 21: Per-checkpoint error histograms on the test set ($n = 1620$). Each panel corresponds to one checkpoint; SLAE in blue, LLAE in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

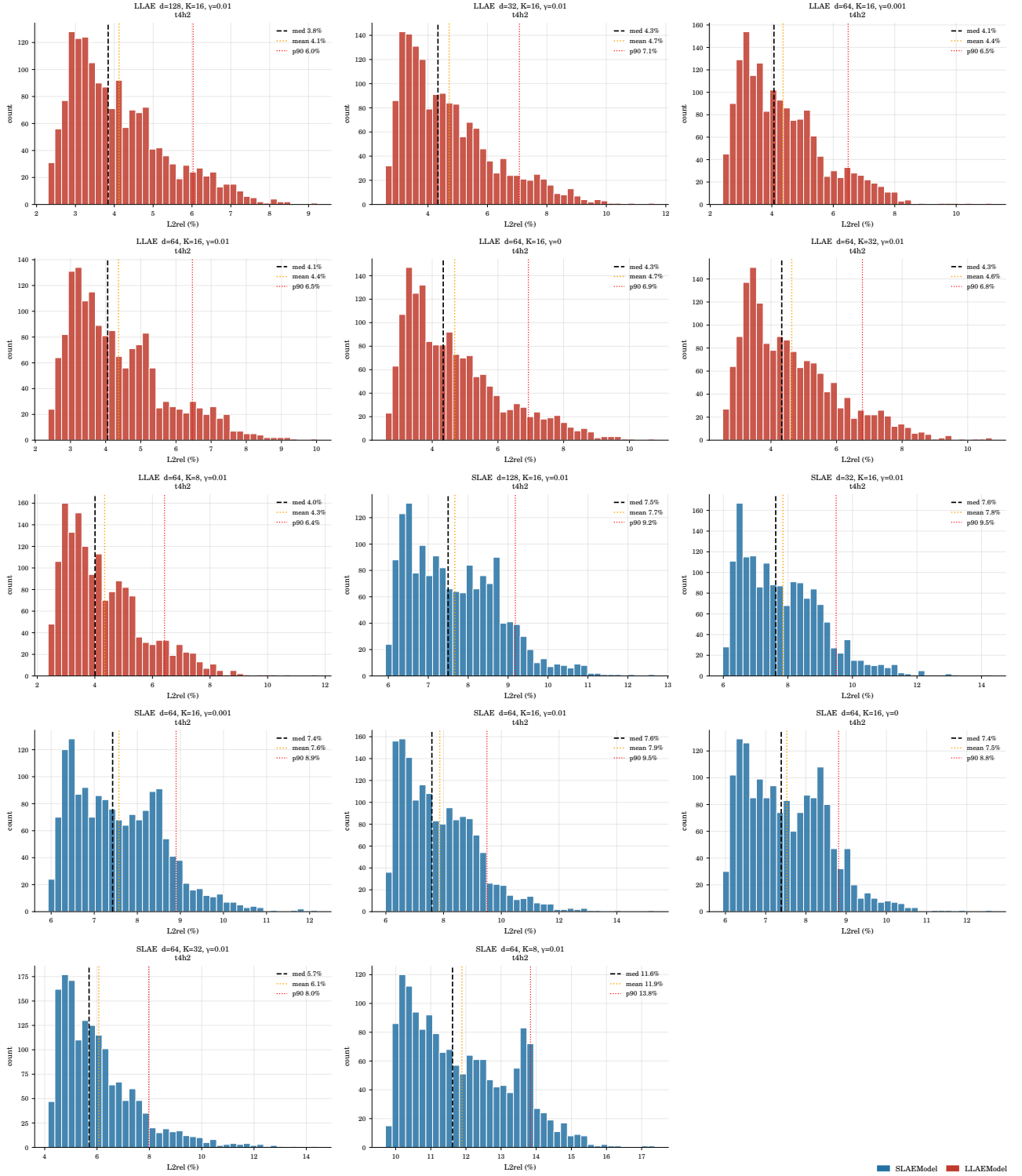


Figure 22: Per-checkpoint error histograms for all 14 direct surrogate models (test set, $n = 1620$). SLAE checkpoints in blue, LLAE checkpoints in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

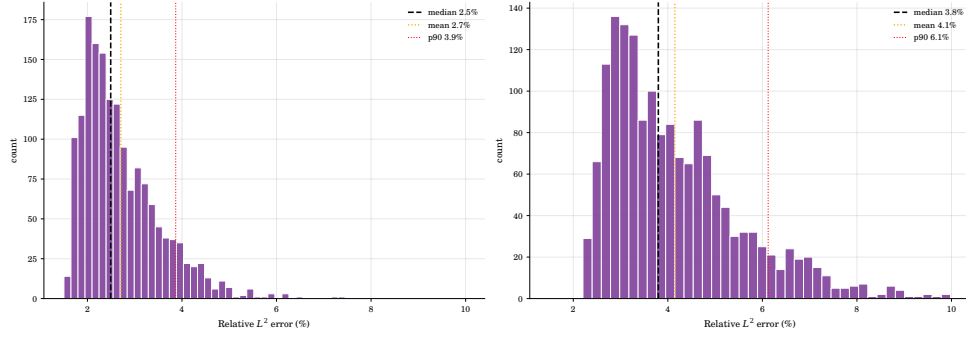


Figure 23: DL-ROM baseline at full temporal resolution ($N_t = 150$, $d_z = 64$): distribution of the relative L^2 error on the physical field $U(t)$ over the test set ($n = 1620$). Median 3.80%, mean 4.15%, p90 6.12%.

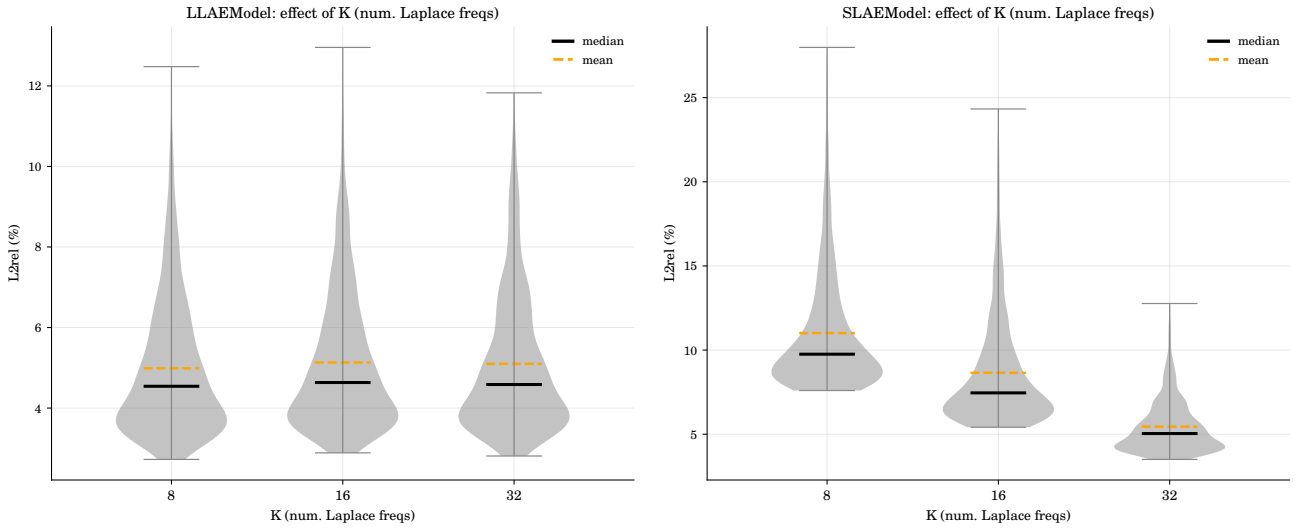


Figure 24: End-to-end error distributions under the optimised contour, as a function of the number of Laplace frequencies K ($d_z = 64$, test set, $n = 1620$). Left: LLAEM, latent learnable contour. Right: SLAEM, offline spatial contour. Black line: median; dashed orange line: mean.

Tucker surrogate checkpoints. Figure 25 shows the per-sample error distribution for each of the 12 Tucker surrogate checkpoints evaluated in Section 3.4.3.

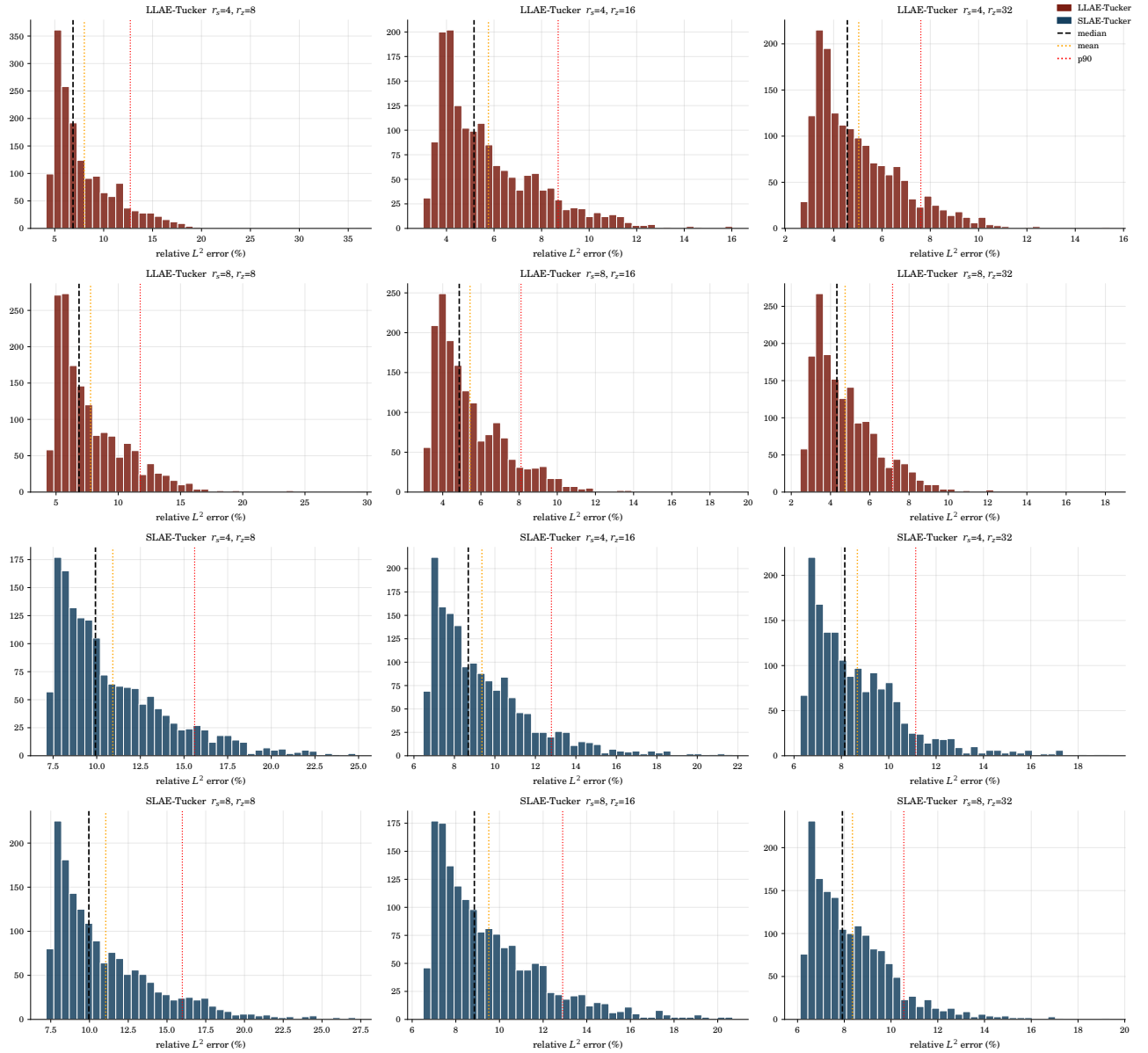


Figure 25: Per-checkpoint error histograms for all 12 Tucker surrogate models (test set, $n = 1620$). SLAE-Tucker in blue, LLAE-Tucker in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

B POD Baseline: Full Ablations

This appendix collects the complete hyperparameter sweeps behind Section 3.2.1, for both the pure POD reconstruction and the end-to-end POD surrogate.

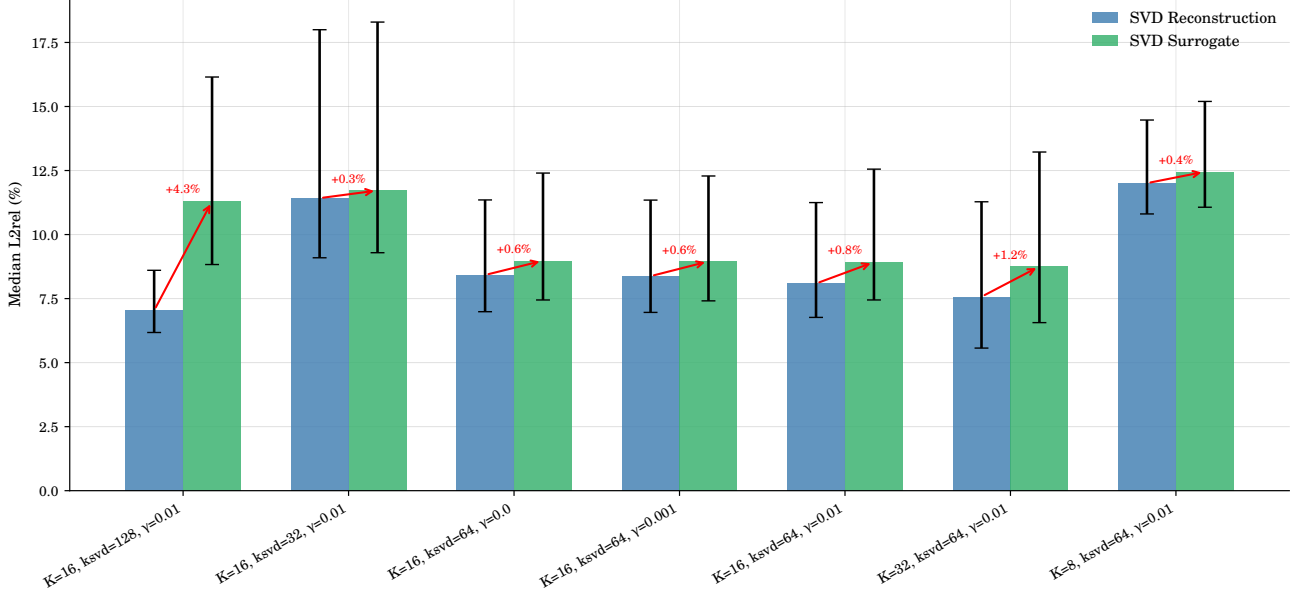


Figure 26: POD reconstruction (blue) vs. end-to-end surrogate (green): median relative L^2 error for all configurations of Table 2 (median $\pm$ IQR). Arrows indicate the surrogate overhead.

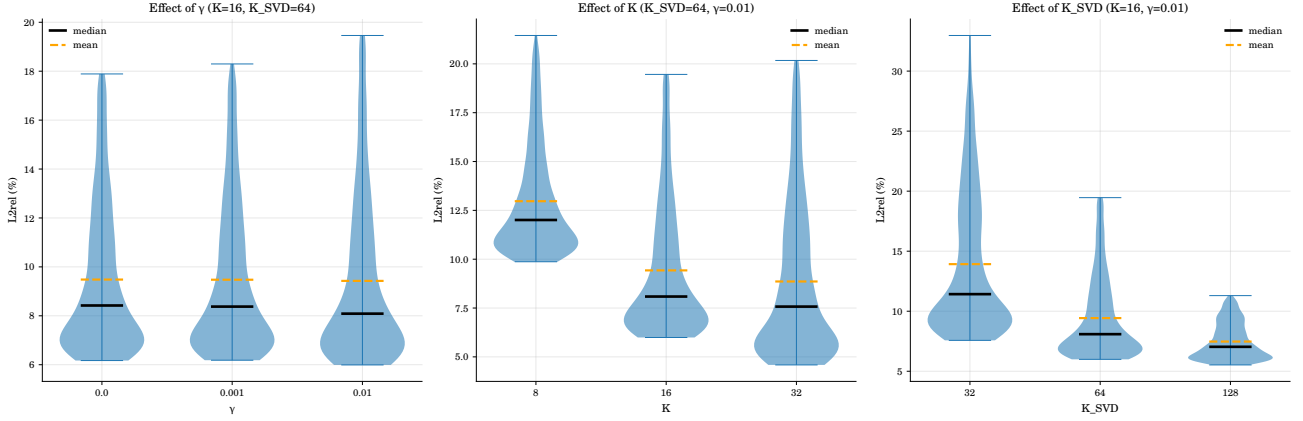


Figure 27: POD reconstruction ablation (test set, median $\pm$ IQR, violin plots; black line = median, dashed orange = mean). From left to right: effect of the damping parameter γ ($K = 16, k_{pod} = 64$), effect of the number of Laplace frequencies K ($k_{pod} = 64, \gamma = 0.01$), and effect of the number of POD modes k_{pod} ($K = 16, \gamma = 0.01$).

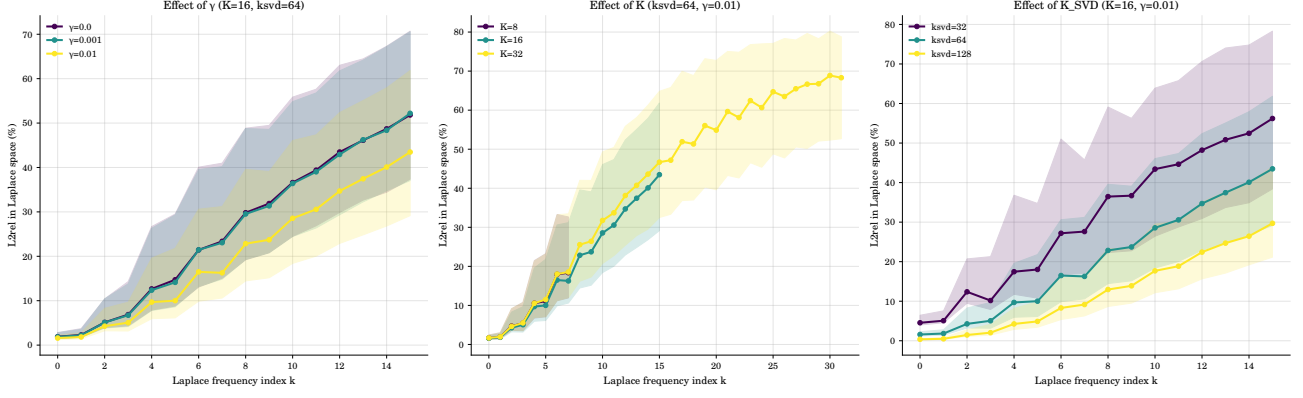


Figure 28: POD reconstruction: per-frequency Laplace-domain error (median $\pm$ IQR, test set). Left: effect of γ ; centre: effect of K ; right: effect of k_{pod} .

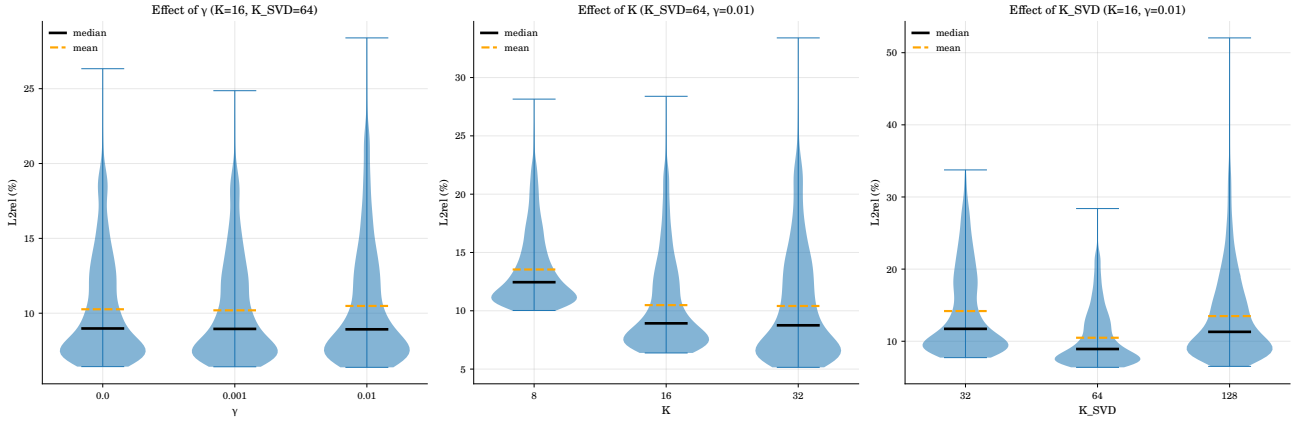


Figure 29: POD surrogate ablation (test set, median $\pm$ IQR). From left to right: effect of γ ($K = 16$, $k_{\text{pod}} = 64$), effect of K ($k_{\text{pod}} = 64$, $\gamma = 0.01$), and effect of k_{pod} ($K = 16$, $\gamma = 0.01$). Black line = median, dashed orange = mean.

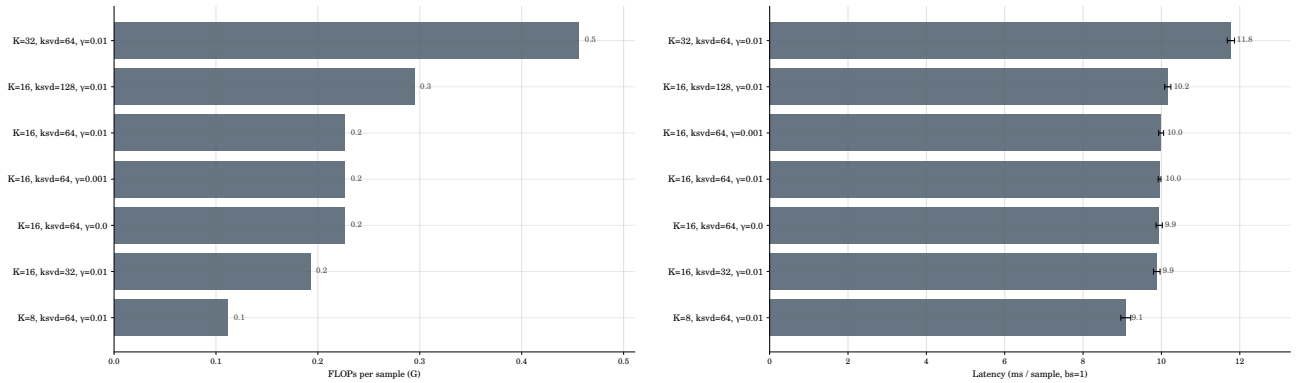


Figure 30: POD surrogate inference benchmark (50 timed CUDA Event runs, 10 warm-up runs). Left: per-sample latency at batch size 1, sorted by latency. Right: throughput (samples/s) vs. k_{pod} at batch size 32.

C Autoencoder Ablations and Reconstruction Cost

Figures 31 and 32 show the effect of each hyperparameter on the reconstruction error of Section 3.3, varying one at a time while holding the others fixed. Figure 33 gives the corresponding inference cost across all 14 checkpoints, of which Table 5 lists 10.

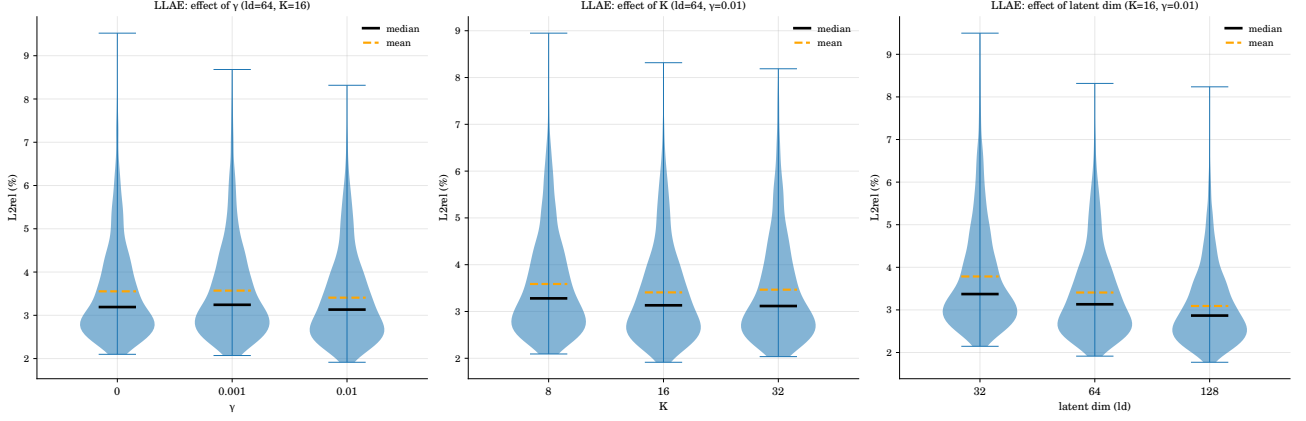


Figure 31: LLAE ablation study. From left to right: effect of latent dimension d_z ($K = 16$, $\gamma = 0.01$), number of Laplace frequencies K ($d_z = 64$, $\gamma = 0.01$), and damping parameter γ ($d_z = 64$, $K = 16$). Violin plots show the full error distribution; black line is the median, dashed orange line is the mean.

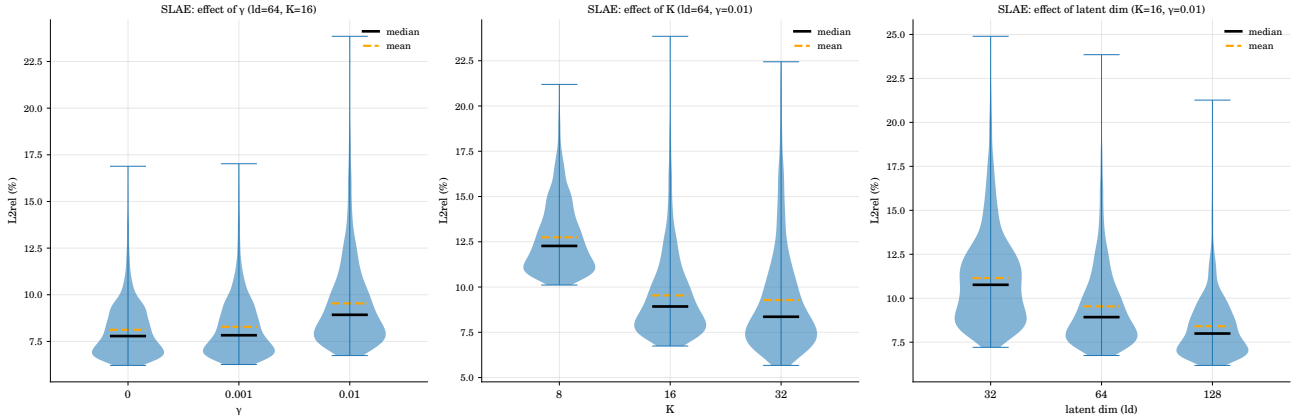


Figure 32: SLAE ablation study. Same layout as Figure 31 (fixed parameters: $K = 16$, $\gamma = 0.01$ / $d_z = 64$, $\gamma = 0.01$ / $d_z = 64$, $K = 16$). Note the larger y-axis range compared to LLAE.

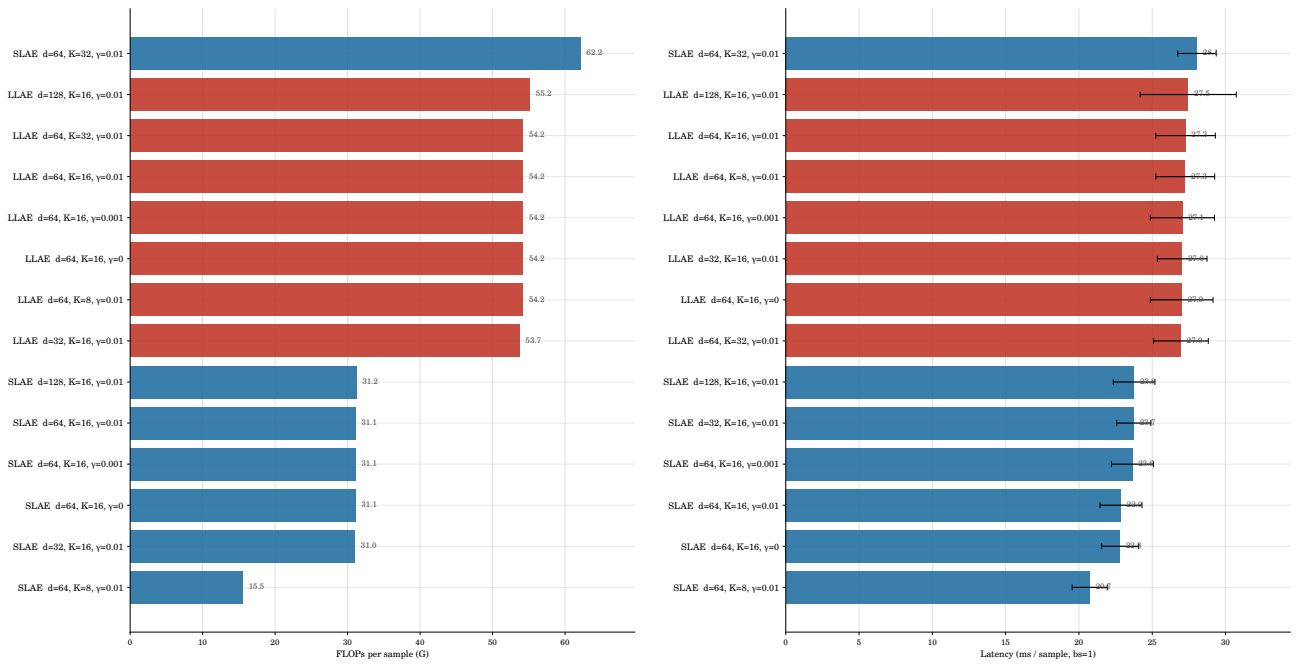


Figure 33: Reconstruction cost for all 14 AE checkpoints (batch size 1). Left: arithmetic work per sample. Right: wall-clock latency. LLAE in red, SLAE in blue.

D Surrogate Ablation Studies

Figures 34 and 35 show the effect of each hyperparameter on the end-to-end surrogate error, using violin plots of the full test-set distribution (one hyperparameter varied at a time, others fixed at the baseline values of Table 14).

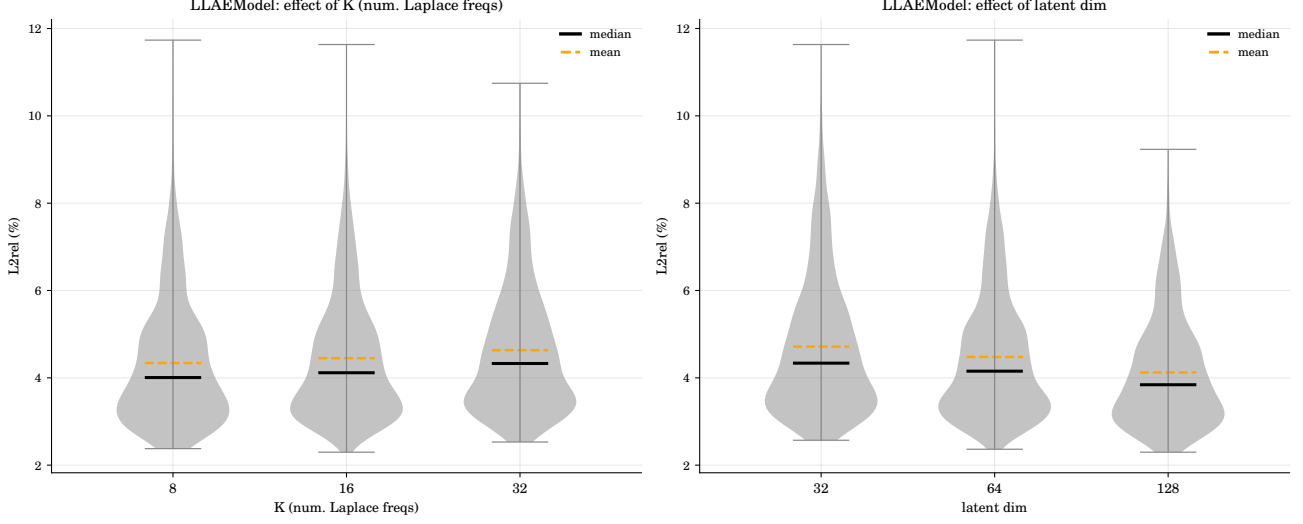


Figure 34: LLAE surrogate ablation (test set, $n = 1620$, violin plots; black line = median, dashed orange = mean). From left to right: effect of the number of Laplace frequencies K ($d_z = 64$, $\gamma = 0.01$) and effect of the latent dimension d_z ($K = 16$, $\gamma = 0.01$).

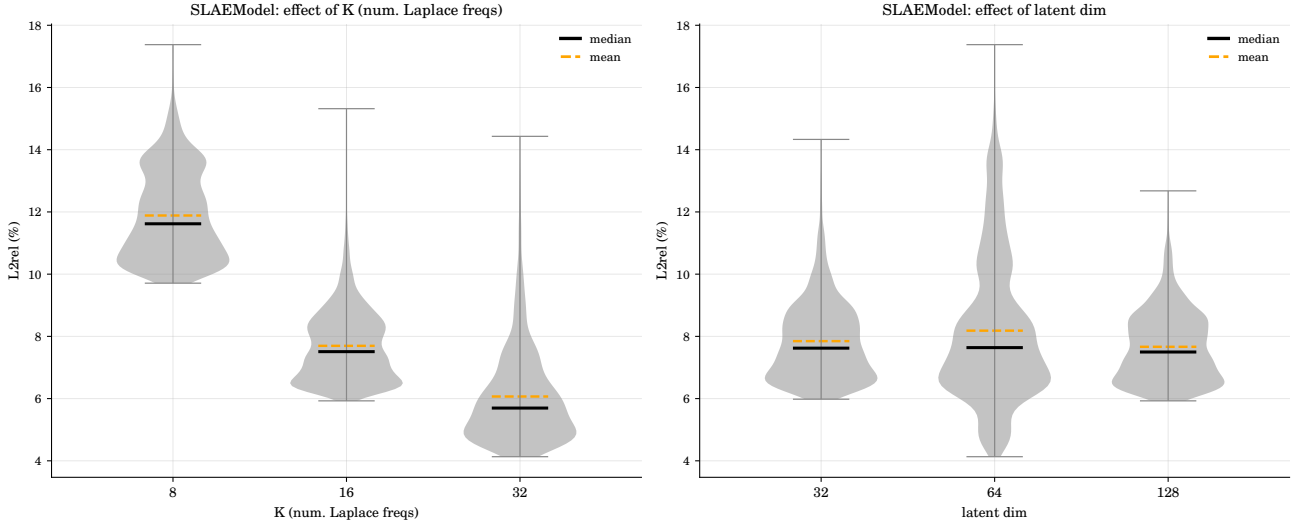


Figure 35: SLAE surrogate ablation (test set, $n = 1620$). Same layout as Figure 34. Note the wider y-axis range.

E Optimised Contours at the Reconstruction Stage

Section 3.5 assesses the two contour-selection strategies at the surrogate stage, which is the deliverable. This appendix reports their behaviour at the reconstruction stage, where the two criteria diverge most visibly. Figure 37 summarises the six configurations.

The latent learnable contour is what contour optimisation amounts to for LLAE, whose poles are differentiable end to end and call for no offline criterion (Section 2.4). At the reconstruction stage it works, and well. The most accurate LLAE autoencoder of the study is obtained at $K = 32$ (2.60% median error), and even $K = 8$ (2.89%) improves upon every prescribed-contour instance of Table 4, including the one at $d_z = 128$ (2.87%): at this stage, optimising the contour is worth more than doubling the latent dimension, at no additional inference cost since the sampling points enter only the transform. The residual sensitivity to K all but vanishes (2.60–2.89% over a fourfold variation), consistent with the geometry of Figure 36, where the sampling points are redistributed towards the spectral region carrying the signal energy; the gain is measured on the held-out set, so nothing here is fitting noise.

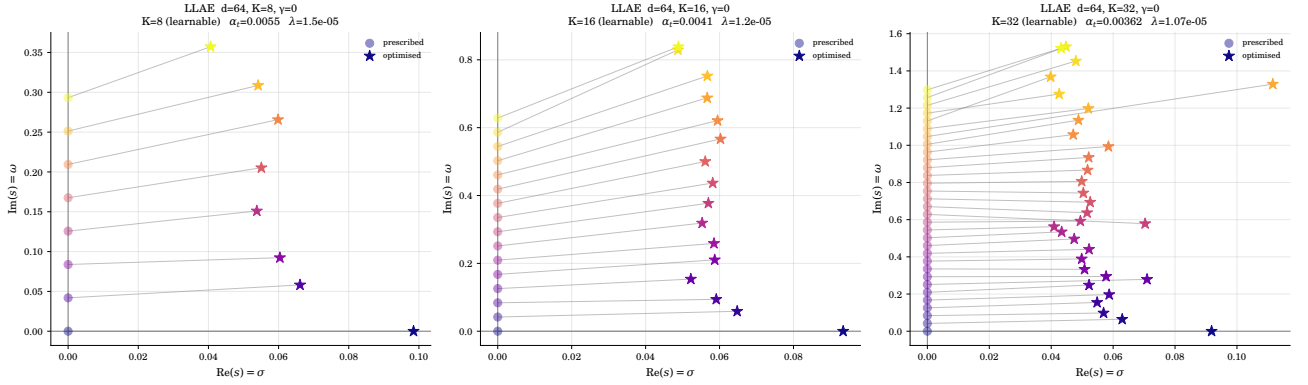


Figure 36: LLAE latent learnable contours in the complex plane ($d_z = 64$, $K \in \{8, 16, 32\}$). Same conventions as Figure 15: circles are the prescribed sampling points, stars the learned ones, joined by a grey segment; colour encodes the frequency index k .

We did not succeed in turning that autoencoder into a better surrogate: the deployed columns of Table 10 run the other way. We draw no further conclusion from the comparison, the two strategies differing in more than their criterion, the learnable poles continuing to be refined during Phase 2 while the offline triplet is frozen after selection.

The offline spatial contour degrades the reconstruction instead, by +0.65 and +0.17 pp at $K = 8$ and $K = 32$ and by +7.12 pp at $K = 16$. The penalty is analysed in Section 3.5: it is incurred below the crossover error ϵ^* of the bound (25), and is subsequently absorbed by the decoder during Phase 2.

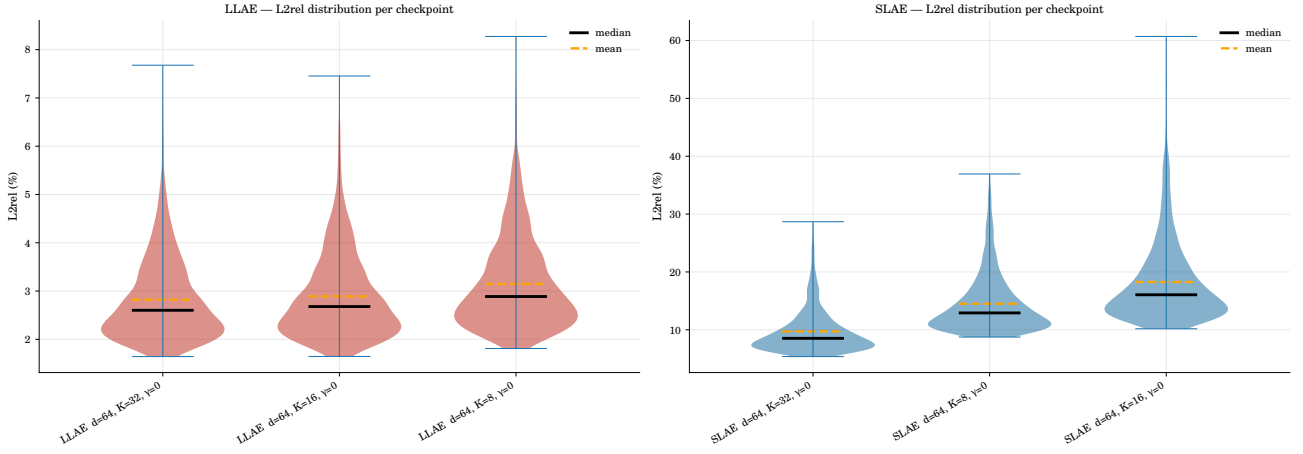


Figure 37: Standalone reconstruction error under the optimised contours: distribution of the relative L^2 error on the test set ($n = 1620$), one panel per family, configurations ordered by median. Black line: median; dashed orange line: mean. Left: LLAE, latent learnable contour. Right: SLAE, offline spatial contour, on a y-axis an order of magnitude wider.

F Governing Equations

Our neural surrogate model is designed to emulate the transient dynamics of a reacting flow system. The physical system is governed by a set of coupled partial differential equations representing the conservation of mass, momentum, energy, and chemical species.

1. Fluid Mechanics (Momentum):

$$(43) \quad \frac{\partial(\rho \mathbf{U})}{\partial t} + \nabla \cdot (\rho \mathbf{U} \mathbf{U}) - \nabla \cdot \boldsymbol{\tau} = -\nabla p + \rho \mathbf{g}$$

2. Chemical Species Transport:

$$(44) \quad \frac{\partial(\rho Y_i)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y_i) - \nabla \cdot (\mu_{\text{eff}} \nabla Y_i) = \dot{\omega}_{i,\text{eff}}$$

under the constraint $\sum Y_i = 1$.

3. Energy:

$$(45) \quad \frac{\partial(\rho h)}{\partial t} + \nabla \cdot (\rho \mathbf{U} h) + \frac{\partial(\rho K)}{\partial t} + \nabla \cdot (\rho \mathbf{U} K) - \frac{\partial p}{\partial t} - \nabla \cdot (\alpha_{\text{eff}} \nabla h) = \dot{q}_{\text{chem}}$$

where $K = \frac{1}{2}|\mathbf{U}|^2$ is the kinetic energy.

4. Combustion Model (PaSR): The effective reaction rate is modulated by the Partially Stirred Reactor model:

$$(46) \quad \dot{\omega}_{i,\text{eff}} = \kappa \dot{\omega}_i(c^*, T^*), \quad \text{with} \quad \kappa = \frac{\tau_c}{\tau_c + \tau_{\text{mix}}}$$

where τ_c and τ_{mix} are the characteristic chemical and turbulent mixing times, respectively.

5. Equation of State: The ideal gas law closes the system:

$$(47) \quad p = \rho R T \quad \text{or} \quad \rho = \psi p \quad \text{with} \quad \psi = \frac{1}{RT}$$

G Architectural Details and Setup

Hardware and software. Every model of this study was trained, and every cost figure measured, on identically configured workstations of the same pool: an Intel Xeon W-1270P CPU (8 cores, 16 threads) with 62 GB of RAM and a single NVIDIA RTX A5000 GPU (24 GB). The software stack is Python 3.13, PyTorch 2.12 (CUDA 13.0) and PyTorch Lightning 2.6. All inference-cost figures reported in this paper (latency, FLOPs, peak GPU memory and energy) were collected on one such idle machine, at batch size 1 unless stated otherwise, with 50 timed CUDA Event runs after 10 warm-up runs; the device is not named again in the individual captions.

Optimisation protocol. Both phases share the same protocol, and it is the one written in the configuration files distributed with the code; the only departures are the sweep overrides named in the tables below. Optimiser AdamW with weight decay 10^{-4} ; learning-rate schedule `ReduceLRonPlateau` (factor 0.5, patience 15, floor 10^{-6}) driven by the monitored validation quantity; mixed 16-bit precision; gradient-norm clipping at 1.0; at most 300 epochs with early stopping at patience 40. The monitored quantity is the validation loss for the autoencoders and the validation relative L^2 error for the surrogates, and it also selects the checkpoint that is subsequently evaluated. The global seed is 42 for every run.

Data. The partition is the one described in Section 3.1: 5,184 training, 1,296 validation and 1,620 test instances, drawn by a fixed permutation of seed 42 and identical across every model, phase and hyperparameter configuration of this study. No test instance enters any training or selection decision at any point. Fields are standardised pixel-wise using training-set statistics; for SLAE the Laplace frames are standardised per frequency and per pixel, and the compressed targets of the SVD and Tucker variants are standardised as well, so all losses below are evaluated in normalised space. The forward quadrature uses the trapezoidal rule on the $N_t = 150$ nodes of the $\Delta t = 1$ s grid.

Offline contour minimisation. The SLAE contours of Section 3.5 are obtained by 500 AdamW steps on the proxy (31) (learning rate 5×10^{-3} on plateau decay, weight decay 10^{-4} , gradient clipping at 1.0, double precision), with $\lambda_{\text{AE}} = 1.25$, $\lambda_{\Delta t} = \lambda_{\Delta x} = 0.5$ and rank $d_z = 64$ for the SVD residual, both terms being evaluated over the same $N_{\text{opt}} = 100$ training instances sampled at a single rotation angle. The poles are initialised on the prescribed Bromwich contour at $\gamma = 0$ and (α_t, λ) at 10^{-3} ; all four are optimised jointly under the box constraints $\sigma_k \geq -0.05$, $\omega_k \geq 0$ and $\alpha_t, \lambda \in [10^{-6}, 1]$.

Table 14 summarises the autoencoders (Phase 1) and Table 15 the surrogates (Phase 2).

Group	Hyperparameter	SLAE	LLAE
Architecture (baseline [†])	Latent dim d_z	64	64
	Frequencies K	16	16
	Contour γ	0.01	0.01
	Conditioning coord.	$k/(K-1)$	$t/(N_t-1)$
	Sinusoidal levels L	8	8
Backbone	Grid $N \times N$	128^2	128^2
	Input channels	2 (Re, Im)	1
	Encoder channels	$16 \rightarrow 32 \rightarrow 64$, then FC $\rightarrow 2d_z \rightarrow d_z$	
	Decoder channels	$128 \rightarrow 128 \rightarrow 64 \rightarrow 32$	$64 \rightarrow 64 \rightarrow 32 \rightarrow 16$
	Decoder norm	BatchNorm	GroupNorm(8)
	Decoder output heads	2 (Re, Im)	1
Regularization	Ridge β on z	10^{-2}	10^{-2}
	Latent roundtrip β_{latent}	—	0.1
	Tikhonov α_t	7×10^{-3} (eval)	7×10^{-3}
	Tikhonov λ	3×10^{-5} (eval)	3×10^{-5}
Training	Epochs (max)	300	300
	Batch size	$16^\ddagger$	16
	Learning rate	5×10^{-4}	5×10^{-4}
	Weight decay	10^{-4}	10^{-4}
	Early stopping patience	40	40
	Monitored quantity	validation reconstruction loss	

Table 14: Autoencoder hyperparameters (Phase 1), as set in `configs/model` and `configs/training/ae.yaml`. FiLM modulation is $x \mapsto x(1 + \tanh \alpha) + \delta$ at every convolutional stage, the $\tanh$ bounding the gain in $(0, 2)$; the last convolution of each decoder head is zero-initialised. [†]Baseline values held fixed when the other architectural parameters are swept: $d_z \in \{32, 64, 128\}$, $K \in \{8, 16, 32\}$, $\gamma \in \{0, 0.001, 0.01\}$. [‡]The batching unit differs between families: a batch of 16 groups 16 (simulation, frequency) pairs for SLAE, which encodes frequency frames independently, against 16 complete simulations for LLAE, which encodes the whole trajectory.

Group	Hyperparameter	SLAE	LLAE
Architecture (fixed)	Trunk layers n_{trunk}	4	4
	Trunk hidden dim	512	512
	Head layers n_{head}	2	2
	Head hidden dim	256	256
	Frequency encoding levels	$L = 6$, raw sin/cos (no MLP)	
Loss	Latent weight α_{lat}	1.0	1.0
	Spatial weight α_{spat}	1.0	1.0
	Tikhonov α_t	7×10^{-3}	7×10^{-3}
	Tikhonov λ	3×10^{-5}	3×10^{-5}
Trainable parts	Encoder	frozen	
	Surrogate MLP	3×10^{-4} direct, 5×10^{-4} SVD/Tucker	
	Decoder (fine-tuned)	$lr = 5 \times 10^{-5}$	
	Poles (s_k, α_t, λ)	frozen	10^{-5} if learnable
Compression variants	SVD rank k_{svd}	$\{8, 16, 32\}$, basis frozen	
	Tucker ranks (r_s, r_z)	$\{4, 8, 16\} \times \{8, 16, 32\}$, frozen	
	HOOI iterations	5	
Training	Epochs (max)	300	300
	Batch size	16	16
	Weight decay	10^{-4}	10^{-4}
	Early stopping patience	40	40
	Monitored quantity	validation relative L^2 error	

Table 15: Surrogate hyperparameters (Phase 2), as set in `configs/training/surrogate_*.yaml`. One surrogate is trained per autoencoder checkpoint, inheriting its d_z , K and contour. The encoder is frozen throughout; the decoder is fine-tuned at a reduced learning rate, and the SVD basis and Tucker factors, computed offline, are held fixed. The DL-ROM baseline uses this same table with $N_t = 150$ per-time-step heads in place of the K frequency heads.

H Derivations for the Bias–Amplification Contour Optimization

Throughout this appendix, $\mathbf{F} \in \mathbb{C}^{K_{\text{full}} \times N_t}$ and the coefficient vectors are those assembled on the conjugate-symmetric set $\mathcal{S}$ of (2), as in Sections 2.1 and 2.4; on $\mathcal{S}$ the matrix $\mathbf{F}^H \mathbf{F}$ and the vector $\mathbf{F}^H \hat{\mathbf{u}}$ are real, so the operator $\Re(\cdot)$ below may be read as an identity and is kept only for uniformity with (4).

Proposition H.1 (Invertibility of $\mathbf{A}$). *Since $\lambda > 0$ and $\mathbf{D}_t^T \mathbf{D}_t$ is positive semi-definite, $\mathbf{L}_{\text{reg}} = \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I}$ is positive definite ($\mathbf{L}_{\text{reg}} \succeq \lambda \mathbf{I}$). Thus $\mathbf{A} = \Re(\mathbf{F}^H \mathbf{F}) + \mathbf{L}_{\text{reg}}$ is symmetric positive definite and hence invertible. The unique reconstructed solution is $\mathbf{u}^*(\boldsymbol{\mu}) = \mathbf{A}^{-1} \Re(\mathbf{F}^H \hat{\mathbf{u}}(\boldsymbol{\mu}))$.*

Derivation of the bound (25). Starting from decomposition (23) and applying the triangle inequality:

$$\|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\| \leq \|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu})\| + \|\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\|.$$

The first term equals $\|\mathbf{A}^{-1} \mathbf{L}_{\text{reg}} \mathbf{u}(\boldsymbol{\mu})\|$ by (24). For the second term:

$$\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) = \mathbf{A}^{-1} \Re\left(\mathbf{F}^H (\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu}))\right).$$

Applying sub-multiplicativity of the operator norm and $\|\Re(M)\| \leq \|M\|$ yields:

$$\|\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\| \leq \|\mathbf{A}^{-1} \mathbf{F}^H\| \cdot \|\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu})\|_{\mathbb{C}^{K_{\text{full}}}},$$

which gives (25). $\square$

On the norm used in practice. The derivation above uses the operator 2-norm, for which sub-multiplicativity holds. The implementation evaluates the amplification factor as the Frobenius norm of the real $2N_t \times K_{\text{full}}$ stack $[\mathbf{A}^{-1} \Re(\mathbf{F}^H); \mathbf{A}^{-1} \Im(\mathbf{F}^H)]$, which is exactly $\|\mathbf{A}^{-1} \mathbf{F}^H\|_F$. Since $\|\cdot\|_2 \leq \|\cdot\|_F$, substituting it in (25) and in the proxy (31) preserves the inequality; it is preferred because it is a smooth function of every entry, hence better behaved under gradient descent than the largest singular value.

Proposition H.2 (SVD residual as a floor for any parameter-to-coefficient predictor). *Let $\mathcal{P}_{d_z}$ be the class of maps $h : \boldsymbol{\mu} \mapsto \mathbf{D} g(\boldsymbol{\mu})$ with $\mathbf{D} \in \mathbb{R}^{2N_t K \times d_z}$ and $g : \mathbb{R}^p \rightarrow \mathbb{R}^{d_z}$ arbitrary. Then, over the optimisation sample $(\boldsymbol{\mu}_i)_{i \leq N_{\text{opt}}}$,*

$$\min_{h \in \mathcal{P}_{d_z}} \frac{1}{N_{\text{opt}}} \sum_{i=1}^{N_{\text{opt}}} \|\mathbf{a}(\boldsymbol{\mu}_i) - h(\boldsymbol{\mu}_i)\|^2 = \mathcal{E}_{\text{SVD}}^2,$$

the minimum being attained.

Derivation. The image of any $h \in \mathcal{P}_{d_z}$ lies in $V = \text{col}(\mathbf{D})$, a subspace of dimension at most d_z . For any such V and any $\mathbf{v}_i \in V$, $\|\mathbf{a}(\boldsymbol{\mu}_i) - \mathbf{v}_i\| \geq \|\mathbf{a}(\boldsymbol{\mu}_i) - P_V \mathbf{a}(\boldsymbol{\mu}_i)\|$, so the mean squared error is bounded below by $\min_{\dim V \leq d_z} \frac{1}{N_{\text{opt}}} \sum_i \|\mathbf{a}(\boldsymbol{\mu}_i) - P_V \mathbf{a}(\boldsymbol{\mu}_i)\|^2$, which equals $\mathcal{E}_{\text{SVD}}^2$ by Eckart–Young. Equality is attained by taking $\mathbf{D}$ a basis of the optimal subspace V^* and $g(\boldsymbol{\mu}_i)$ the coordinates of $P_{V^*} \mathbf{a}(\boldsymbol{\mu}_i)$. $\square$

What the statement does and does not assume. Nothing is assumed of g : no regularity, no trainability, not even measurability, and g may interpolate the sample exactly. The floor is therefore a property of the **contour** and of the bottleneck width, not of any learning procedure, which is what makes $\mathcal{E}_{\text{SVD}}$ usable as a score over candidate triplets.

Conversely the bound is confined to a rank- d_z **linear** decoder reading a single unconditioned basis, with a budget of d_z coefficients for the whole spectrum, whereas SLAE decodes non-linearly, modulates that decoder per frequency through FiLM, and disposes of d_z coefficients at each frequency, Kd_z in total. Weight sharing across frequencies is common to both and is not what separates them; the conditioning and the budget are, and on both SLAE is strictly more expressive, so the floor does not bind it. The per-frequency variant $\sum_k \mathcal{E}_{\text{SVD}}(s_k)^2$ would bound SLAE’s class proper and is the natural alternative; it was not the one used here, the joint form being retained because it depends on the contour through a single spectrum.

Floors bound classes, not trained networks. A network trained by stochastic gradient descent for a finite number of epochs under a particular loss is guaranteed to attain no floor at all: SLAE exceeds the linear reconstruction floor of Section 3.2.1 in four of its seven configurations (up to +0.94 pp). Two effects account for this, both outside the proposition: SLAE is trained on a Laplace-domain objective that is not the physical-field error being reported, so the quantity it minimises is not the quantity the comparison scores, and the attained optimum is a local one. Proposition H.2 is accordingly used in Section 2.4 as a computable, deterministic **score** whose variation with the contour is **assumed** to track that of the surrogate error — an assumption, not a corollary, since the linear rank of the image and the regularity of $\mu \mapsto a(\mu)$ are independent properties in general. The scalar λ_{AE} introduced there does not repair this: it fixes the nominal magnitude $\mathcal{E}_{\text{SVD}}/\lambda_{\text{AE}}$ at which the variance term enters (31), and that magnitude is set a priori rather than measured.

References

- [1] Qianying Cao, Somdatta Goswami, and George Em Karniadakis. LNO: Laplace neural operator for solving differential equations, 2023.
- [2] L. De Lathauwer, B. De Moor, and J. Vandewalle. On the best rank-1 and rank- $(r_1, r_2, \dots, r_n)$ approximation of higher-order tensors. *SIAM Journal on Matrix Analysis and Applications*, 21(4):1324–1342, 2000.
- [3] Mostafa Dehghani, Yi Tay, Anurag Arnab, Lucas Beyer, and Ashish Vaswani. The efficiency misnomer. *International Conference on Learning Representations (ICLR)*, 2022.
- [4] X. Fan, Y. Lin, B. Gong, and H. Li. Offline meteorology-pollution coupling global air pollution forecasting model with bilinear pooling, 2025.
- [5] Stefania Fresca, Luca Dedè, and Andrea Manzoni. A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized pdes. *Journal of Scientific Computing*, 87(2):61, 2021.
- [6] Stefania Fresca and Andrea Manzoni. POD-DL-ROM: Enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. *Computer Methods in Applied Mechanics and Engineering*, 388:114181, 2022.
- [7] Matteo Giacomini and Pedro Díez. Surrogates for physics-based and data-driven modelling of parametric systems, 2026. Preprint. TODO: confirm venue.

- [8] Silke Glas, Antonia Mayerhofer, and Karsten Urban. Two ways to treat time in reduced basis methods. In Peter Benner, Mario Ohlberger, Anthony Patera, Gianluigi Rozza, and Karsten Urban, editors, *Model Reduction of Parametrized Systems*, volume 17 of *MS&A*, pages 113–128. Springer, 2017.
- [9] Goeckner et al. Model order reduction in contour integral methods for parametric PDEs, 2022. Preprint. TODO: complete author list and venue.
- [10] Nicola Guglielmi, María López-Fernández, and Giancarlo Nino. Numerical inverse Laplace transform for convection-diffusion equations, 2020. TODO: confirm venue (Mathematics of Computation, vol. 89?).
- [11] Q. Guo, M. Ren, S. Wu, et al. Applications of artificial intelligence in the field of air pollution: a bibliometric analysis. *Frontiers in Public Health*, 10:933665, 2022.
- [12] N. Halko, P.-G. Martinsson, and J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.
- [13] Jan S Hesthaven, Gianluigi Rozza, and Benjamin Stamm. *Certified Reduced Basis Methods for Parametrized Partial Differential Equations*. Springer, 2015.
- [14] Jan S Hesthaven and Stefano Ubbiali. Non-intrusive reduced order modeling of nonlinear problems using neural networks. *Journal of Computational Physics*, 363:55–78, 2018.
- [15] Michael Urs Lars Kastor, Jan Rottmayer, Anna Hundertmark, and Nicolas Ralph Gauger. Parameter conditioned interpretable U-Net surrogate model for data-driven predictions of convection-diffusion-reaction processes, 2026. Preprint. TODO: confirm venue.
- [16] T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.
- [17] Toni Lassila, Andrea Manzoni, Alfio Quarteroni, and Gianluigi Rozza. Model order reduction in fluid dynamics: Challenges and perspectives. In Alfio Quarteroni and Gianluigi Rozza, editors, *Reduced Order Methods for Modeling and Computational Reduction*, volume 9 of *MS&A*, pages 235–274. Springer, 2014.
- [18] Kookjin Lee and Kevin T Carlberg. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.
- [19] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.08895.
- [20] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.
- [21] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*, 393:114778, 2022.

- [22] Swaroop Mallick and Monika Mittal. AI-based model order reduction techniques: A survey, 2025. Preprint. TODO: confirm venue.
- [23] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020.
- [24] Jad Mounayer, Sebastian Rodriguez, Chady Ghnatios, Charbel Farhat, and Francisco Chinesta. Rank reduction autoencoders, 2025. Preprint. TODO: confirm venue.
- [25] Pranshu Pant, Ruchit Doshi, Pranav Bahl, and Amir Barati Farimani. Deep learning for reduced order modelling and efficient temporal evolution of fluid simulations. *Physics of Fluids*, 33(10), 2021.
- [26] Eric Parish, Masayuki Yano, Irina Tezaur, and Traian Iliescu. Stabilization of CDR ROMs, 2025. Preprint. TODO: confirm venue.
- [27] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [28] Ricardo Reyes. The frequency reduced-basis method: Reduced order models for time-dependent problems using the Laplace transform, 2025. Preprint. TODO: confirm venue.
- [29] Tian Wang and Chuang Wang. Latent Neural Operator for solving forward and inverse PDE problems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.